Universitatea de Științe și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Sistem E-Medicine pentru Analizarea și Monitorizarea Datelor Epidemiologice cu Modele Compartimentale din Familia SIR

Lucrare de dizertație

program de master:
Ingineria Informației și Sisteme de Calcul

Conducător științific
Prof. Radu Rădescu

Absolvent
Geană Tudor-Bogdan

Anul 2026

Declaraţie de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Sistem E-Medicine pentru Analizarea și Monitorizarea*
Datelor Epidemiologice cu Modele Compartimentale
din Familia SIR, prezentată în cadrul Facultăţii de Electronică, Telecomunicaţii şi Tehnologia Informaţiei a Universităţii de Științe și Tehnologie POLITEHNICA Bucureşti, în program de master *Ingineria Informației și Sisteme de Calcul*, este scrisă de mine şi nu a mai fost prezentată niciodată la o facultate sau instituţie de învăţământ superior din ţară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referinţe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar şi în traducere proprie din altă limbă, sunt scrise între ghilimele şi fac referinţă la sursă. Reformularea în cuvinte proprii a textelor scrise de către alţi autori face referinţă la sursă. Înţeleg că plagiatul constituie infracţiune şi se sancţionează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor şi măsurătorilor pe care le prezint ca fiind făcute de mine, precum şi metodele prin care au fost obţinute, sunt reale şi provin din respectivele simulări, experimente şi măsurători. Înţeleg că falsificarea datelor şi rezultatelor constituie fraudă şi se sancţionează conform regulamentelor în vigoare.

Bucureşti, Iunie 2026.
Absolvent: Geană Tudor-Bogdan

.....

Cuprins

Lista figurilor . . . iii

Lista tabelelor . . . v

Rezumat . . . 1

1. Introducere . . . 3
1.1. Context și motivație . . . 3
1.2. Lucrări și instrumente conexe . . . 3
1.3. Obiectivele lucrării . . . 4
1.4. Contribuții originale . . . 4
1.5. Structura lucrării . . . 5

2. Epidemiologie Computațională și Modelarea Matematică a Bolilor Infecțioase 7
2.1. Fundamente de epidemiologie . . . 7
2.1.1. Definiții și concepte fundamentale . . . 7
2.1.2. Parametrii fundamentali ai transmiterii . . . 7
2.2. Modele compartimentale în epidemiologie . . . 8
2.2.1. Cadrul general . . . 8
2.2.2. Modelul SIS . . . 8
2.2.3. Modelul SEIR . . . 8
2.2.4. Modelul SIRS . . . 9
2.3. Modelul SIDARTHE . . . 9
2.4. Modelul SIR ca reper secundar . . . 10
2.4.1. Ipoteze și derivarea ecuațiilor din principii prime . . . 10
2.4.2. Proprietăți matematice fundamentale . . . 11
2.4.3. Analiza echilibrelor și stabilitate . . . 12
2.4.4. Numărul de reproducere de bază $\mathcal{R}_0$ . . . 13
2.4.4.1. Definiție prin matricea generației următoare . . . 13
2.4.4.2. Calculul $\mathcal{R}_0$ pentru modelul SIR . . . 13
2.4.5. Pragul de imunizare în turmă (herd immunity) . . . 14
2.4.6. Ecuația dimensiunii finale a epidemiei . . . 14
2.4.7. Extensii ale modelului SIR . . . 15
2.5. Metode Numerice pentru Ecuații Diferențiale Ordinare . . . 16
2.5.1. Clasificarea metodelor numerice . . . 16
2.5.2. Metoda Euler explicită . . . 16
2.5.3. Metoda Runge-Kutta de ordinul 4 (RK4) . . . 16
2.5.3.1. Derivare și formulă . . . 17
2.5.3.2. Schiță de demonstrație. . . . 17
2.5.4. Metode Runge-Kutta adaptive (RK45 / Dormand-Prince) . . . 17
2.5.4.1. Metode Runge-Kutta încorporate . . . 17

2.5.4.2. Metoda Dormand-Prince (DOPRI5 / RK45) . . . 18
2.5.4.3. Control adaptiv al pasului . . . 18
2.5.5. Stabilitate numerică și probleme stiff . . . 18
2.6. Date Epidemiologice — Surse și Preprocesare . . . 19
2.6.1. Surse de date epidemiologice . . . 19
2.6.2. Structura datelor și preprocesare . . . 19
2.6.3. Estimarea parametrilor din date reale . . . 19

3. Tehnologii și Instrumente Utilizate . . . 21
3.1. Limbaje de programare pentru calcul științific . . . 21
3.1.1. R . . . 21
3.1.1.1. Istoric și filosofie . . . 21
3.1.1.2. Caracteristici tehnice relevante . . . 21
3.1.1.3. Pachete relevante pentru modele compartimentale . . . 21
3.1.2. Julia . . . 22
3.1.2.1. Istoric și filosofie . . . 22
3.1.2.2. Compilare Just-In-Time și sistemul de tipuri . . . 22
3.1.2.3. DifferentialEquations.jl . . . 22
3.1.3. Python . . . 22
3.1.3.1. Istoric și filosofie . . . 22
3.1.3.2. Caracteristici tehnice și ecosistem științific . . . 22
3.1.3.3. Funcțiile de rezolvare a EDO în SciPy . . . 23
3.1.4. Analiză comparativă a limbajelor . . . 23
3.2. Baze de date distribuite — YugabyteDB . . . 23
3.2.1. Concepte de baze de date distribuite . . . 23
3.2.2. Arhitectura YugabyteDB . . . 24
3.2.3. Sharding și replicare . . . 24
3.3. Orchestrarea containerelor — Kubernetes . . . 24
3.3.1. Containere și nevoia de orchestrare . . . 24
3.3.2. Arhitectura Kubernetes . . . 25
3.3.3. Resurse Kubernetes relevante . . . 25
3.4. Backend API — Go . . . 25
3.4.1. Caracteristici și filosofia limbajului Go . . . 25
3.4.2. Ecosistem pentru API-uri REST . . . 25
3.5. Frontend — Vue.js . . . 25
3.5.1. Arhitectura Vue 3 și Composition API . . . 25
3.5.2. Ecosistem și biblioteci utilizate . . . 26

4. Arhitectura Sistemului E-Medicine . . . 27
4.1. Vedere de ansamblu . . . 27
4.2. Cerințe funcționale și non-funcționale . . . 27
4.2.1. Cerințe funcționale . . . 27
4.2.2. Cerințe non-funcționale . . . 30
4.3. Arhitectura bazei de date distribuite . . . 30
4.3.1. Schema relațională . . . 30
4.3.2. Modelul datelor epidemiologice . . . 31
4.3.3. Strategia de partiționare și replicare în YugabyteDB . . . 32

4.4. Arhitectura API-ului backend Go . . . 33
4.4.1. Structura modulară a proiectului . . . 33
4.4.2. Mecanismul de autentificare JWT . . . 34
4.4.3. RBAC pe studii . . . 34
4.4.4. Diagrama endpointurilor REST . . . 34
4.4.5. Integrarea cu runner-ele de algoritmi . . . 34
4.5. Arhitectura frontend Vue.js . . . 35
4.5.1. Structura componentelor . . . 35
4.5.2. Fluxul de date și state management . . . 36
4.6. Infrastructura Kubernetes . . . 37
4.6.1. Topologia cluster-ului . . . 37
4.6.2. Deployment YugabyteDB pe Kubernetes . . . 37
4.6.3. Stratul aplicativ: imagini container și registru . . . 38
4.6.4. Coadă de mesaje și migrațiile bazei de date . . . 39
4.6.5. Expunere externă și ingress . . . 39
4.7. Sistem de plug-in-uri pentru algoritmi . . . 40
4.7.1. Forma standard: *runner protocol* . . . 40
4.7.2. Provisioning Kubernetes . . . 41
4.7.3. Exemplu concret de runner plug-in . . . 41
4.7.4. Securitate și limitări . . . 42

5. Implementarea modelelor compartimentale . . . 45
5.1. Datele de intrare și inițializarea compartimentelor . . . 45
5.2. Contractul comun de simulare . . . 47
5.3. Implementarea modelului SIDARTHE . . . 47
5.4. Variantele SIR, SIS, SEIR și SIRS . . . 49
5.5. Persistența rezultatelor și exportul . . . 50
5.6. Metrici de performanță . . . 50

6. Predicție Epidemiologică folosind Rețele LSTM . . . 51
6.1. Rolul predicției statistice în sistem . . . 51
6.2. Formularea problemei de predicție . . . 51
6.3. Arhitectura celulei LSTM . . . 51
6.4. Normalizarea și construcția exemplelor . . . 53
6.5. Antrenarea prin backpropagation through time . . . 53
6.6. Împărțirea cronologică și evaluarea . . . 54
6.7. Integrarea în arhitectura aplicației . . . 55
6.8. Linii de bază și limitări . . . 56
6.9. Direcții de extindere . . . 57

7. Comparația Modelelor, Analiză Numerică și Calibrare . . . 59
7.1. Rolul comparației între modele . . . 59
7.2. Comparație structurală . . . 59
7.3. Operatorul de observație . . . 59
7.4. Metrici de eroare numerică . . . 60
7.5. Protocol de analiză comparativă . . . 61
7.6. Calibrarea parametrilor . . . 61
7.6.1. Aplicare pe România, valul 1 din 2020 . . . 62

7.7. Identificabilitate și limitări ale calibrării . . . 63
7.7.1. Vizualizarea identifiabilității practice pentru SIR . . . 63
7.7.2. Generalizare pe mai multe regiuni . . . 64
7.8. Integrarea în sistemul software . . . 66
7.8.1. Costul software al calibrării . . . 67

8. Evaluarea Software, Performanței și Scalabilității . . . 69
8.1. Scopul evaluării . . . 69
8.2. Metrici colectate . . . 69
8.3. Matricea experimentală . . . 70
8.4. Protocol de rulare . . . 71
8.5. Scalabilitate și complexitatea modelului . . . 72
8.6. Compararea implementărilor R, Julia și Python . . . 72
8.7. Indicatori agregați . . . 72
8.7.1. Rezultate măsurate . . . 72
8.7.2. Costul componentei LSTM . . . 75
8.7.3. Variabilitatea cross-hardware . . . 76
8.8. Acuratețe numerică și verificări de regresie . . . 77
8.9. Evaluarea API-ului și a bazei de date . . . 78
8.10. Interpretarea rezultatelor . . . 79

9. Aplicația Web E-Medicine . . . 81
9.1. Rolul aplicației . . . 81
9.2. Backend Go . . . 81
9.3. Gestionarea dataseturilor . . . 81
9.4. Execuția simulărilor . . . 82
9.5. Editor de cod pentru cercetători . . . 82
9.5.1. Flux de lucru . . . 82
9.5.2. Contractul scriptului . . . 83
9.5.3. Execuție și separare de runner-e persistente . . . 83
9.5.4. Izolare între utilizatori și securitate . . . 83
9.6. Consola super-admin . . . 84
9.7. Considerații de securitate . . . 85
9.7.1. Mecanisme implementate . . . 85
9.7.2. Limitări cunoscute și remedieri . . . 86
9.8. Validare funcțională . . . 86

10. Concluzii și Direcții de Dezvoltare Viitoare . . . 87
10.1. Sinteza lucrării . . . 87
10.1.1. Răspuns la obiectivele propuse . . . 87
10.2. Concluzii privind modelele . . . 88
10.3. Concluzii privind implementarea . . . 88
10.4. Concluzii experimentale . . . 89
10.5. Limitări . . . 90
10.6. Direcții viitoare . . . 90

Bibliografie . . . 93

Anexa A. Cod Sursă — Implementarea R 97

Anexa B. Cod Sursă — Implementarea Julia 103

Anexa C. Cod Sursă — Implementarea Python 107

Anexa D. Cod Sursă — Runner LSTM 111

Anexa E. Schema Bazei de Date 119

Anexa F. Configurații Kubernetes 125

Anexa G. Documentația API REST 139

Lista figurilor

2.1. Diagrama de flux a compartimentelor modelului SIR. Fluxul $\beta SI/N$ reprezintă rata de noi infecții, iar γI reprezintă rata de recuperare. 11
4.1. Arhitectura globală a sistemului e-medicine. Săgețile indică direcția de flux a datelor: browserul accesează aplicația prin Ingress, care rutează cererile către cele două SPA-uri sau API; API-ul publică job-uri în Redis și le consumă runner-ele dedicate; atât API-ul, cât și runner-ele accesează același cluster YugabyteDB. . . 28
4.2. Ciclul de viață al unei simulări, de la cererea utilizatorului în UI până la afișarea rezultatului. Timpul curge de sus în jos. Săgețile pline indică mesaje sincrone HTTP/queue/SQL; săgețile întrerupte indică polling-ul repetat al statusului până la finalizare. API-ul răspunde `201 Created` *înainte* ca solver-ul să ruleze — întreaga execuție numerică se petrece asincron în runner, decuplată prin Redis. Aceasta este complementară Figurii 4.1, care arată topologia: cine vorbește cu cine; aici se vede *când*. 29
4.3. Diagrama entitate-relație a bazei de date curente. PK-urile sunt marcate cu aldine și subliniere; FK-urile sunt marcate cu *FK*. Tabelele `datasets` și `simulation_runs` sunt scope-uite pe `study_id`, iar tabela `study_members` definește rolul fiecărui utilizator într-un studiu. 31
4.4. Topologia cluster-ului Kubernetes cu 3 noduri control plane HA și 3 noduri worker. Pod-urile YugabyteDB sunt distribuite uniform prin `podAntiAffinity`, garantând că pierderea unui worker afectează cel mult o replică `yb-master` și o replică `yb-tserver` — în limita toleranței quorum-ului Raft. 38
6.1. Predicții LSTM pe România OWID 55
7.1. Calibrare SIR pe valul 1 (2020) 64
7.2. Peisaj de pierdere SIR 65
7.3. Calibrare SIR pe 4 țări europene 66
8.1. Scalare wall-time vs $t_{\max}$ 75

Lista tabelelor

3.1. Comparație a limbajelor de programare utilizate 23
4.1. Structura minimă a fișierului CSV de date epidemiologice. 32
4.2. Endpointurile REST principale ale API-ului backend. 35
4.3. Ciclul de viață runner plug-în 41
5.1. Maparea datelor observabile în condiții inițiale pentru modelele suportate. . . . 46
5.2. Comparație runner-e compartimentale 49
6.1. Corespondența dintre formularea matematică a celulei LSTM și implementarea din `runner-lstm`. 52
6.2. LSTM vs predictori naivi 56
7.1. Comparație structurală între modelele utilizate în lucrare. 60
7.2. Locurile din analiza comparativă care necesită testare pentru obținerea rezultatelor. 62
7.3. Calibrare SIR pe România 63
7.4. Calibrare SIR multi-regiune 65
7.5. Cost software calibrare SIR 67
8.1. Metricile colectate sau derivabile pentru evaluarea software. 69
8.2. Matrice minimă pentru campania de benchmark. 70
8.3. Teste necesare pentru obținerea rezultatelor de evaluare software. 71
8.4. Wall-time median solver 73
8.5. Speedup solver vs Python 74
8.6. Memorie peak (mediană) 74
8.7. Cost cold-start JIT Julia 74
8.8. Cost runner LSTM 75
8.9. Comparare hardware M1 vs cluster 76
8.10. Regresie numerică cross-limbaj 77
8.11. Latența API pe model ($t_{\max} = 365$) 78
8.12. Scalare latență read cu orizontul 79
F.1. Manifestele Kubernetes incluse în repository. 126

Rezumat

Această lucrare tratează proiectarea unui sistem e-medicine pentru analiza datelor epidemiologice prin modele compartimentale, cu modelul SIDARTHE folosit ca direcție clinică principală de explorare. Spre deosebire de modelele SIR/SIS/SEIR/SIRS, care agregă dinamica în câteva stări canonice, SIDARTHE separă infecțiile nedepistate, cazurile diagnosticate, formele simptomatice, formele severe, vindecările și decesele. Această granularitate face modelele mai potrivite pentru discutarea efectelor testării, diagnosticării și severității clinice asupra evoluției unei epidemii.

Modelele SIR, SIS, SEIR și SIRS rămân prezente în lucrare ca repere secundare, folosite pentru comparații și pentru verificarea comportamentului numeric al implementărilor. Sistemul construit separă explicit stocarea datelor, expunerea API-ului și execuția simulărilor. Datele epidemiologice sunt persistate într-o schemă relațională compatibilă PostgreSQL, rulată peste YugabyteDB, iar simulările sunt executate asincron de runner-e specializate, coordonate prin Redis.

Lucrarea introduce un cadru explicit pentru compararea modelelor și calibrarea parametrilor. Modelele sunt comparate structural, prin compartimente și ipoteze, și numeric, prin operatori de observație, MAE, RMSE și nRMSE. Calibrarea este formulată ca problemă de optimizare pe parametrii fiecărui model, cu accent pe separarea dintre fereastra de antrenare și fereastra de test. În plus, este integrată o extensie predictivă bazată pe LSTM, cu runner dedicat, folosită ca model de serie temporală și ca posibilă corecție a reziduurilor modelelor compartimentale.

Componenta web, alcătuită dintr-un API Go și două interfețe Vue.js separate, oferă fluxurile necesare organizării datelor pe studii epidemiologice, gestionării rolurilor de acces, încărcării seturilor de date, lansării simulărilor, urmăririi statusului și vizualizării traiectoriilor compartimentale. Aplicația principală este destinată cercetătorilor și colaboratorilor dintr-un studiu, iar consola administrativă este rezervată utilizatorilor cu drepturi globale de super-admin. Schema datelor epidemiologice acoperă, dincolo de cele patru câmpuri SIR-centrice, nouă metrici suplimentare (decese, spitalizări, terapie intensivă, vaccinări, teste, indicele Oxford de stringență, R_t observat) propagate din sursa OWID; aceste valori nu sunt folosite de runner-ele compartimentale built-in, dar sunt expuse integral scripturilor scrise de cercetător într-un editor web dedicat, ceea ce permite formularea de modele ad-hoc fără modificări de cod în API. Întregul sistem este deployabil pe un cluster Kubernetes prin manifeste declarative, pe o platformă cloud privată cu factor de replicare 3 pentru baza de date. Sistemul este validat empiric printr-o campanie de benchmark executată prin aplicație (180 de rulări ODE cu cinci modele compartimentale, trei limbaje și patru orizonturi temporale) și prin calibrarea SIR pe valul 1 al pandemiei COVID-19 în România, Italia, Germania și Spania, cu analiza identifiabilității practice prin scanarea peisajului de pierdere. Accentul lucrării cade pe legătura dintre modelul matematic, implementarea numerică și arhitectura software care face posibilă evaluarea reproductibilă a rezultatelor pentru modele cu grade diferite de complexitate.

Cuvinte cheie: model SIDARTHE, modele SIR, epidemiologie computațională, e-medicine, calibrare de parametri, LSTM, RBAC, YugabyteDB, Kubernetes, Go, Vue.js, benchmark de performanță.

Capitolul 1
Introducere

1.1 Context și motivație

Bolile infecțioase rămân una dintre formele prin care vulnerabilitatea unui sistem sanitar devine vizibilă la scară socială. Pandemia de gripă din 1918, epidemia de SARS din 2003 și pandemia de COVID-19 declanșată în 2019 arată că propagarea unui agent patogen nu poate fi înțeleasă doar ca fenomen clinic, ci și ca proces populațional, dependent de contacte, mobilitate, raportare și capacitatea de răspuns instituțional [1, 2].

Modelele matematice compartimentale oferă un limbaj riguros pentru descrierea acestor procese. Ele reduc populația la stări epidemiologice relevante și descriu tranzițiile dintre stări prin ecuații diferențiale. Modelul SIR (Susceptibili–Infectați–Recuperați), introdus de Kermack și McKendrick în 1927 [3], rămâne un reper important deoarece permite o analiză matematică transparentă. Totuși, pentru epidemii în care testarea, diagnosticul și severitatea clinică modifică puternic raportarea, un model cu trei compartimente este prea agregat. Din acest motiv, lucrarea folosește SIDARTHE ca model clinic principal, păstrând modelele SIR, SIS, SEIR și SIRS ca repere secundare pentru comparații și verificări.

În practica actuală, problema nu se oprește la scrierea ecuațiilor. Datele provin din surse eterogene, sunt actualizate frecvent și conțin corecții, goluri sau inconsistențe administrative. În același timp, implementarea numerică poate fi realizată în limbaje diferite, fiecare cu avantaje și costuri proprii. Alegerea dintre R, Julia și Python nu ține doar de sintaxă, ci influențează timpul de execuție, consumul de memorie, modul de integrare cu infrastructura și reproductibilitatea analizei.

Pornind de la aceste observații, lucrarea propune un sistem e-medicine în care modelul SIDARTHE este integrat într-un flux complet: ingestie de date epidemiologice, execuție asincronă a simulărilor, stocare a rezultatelor și vizualizare în interfață web. Modelele SIR, SIS, SEIR și SIRS rămân utile ca repere mai simple, prin care se poate observa ce se pierde sau ce se câștigă atunci când modelul introduce compartimente suplimentare.

1.2 Lucrări și instrumente conexe

Modelarea computațională a epidemiilor se sprijină pe un ecosistem matur de biblioteci de rezolvare a ecuațiilor diferențiale. Lucrarea de față folosește direct trei dintre ele, fiecare reprezentativă pentru limbajul său: `deSolve` în R [4], `DifferentialEquations.jl` în Julia [5] și `scipy.integrate` în Python [6]. Aceste biblioteci sunt cărămizi de bază, optimizate și bine testate, dar rămân componente de calcul: ele rezolvă un sistem ODE dat, fără a impune o structură de date epidemiologice, o disciplină de colectare a metricilor sau un cadru de comparație între limbaje. În această lucrare ele sunt tratate exact ca atare — componente interschimbabile, evaluate sistematic în aceleași condiții.

La un nivel mai înalt există instrumente dedicate modelării epidemiologice, dar fiecare se specializează pe un ecosistem sau pe o paradigmă. EpiModel oferă în R un cadru pentru modele compartimentale și pe rețele de contact [7]; Covasim implementează în Python un model agent-based detaliat pentru COVID-19 [8]; iar microsimularea de tip CovidSim folosită în raportul

Imperial College a influențat deciziile de politică sanitară din 2020 [9]. Aceste instrumente sunt puternice în domeniul lor, însă rămân legate de un singur limbaj și nu urmăresc o comparație cantitativă cross-limbaj a costului numeric, nici integrarea într-un sistem web colaborativ.

Pentru estimarea parametrilor, abordarea standard în literatura recentă este calibrarea bayesiană, cu instrumente precum Stan [10] sau NIMBLE [11], care oferă cuantificarea incertitudinii prin metode de tip MCMC/HMC. Lucrarea de față folosește o calibrare deterministă prin metoda Nelder–Mead, suficientă pentru obiectivul comparativ urmărit; abordarea bayesiană este menționată ca direcție viitoare, nu ca element exclus din motive metodologice.

În fine, dincolo de modelare există platforme de agregare și diseminare a prognozelor, precum COVID-19 Forecast Hub [12], care combină predicții din mai multe modele independente în prognoze de ansamblu. Acestea rezolvă o problemă complementară — coordonarea și evaluarea prognozelor la scară — și nu un toolkit de modelare reutilizabil.

Raportat la aceste direcții, contribuția lucrării se află la intersecția lor: un benchmarking comparativ al aceluiași model implementat în R, Julia și Python, combinat cu o platformă web integrată, cu control al accesului bazat pe roluri la nivel de studiu și cu o evaluare empirică sistematică a costului software. Această combinație o deosebește atât de bibliotecile mono-limbaj de rezolvare a ODE-urilor, cât și de platformele epidemiologice specializate care nu oferă această integrare de sistem.

1.3 Obiectivele lucrării

Obiectivul principal este proiectarea unui sistem care leagă explicit modelarea matematică de implementarea software. Modelul SIDARTHE este tratat ca model de lucru principal, deoarece separă cazurile nedepistate, diagnosticate, simptomatice, severe, vindecate și decedate. Familia SIR/SIS/SEIR/SIRS este analizată în continuare, dar rolul său devine acela de reper teoretic și computațional, util pentru înțelegerea proprietăților de bază ale modelelor compartimentale.

Al doilea obiectiv este implementarea aceluiași contract de simulare în R, Julia și Python, folosind bibliotecile uzuale ale fiecărui ecosistem pentru rezolvarea ecuațiilor diferențiale. Comparația urmărește atât compatibilitatea rezultatelor, cât și costurile practice ale execuției: timp wall-clock, timp CPU, memorie și integrare cu baza de date.

Al treilea obiectiv este definirea unei metodologii explicite de comparație și calibrare. Modelele nu sunt evaluate doar vizual, ci prin operatori de observație, metrici de eroare precum MAE și RMSE, separare între fereastra de calibrare și fereastra de test și raportarea costului software asociat fiecărei rulări.

Un obiectiv suplimentar este integrarea unei extensii predictive bazate pe LSTM. Aceasta este tratată ca model de serie temporală pentru predicții numerice pe termen scurt, implementat prin runner dedicat, și ca posibilă corecție a reziduurilor produse de modelele compartimentale calibrate.

În paralel, lucrarea construiește infrastructura necesară rulării acestor experimente într-un cadru accesibil prin web. API-ul Go, baza de date YugabyteDB, coada Redis și cele două interfețe Vue.js sunt tratate ca părți ale aceluiași instrument de cercetare, nu ca module izolate.

1.4 Contribuții originale

Contribuția lucrării constă în combinarea unei analize matematice clasice cu un sistem software funcțional, capabil să ruleze modele compartimentale cu structuri de stare diferite în aceleași medii de execuție. Partea matematică fixează ipotezele și limitele modelelor, partea de implementare arată cum se schimbă costul numeric între limbaje, iar partea de arhitectură oferă un cadru în care datele, simulările și benchmark-urile pot fi gestionate unitar.

O a doua contribuție este metodologia de colectare a metricilor. Fiecare runner marchează explicit stările unei simulări, scrie rezultatele în aceeași schemă de bază de date și salvează metrici comparabile. Această disciplină de integrare este necesară pentru a evita comparațiile informale între limbaje, care pot fi ușor influențate de diferențe de I/O, inițializare sau serializare.

O altă contribuție este separarea între comparația epidemiologică și comparația software. Lucrarea definește operatori de observație pentru a transforma stările interne ale modelelor în serii comparabile cu datele reale și introduce un cadru de calibrare care poate fi extins ulterior într-un job dedicat în aplicație.

În fine, sistemul web rezultat permite folosirea modelelor într-un flux apropiat de un scenariu real de analiză colaborativă: crearea unui studiu, acordarea rolurilor, încărcarea unui set de date, alegerea modelului și a parametrilor, execuția asincronă, vizualizarea traiectoriilor și compararea rulărilor.

O contribuție arhitecturală suplimentară este **sistemul de plug-in-uri pentru algoritmi** (capitolul 4, secțiunea 4.7): un super-admin poate înregistra într-un studiu fie un script Python (rulat de un container generic), fie o referință către o imagine container construită extern (în orice limbaj), iar API-ul provisionează automat un Deployment Kubernetes dedicat. Această extensie deschide aplicația de la cinci modele compartimentale plus LSTM la *orice* algoritm care respectă un contract minimal de I/O, fără modificări la API sau la schema bazei de date.

1.5 Structura lucrării

Lucrarea este organizată în zece capitole.

Capitolul 2 (*Epidemiologie Computațională și Modelarea Matematică*) prezintă fundamentele epidemiologiei matematice. Sunt introduse modelele compartimentale (SIR, SIS, SEIR, SIRS), apoi este prezentat modelul SIDARTHE ca model clinic principal. Modelul SIR este păstrat pentru analiza matematică de bază, deoarece permite discutarea clară a pozitivității, conservării populației și pragului epidemic $\mathcal{R}_0$. Capitolul include și o trecere în revistă a metodelor numerice utilizate pentru rezolvarea sistemelor de ecuații diferențiale ordinare.

Capitolul 3 (*Tehnologii și Instrumente Utilizate*) explică de ce au fost alese limbajele R, Julia și Python, baza de date YugabyteDB, orchestrarea containerelor prin Kubernetes, backend-ul Go și frontend-urile Vue.js.

Capitolul 4 (*Arhitectura Sistemului E-Medicine*) descrie arhitectura globală a sistemului, cerințele funcționale și non-funcționale, schema bazei de date distribuite, arhitectura API-ului backend și structura componentelor frontend.

Capitolul 5 (*Implementarea modelelor compartimentale*) detaliază implementările practice ale modelelor în fiecare limbaj de programare, inclusiv seturile de date utilizate, parametrii, schema generică de rezultate și interfața de integrare cu API-ul Go.

Capitolul 6 (*Predicție Epidemiologică folosind Rețele LSTM*) prezintă rolul unei componente LSTM pentru prognoza seriilor temporale, formularea problemei de predicție, integrarea cu modelele compartimentale, validarea față de linii de bază și formularea unui model hibrid compartimental–LSTM ca direcție de extindere.

Capitolul 7 (*Comparația Modelelor, Analiză Numerică și Calibrare*) introduce tabelul comparativ al modelelor, operatorii de observație, metricile MAE/RMSE și formularea calibrării parametrilor ca problemă de optimizare.

Capitolul 8 (*Evaluarea Software, Performanței și Scalabilității*) prezintă metodologia de benchmarking, matricea experimentală, metricile colectate, verificările de regresie numerică și evaluarea API-ului și a bazei de date.

Capitolul 9 (*Aplicația Web E-Medicine*) detaliază implementarea aplicației web, documentând backend-ul Go, aplicația principală Vue.js și consola Vue.js pentru super-admini, cu

descrierea funcționalităților și a fluxurilor de utilizare.

Capitolul 10 (*Concluzii și Direcții Viitoare*) sintetizează contribuțiile lucrării, discută limitele sistemului și identifică direcții de dezvoltare ulterioară.

Lucrarea se încheie cu **Bibliografie** și **Anexe** care conțin codul sursă complet al implementărilor, schema bazei de date, configurațiile Kubernetes și documentația API-ului REST.

Capitolul 2
Epidemiologie Computațională și Modelarea Matematică a Bolilor Infecțioase

2.1 Fundamente de epidemiologie

2.1.1 Definiții și concepte fundamentale

Epidemiologia este știința care studiază distribuția, determinanții și controlul stărilor sau evenimentelor legate de sănătate în populații definite [1]. În sens mai specific, *epidemiologia infecțioasă* se ocupă cu dinamica transmiterii bolilor cauzate de agenți patogeni biologici (virusuri, bacterii, paraziți) în populații umane sau animale.

Definiție 2.1 (Epidemic, pandemic, endemic)**.** O boală infecțioasă se numește **endemică** dacă persistă la nivel constant într-o regiune geografică definită. Se numește **epidemie** dacă numărul de cazuri depășește clar nivelul endemic așteptat, și **pandemie** dacă afectează mai multe continente sau o mare parte a globului.

Definiție 2.2 (Incidență și prevalență)**. Incidența** reprezintă numărul de cazuri noi ale unei boli apărute într-o populație de risc, într-un interval de timp specificat. Se exprimă ca rată:

$$\text{Incidență} = \frac{\text{cazuri noi în intervalul } [t_1, t_2]}{\text{populație la risc} \times (t_2 - t_1)}. \tag{2.1}$$

Prevalența reprezintă proporția indivizilor afectați la un moment dat:

$$\text{Prevalență} = \frac{\text{cazuri existente la momentul } t}{\text{populație totală la momentul } t}. \tag{2.2}$$

2.1.2 Parametrii fundamentali ai transmiterii

Definiție 2.3 (Intervalul serial și intervalul de generație)**. Intervalul serial** este durata medie dintre apariția simptomelor la cazul primar și la cazul secundar generat de acesta. **Intervalul de generație** este durata medie dintre infecția cazului primar și infecția unui caz secundar.

Definiție 2.4 (Perioada de incubație)**. Perioada de incubație** este intervalul de timp dintre momentul infectării și apariția primelor simptome. Este distinctă de *perioada de latență*, care reprezintă intervalul dintre infecție și debutul infecțiozității.

Definiție 2.5 (Numărul de reproducere de bază $\mathcal{R}_0$)**.** Numărul de reproducere de bază $\mathcal{R}_0$ este definit ca numărul mediu de infecții secundare produse de un singur individ infectat introdus într-o populație complet susceptibilă, pe durata perioadei sale infecțioase [13]. Este cel mai important parametru al epidemiologiei matematice, deoarece determină potențialul epidemic al unui agent patogen.

O consecință imediată a Teoremă 2.5 este existența unui prag epidemic. Dacă $\mathcal{R}_0 < 1$, fiecare caz infectat produce în medie mai puțin de un caz nou, iar epidemia se stinge spontan. Dacă

$\mathcal{R}_0 > 1$, numărul de cazuri crește în faza inițială, de regulă cu profil aproximativ exponențial. Cazul $\mathcal{R}_0 = 1$ separă cele două regimuri și corespunde unei stări de echilibru instabil.

Numărul de reproducere *efectiv* (sau în timp real) $\mathcal{R}(t)$ generalizează $\mathcal{R}_0$ pentru o populație parțial imunizată:

$$\mathcal{R}(t) = \mathcal{R}_0 \cdot \frac{S(t)}{N}, \tag{2.3}$$

unde $S(t)$ este numărul de indivizi susceptibili la momentul t și N este mărimea populației totale.

2.2 Modele compartimentale în epidemiologie

2.2.1 Cadrul general

Modelele compartimentale reprezintă clasa fundamentală de modele matematice în epidemiologie [2]. Principiul lor constă în împărțirea populației în *compartimente* care descriu stări epidemiologice distincte, iar tranziția dintre compartimente este guvernată de un sistem de ecuații diferențiale ordinare (EDO).

Fie N mărimea totală a populației (presupusă constantă). Fiecare compartiment $X_i(t)$ reprezintă numărul de indivizi în starea i la momentul t, cu $\sum_i X_i(t) = N$. Dinamica tranzițiilor este descrisă de sistemul:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t, \boldsymbol{\theta}), \quad \mathbf{x}(0) = \mathbf{x}_0, \tag{2.4}$$

unde $\mathbf{x} = (X_1, \ldots, X_n)^\intercal$, $\boldsymbol{\theta}$ este vectorul parametrilor și $\mathbf{f}$ descrie fluxurile nete dintre compartimente.

2.2.2 Modelul SIS

Modelul **SIS** (Susceptibili-Infectați-Susceptibili) descrie infecții în care recuperarea nu conferă imunitate de durată, permițând re-infectarea:

$$\begin{cases} \frac{dS}{dt} = -\dfrac{\beta SI}{N} + \gamma I, \\ \frac{dI}{dt} = \quad \dfrac{\beta SI}{N} - \gamma I. \end{cases} \tag{2.5}$$

Parametrul $\gamma > 0$ reprezintă rata de recuperare ($1/\gamma$ este durata medie a perioadei infecțioase). Pentru modelul SIS, $\mathcal{R}_0 = \beta/\gamma$. Dacă $\mathcal{R}_0 \leq 1$, echilibrul fără boală $I^* = 0$ este stabil global; dacă $\mathcal{R}_0 > 1$, sistemul admite un echilibru endemic stabil $I^* = N(1 - 1/\mathcal{R}_0) > 0$.

2.2.3 Modelul SEIR

Modelul **SEIR** introduce compartimentul E (Expuși / în perioadă de latență), care ține cont de faptul că indivizii infectați nu sunt imediat infecțioși:

$$\begin{cases} \frac{dS}{dt} = -\dfrac{\beta SI}{N}, \\ \frac{dE}{dt} = \quad \dfrac{\beta SI}{N} - \sigma E, \\ \frac{dI}{dt} = \quad \sigma E - \gamma I, \\ \frac{dR}{dt} = \quad \gamma I, \end{cases} \tag{2.6}$$

unde $\sigma = 1/T_E$ este rata de tranziție din starea expusă în starea infecțioasă (T_E = durata medie de incubație). Numărul de reproducere de bază pentru modelul SEIR este $\mathcal{R}_0 = \beta/\gamma$, identic cu cel al modelului SIR.

2.2.4 Modelul SIRS

Modelul **SIRS** generalizează modelul SIR prin imunitate temporară (indivizii recuperați pot deveni din nou susceptibili după un interval de timp):

$$\begin{cases} \frac{dS}{dt} = -\dfrac{\beta SI}{N} + \xi R, \\ \frac{dI}{dt} = \quad \dfrac{\beta SI}{N} - \gamma I, \\ \frac{dR}{dt} = \quad \gamma I - \xi R, \end{cases} \tag{2.7}$$

unde $\xi = 1/T_R$ este rata de pierdere a imunității (T_R = durata medie a imunității). Dacă $\mathcal{R}_0 > 1$, modelul SIRS admite un echilibru endemic stabil.

2.3 Modelul SIDARTHE

Modelul SIDARTHE, propus pentru analiza epidemiei COVID-19 [14], separă două dimensiuni care lipsesc din SIR: statutul de diagnosticare și severitatea clinică. Într-un sistem de raportare real, persoanele infectate dar nedepistate, persoanele diagnosticate, cazurile simptomatice și cazurile severe nu au aceeași contribuție la transmitere și nu sunt observate cu aceeași probabilitate. Sistemul implementat în această lucrare folosește varianta **SIDARTHE**, care include explicit compartimentul de decese.

Compartimentele modelului sunt S (susceptibili), I (infectați nedepistați), D (diagnosticați asimptomatici), A (simptomatici nedepistați), R (simptomatici diagnosticați), T (cazuri severe), H (vindecați) și E (extinși / decedați). Compartimentul E este alimentat din T prin rata de mortalitate τ, ceea ce permite analiza separată a vindecării și a deceselor.

$$\frac{dS}{dt} = -S \cdot \frac{\alpha I + \beta D + \gamma A + \delta R}{N}, \tag{2.8}$$

$$\frac{dI}{dt} = S \cdot \frac{\alpha I + \beta D + \gamma A + \delta R}{N} - (\varepsilon + \zeta + \lambda) I, \tag{2.9}$$

$$\frac{dD}{dt} = \varepsilon I - (\eta + \rho) D, \tag{2.10}$$

$$\frac{dA}{dt} = \zeta I - (\theta + \mu + \kappa) A, \tag{2.11}$$

$$\frac{dR}{dt} = \eta D + \theta A - (\nu + \xi) R, \tag{2.12}$$

$$\frac{dT}{dt} = \mu A + \nu R - (\sigma + \tau) T, \tag{2.13}$$

$$\frac{dH}{dt} = \lambda I + \rho D + \kappa A + \xi R + \sigma T, \tag{2.14}$$

$$\frac{dE}{dt} = \tau T. \tag{2.15}$$

Parametrii α, β, γ și δ controlează contribuția la transmitere a compartimentelor infecțioase. Parametrii ε, ζ și λ descriu ieșirile din starea infectată nedepistată: diagnosticare, apariția

simptomelor și vindecare fără diagnostic. Restul parametrilor descriu tranzițiile către forme diagnosticate, severe sau vindecate. Spre deosebire de SIR, unde aproape toată interpretarea se concentrează în raportul β/γ, SIDARTHE distribuie dinamica pe un număr mai mare de rate; de aceea, analiza numerică și sensibilitatea la parametri devin mai importante decât obținerea unei formule scalare simple.

Conservarea populației se verifică prin însumarea ecuațiilor din (2.15). Toți termenii interni se anulează, iar relația devine

$$\frac{d}{dt}(S + I + D + A + R + T + H + E) = 0. \tag{2.16}$$

Prin urmare, pentru condiții inițiale nenegative, suma compartimentelor rămâne egală cu populația totală N. Pozitivitatea soluției se obține prin același argument folosit la modelele compartimentale clasice: pe frontiera fiecărui compartiment, derivata nu poate împinge soluția în regiunea negativă, deoarece termenii de ieșire dispar când variabila respectivă este zero.

2.4 Modelul SIR ca reper secundar

2.4.1 Ipoteze și derivarea ecuațiilor din principii prime

Modelul SIR, formulat de Kermack și McKendrick [3], se bazează pe o serie de ipoteze simplificatoare care trebuie înțelese explicit:

1. **Populație închisă și omogenă:** Nu există nașteri, decese naturale sau migrații; $N = \text{const}$. Fiecare individ are aceeași probabilitate de a intra în contact cu orice alt individ (amestecare omogenă).

2. **Transmitere prin acțiune de masă:** Rata de contact eficient (producătoare de infecție) este proporțională cu produsul $S \cdot I$.

3. **Imunitate permanentă:** Un individ recuperat nu poate fi re-infectat; compartimentul R este absorbant.

4. **Durata infecțioasă exponențial distribuită:** Rata de recuperare γ este constantă, iar durata medie a perioadei infecțioase este $1/\gamma$.

5. **Fără mortalitate indusă de boală:** Sau, echivalent, mortalitatea specifică bolii este neglijabilă față de dinamica transmiterii.

Derivarea termenului de infecție. Considerăm că în unitatea de timp, un individ susceptibil are în medie c contacte cu alți indivizi aleși uniform din populație. Probabilitatea ca un contact arbitrar să fie cu un individ infectat este I/N, iar probabilitatea de transmitere la contact eficient este p. Astfel, rata de infecție a unui susceptibil este $c \cdot p \cdot I/N = \beta I/N$, unde $\beta = cp$ este parametrul de transmitere. Fluxul total de noi infecții pe unitate de timp este $(\beta I/N) \cdot S$.

Sistemul de ecuații diferențiale ordinare al modelului SIR este:

$$\boxed{\begin{aligned} \frac{dS}{dt} &= -\frac{\beta SI}{N}, \\ \frac{dI}{dt} &= \frac{\beta SI}{N} - \gamma I, \\ \frac{dR}{dt} &= \gamma I, \end{aligned}} \tag{2.17}$$

cu condiții inițiale $S(0) = S_0 > 0$, $I(0) = I_0 > 0$, $R(0) = R_0 \geq 0$ și $S_0 + I_0 + R_0 = N$.

Diagrama de flux a compartimentelor este prezentată în Fig. 2.1.

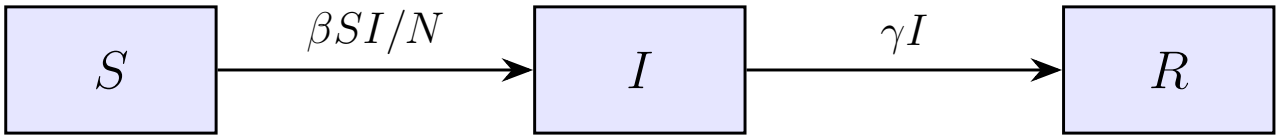


Figura 2.1: Diagrama de flux a compartimentelor modelului SIR. Fluxul $\beta SI/N$ reprezintă rata de noi infecții, iar γI reprezintă rata de recuperare.

2.4.2 Proprietăți matematice fundamentale

Înainte de a analiza comportamentul calitativ al soluțiilor, stabilim proprietățile matematice esențiale: existența și unicitatea soluției, conservarea populației totale și invarianța domeniului pozitiv.

Teoremă 2.6 (Existență și unicitate globală)**.** *Fie $\beta, \gamma > 0$, $N > 0$ și $(S_0, I_0, R_0) \in \mathbb{R}^3$ cu $S_0, I_0 > 0$, $R_0 \geq 0$. Atunci sistemul* (2.17) *admite o unică soluție globală $(S, I, R) \in C^1([0, \infty); \mathbb{R}^3)$.*

Demonstrație. Definim $\mathbf{x} = (S, I, R)^\intercal$ și $\mathbf{f}(\mathbf{x}) = (-\beta SI/N,\ \beta SI/N - \gamma I,\ \gamma I)^\intercal$. Funcția $\mathbf{f} : \mathbb{R}^3 \to \mathbb{R}^3$ este de clasă C^∞ (polinomul de gradul doi în S și I este neted), în particular continuu diferențiabilă. Prin *teorema Picard-Lindelöf* (Cauchy-Lipschitz), există o soluție locală unică pe un interval $[0, T_{\max})$.

Existența *globală* ($T_{\max} = \infty$) se stabilește prin mărginirea soluției. Deoarece $S(t), I(t), R(t) \geq 0$ (Teorema 2.8) și $S + I + R = N$ (Teorema 2.7), rezultă că $\|\mathbf{x}(t)\| \leq N\sqrt{3}$ pentru orice $t \in [0, T_{\max})$. Soluția mărginită nu poate prezenta blow-up în timp finit, deci $T_{\max} = \infty$. □

Teoremă 2.7 (Conservarea populației totale)**.** *Pentru orice soluție a sistemului* (2.17)*,*

$$S(t) + I(t) + R(t) = N \quad \forall\, t \geq 0. \tag{2.18}$$

Demonstrație. Calculăm derivata sumei:

$$\begin{aligned} \frac{d}{dt}(S + I + R) &= \frac{dS}{dt} + \frac{dI}{dt} + \frac{dR}{dt} \\ &= -\frac{\beta SI}{N} + \left(\frac{\beta SI}{N} - \gamma I\right) + \gamma I \\ &= 0. \end{aligned}$$

Prin integrare, $S(t) + I(t) + R(t) = S_0 + I_0 + R_0 = N$. □

Teoremă 2.8 (Pozitivitate și mărginire)**.** *Dacă $S_0 > 0$, $I_0 > 0$, $R_0 \geq 0$, atunci:*

$$S(t) > 0, \quad I(t) > 0, \quad R(t) \geq 0, \quad \forall\, t > 0. \tag{2.19}$$

În particular, $S(t), I(t), R(t) \in [0, N]$ pentru orice $t \geq 0$.

Demonstrație. **(a) Pozitivitatea lui S:** Din ecuația $dS/dt = -(\beta I/N)S$, aceasta este o ecuație diferențială liniară omogenă în S cu coeficient $-\beta I(t)/N$. Soluția este:

$$S(t) = S_0 \cdot \exp\left(-\frac{\beta}{N}\int_0^t I(s)\, ds\right). \tag{2.20}$$

Deoarece integrala este finită și $S_0 > 0$, rezultă $S(t) > 0$ pentru orice $t \geq 0$ (presupunând $I(s) \geq 0$, demonstrat mai jos).

(b) Pozitivitatea lui I: Rescrierea ecuației a doua ca $dI/dt = (\beta S(t)/N - \gamma)\, I$ arată că aceasta este liniară în I:

$$I(t) = I_0 \cdot \exp\left(\int_0^t \left[\frac{\beta S(\tau)}{N} - \gamma \right] d\tau \right) > 0, \tag{2.21}$$

deoarece $I_0 > 0$ și exponențiala este strict pozitivă.

(c) Non-negativitatea lui R: Din $dR/dt = \gamma I(t) > 0$ și $R_0 \geq 0$, funcția R este strict crescătoare, deci $R(t) > R_0 \geq 0$.

Mărginirea urmează din Teorema 2.7: $S(t) \leq N$, $I(t) \leq N$, $R(t) \leq N$. □

Observație 2.9. Teorema 2.8 implică că domeniul pozitiv

$$\Omega = \{(S, I, R) \in \mathbb{R}^3_{\geq 0} : S + I + R = N\} \tag{2.22}$$

este *invariant pozitiv* față de fluxul sistemului (2.17). Prin urmare, este suficient să studiem comportamentul soluțiilor pe Ω, care este un simplex în $\mathbb{R}^3$.

Teoremă 2.10 (Monotonia compartimentelor)**.** *De-a lungul oricărei soluții a sistemului* (2.17)*:*

(i) $S(t)$ *este strict descrescătoare;*

(ii) $R(t)$ *este strict crescătoare;*

(iii) $I(t)$ *atinge un singur maxim în* $t^* = \inf\{t > 0 : S(t) = \gamma N/\beta\}$ *(dacă* $S_0 > \gamma N/\beta$*); altfel este descrescătoare.*

Demonstrație. (i) $dS/dt = -\beta SI/N < 0$ deoarece $S > 0$, $I > 0$. *(ii)* $dR/dt = \gamma I > 0$. *(iii)* $dI/dt = (\beta S/N - \gamma)I > 0 \Leftrightarrow S > \gamma N/\beta$. Deoarece S este strict descrescătoare, există cel mult un moment t^* când $S(t^*) = \gamma N/\beta$, care este maximul lui I. □

2.4.3 Analiza echilibrelor și stabilitate

Punctele de echilibru ale sistemului (2.17) sunt obținute din condiția $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$. Ecuațiile conduc la:

$$-\frac{\beta S^* I^*}{N} = 0, \quad \frac{\beta S^* I^*}{N} - \gamma I^* = 0, \quad \gamma I^* = 0. \tag{2.23}$$

Din a treia ecuație, $I^* = 0$. Substituind, obținem că orice punct de forma $(S^*, 0, N - S^*)$ cu $S^* \in [0, N]$ este echilibru. Mulțimea echilibrelor este:

$$\mathcal{E} = \{(S^*, 0, N - S^*) : S^* \in [0, N]\} \subset \Omega. \tag{2.24}$$

Observație 2.11. Spre deosebire de modelele SIS și SIRS, modelul SIR nu admite echilibru endemic cu $I^* > 0$. Aceasta reflectă faptul că, pe termen lung, epidemia se stinge întotdeauna (deși poate infecta o fracție mare din populație înainte de stingere).

Analiza stabilității locale. Deoarece $R = N - S - I$, sistemul se reduce la două ecuații (eliminând R):

$$\begin{cases} \frac{dS}{dt} = -\dfrac{\beta SI}{N}, \\ \frac{dI}{dt} = \dfrac{\beta SI}{N} - \gamma I. \end{cases} \tag{2.25}$$

Jacobianul sistemului (2.25) evaluat la un echilibru $(S^*, 0)$ este:

$$J(S^*, 0) = \begin{pmatrix} 0 & -\beta S^*/N \\ 0 & \beta S^*/N - \gamma \end{pmatrix}. \tag{2.26}$$

Valorile proprii sunt $\lambda_1 = 0$ și $\lambda_2 = \beta S^*/N - \gamma$.

Teoremă 2.12 (Stabilitatea echilibrului fără boală)**.** *Echilibrul fără boală $(S^*, 0)$ este stabil nelinear dacă $\beta S^*/N < \gamma$, adică dacă $S^*/N < 1/\mathcal{R}_0$, și este instabil dacă $\beta S^*/N > \gamma$.*

Demonstrație. Valorea proprie $\lambda_2 = \beta S^*/N - \gamma$ determină stabilitatea pe direcția I. Deoarece $\lambda_1 = 0$, analiza liniară este degenerată și trebuie completată cu argumente neliniare. Totuși, din Teorema 2.10, $I(t)$ este descrescătoare dacă $S(t) < \gamma N/\beta$ pentru tot $t > 0$, ceea ce se întâmplă când $S_0 < \gamma N/\beta$, implicând stingerea epidemiei. □

2.4.4 Numărul de reproducere de bază $\mathcal{R}_0$

2.4.4.1 Definiție prin matricea generației următoare

Metoda *matricei generației următoare* (next-generation matrix), formalizată de Diekmann, Heesterbeek și Metz [13] și extinsă de van den Driessche și Watmough [15], furnizează o procedură sistematică pentru calculul $\mathcal{R}_0$ în modele compartimentale de complexitate arbitrară.

Fie $\mathbf{x}_I$ vectorul compartimentelor infectate. Scriem:

$$\frac{d\mathbf{x}_I}{dt} = \mathcal{F}(\mathbf{x}) - \mathcal{V}(\mathbf{x}), \tag{2.27}$$

unde $\mathcal{F}_i$ reprezintă rata de apariție a noilor infecții în compartimentul i, iar $\mathcal{V}_i$ reprezintă rata netă de ieșire din compartimentul i prin toate celelalte procese.

Matricele Jacobiene evaluate la echilibrul fără boală $\mathbf{x}_0$ sunt:

$$F = \left[\frac{\partial \mathcal{F}_i}{\partial x_j}\right]_{\mathbf{x}_0}, \quad V = \left[\frac{\partial \mathcal{V}_i}{\partial x_j}\right]_{\mathbf{x}_0}. \tag{2.28}$$

Matricea generației următoare este $K = FV^{-1}$, iar:

$$\mathcal{R}_0 = \rho(K), \tag{2.29}$$

unde $\rho(\cdot)$ este raza spectrală (cea mai mare valoare proprie în modul).

2.4.4.2 Calculul $\mathcal{R}_0$ pentru modelul SIR

Pentru modelul SIR, singurul compartiment infectat este $\{I\}$. La echilibrul fără boală $(S, I, R) = (N, 0, 0)$:

$$\mathcal{F}(I) = \frac{\beta S I}{N} \implies F = \left[\frac{\partial \mathcal{F}}{\partial I}\right]_{S=N} = \frac{\beta N}{N} = \beta. \tag{2.30}$$

$$\mathcal{V}(I) = \gamma I \implies V = \gamma. \tag{2.31}$$

Matricea K este scalarul:

$$K = FV^{-1} = \frac{\beta}{\gamma}. \tag{2.32}$$

Prin urmare:

$$\boxed{\mathcal{R}_0 = \frac{\beta}{\gamma}.} \tag{2.33}$$

Interpretarea biologică este directă: $1/\gamma$ este durata medie a perioadei infecțioase, iar β este numărul mediu de contacte eficiente pe unitate de timp; produsul lor dă numărul de infecții secundare produse pe durata infecțiozității.

Teoremă 2.13 (Pragul epidemic)**.** *Considerăm o populație inițial complet susceptibilă ($S_0 \approx N$). Atunci:*

(i) Dacă $\mathcal{R}_0 > 1$, numărul de infectați crește inițial exponențial și are loc o epidemie ($I(t)$ atinge un maxim $I_{\max} > I_0$).

(ii) Dacă $\mathcal{R}_0 \leq 1$, $I(t)$ este imediat descrescătoare și nu are loc nicio epidemie.

Demonstrație. Din $dI/dt|_{t=0} = I_0(\beta S_0/N - \gamma) \approx I_0(\beta - \gamma)$ (pentru $S_0 \approx N$), concluzia urmează din semnul expresiei $\beta - \gamma$:

$$\left.\frac{dI}{dt}\right|_{t=0} > 0 \iff \beta > \gamma$$
$$\iff \frac{\beta}{\gamma} > 1 \iff \mathcal{R}_0 > 1. \qquad \square$$

2.4.5 Pragul de imunizare în turmă (herd immunity)

Definiție 2.14 (Imunitate de grup)**.** Imunitatea de grup (*herd immunity*) se referă la protecția indirectă a persoanelor susceptibile, generată atunci când o fracție suficient de mare din populație este imunizată (fie natural, fie prin vaccinare), astfel încât transmiterea susținută a agentului patogen devine imposibilă.

Epidemia se stinge când numărul de reproducere efectiv devine subunitar:

$$\mathcal{R}(t) = \mathcal{R}_0 \cdot \frac{S(t)}{N} < 1 \iff \frac{S(t)}{N} < \frac{1}{\mathcal{R}_0}. \tag{2.34}$$

Dacă o fracție p din populație este imunizată (cu $S/N = 1 - p$), condiția de blocare a epidemiei devine:

$$(1-p) \cdot \mathcal{R}_0 < 1 \iff p > 1 - \frac{1}{\mathcal{R}_0}. \tag{2.35}$$

Definiție 2.15 (Pragul de imunizare în turmă)**.** Fracția minimă a populației care trebuie imunizată pentru a preveni propagarea epidemiei este:

$$p_c = 1 - \frac{1}{\mathcal{R}_0}. \tag{2.36}$$

Exemplu 2.16. Pentru rujeolă, $\mathcal{R}_0 \approx 12$–18, deci $p_c \approx 92$–94%. Pentru COVID-19 (tulpina originală), $\mathcal{R}_0 \approx 2$–3, deci $p_c \approx 50$–67%. Variantele ulterioare (Delta, Omicron) au $\mathcal{R}_0$ mai mare, crescând corespunzător pragul de imunizare.

2.4.6 Ecuația dimensiunii finale a epidemiei

O caracteristică remarcabilă a modelului SIR este că permite calculul analitic al *dimensiunii finale* a epidemiei — fracția totală din populație care este infectată în cursul epidemiei — fără a rezolva explicit sistemul diferențial.

Teoremă 2.17 (Ecuația dimensiunii finale)**.** *Fie $z = 1 - S(\infty)/N$ fracția din populație infectată cumulativ pe durata epidemiei, presupunând $S_0 = N$ și $R_0 = 0$. Atunci z este unica rădăcină pozitivă a ecuației transcendentale:*

$$z = 1 - e^{-\mathcal{R}_0 z}. \tag{2.37}$$

Demonstrație. Împărțind ecuația pentru S la ecuația pentru R:

$$\frac{dS}{dR} = \frac{-\beta SI/N}{\gamma I} = -\frac{\beta}{\gamma N} S = -\frac{\mathcal{R}_0}{N} S. \tag{2.38}$$

Această ecuație separabilă în S și R se integrează direct:

$$\int_{S_0}^{S(t)} \frac{dS'}{S'} = -\frac{\mathcal{R}_0}{N} \int_0^{R(t)} dR', \tag{2.39}$$

$$\ln \frac{S(t)}{S_0} = -\frac{\mathcal{R}_0}{N} R(t). \tag{2.40}$$

Deci $S(t) = S_0 \exp(-\mathcal{R}_0 R(t)/N)$.

La $t \to \infty$, $I(\infty) = 0$ (demonstrat mai jos), deci $R(\infty) = N - S(\infty)$ (pentru $R_0 = 0$). Substituind cu $S_0 = N$:

$$S(\infty) = N e^{-\mathcal{R}_0 (N - S(\infty))/N}. \tag{2.41}$$

Notând $z = 1 - S(\infty)/N$ și împărțind prin N:

$$1 - z = e^{-\mathcal{R}_0 z} \implies z = 1 - e^{-\mathcal{R}_0 z}. \tag{2.42}$$

Că $I(\infty) = 0$: Funcția $R(t)$ este monoton crescătoare și mărginită de N, deci $\lim_{t\to\infty} R(t)$ există. Dacă $I(\infty) = \ell > 0$, atunci $R'(t) = \gamma I(t) \geq \gamma \ell > 0$ pentru t suficient de mare, ceea ce implică $R(t) \to \infty$, contradicție. Deci $I(\infty) = 0$.

Unicitatea rădăcinii pozitive: Fie $g(z) = 1 - e^{-\mathcal{R}_0 z} - z$. Avem $g(0) = 0$, $g'(0) = \mathcal{R}_0 - 1 > 0$ (dacă $\mathcal{R}_0 > 1$), și $g(1) = 1 - e^{-\mathcal{R}_0} - 1 = -e^{-\mathcal{R}_0} < 0$. Deoarece $g''(z) = -\mathcal{R}_0^2 e^{-\mathcal{R}_0 z} < 0$, funcția g este concavă, deci are exact o singură rădăcină în $(0, 1)$. □

Observație 2.18. Ecuația (2.37) este *transcendentală* și nu admite soluție în formă închisă pentru $\mathcal{R}_0 \neq 1$. Se rezolvă numeric, de exemplu prin metoda Newton-Raphson sau prin iterație de punct fix.

Corolar 2.19 (Rata de atac finală)**.** *Dacă $\mathcal{R}_0 > 1$, rata de atac finală z^* satisface $z^* \in (0, 1)$ și crește monoton cu $\mathcal{R}_0$. Limita $\lim_{\mathcal{R}_0 \to \infty} z^* = 1$.*

2.4.7 Extensii ale modelului SIR

Modelul SIR de bază poate fi extins în mai multe direcții. Introducerea nașterilor și mortalității naturale permite studiul echilibrelor endemice pe termen lung. Structura pe vârste poate fi inclusă prin matrice de contact, iar propagarea geografică prin modele cu patch-uri sau ecuații diferențiale parțiale. Pentru populații mici sau pentru fazele incipiente ale unei epidemii, modelele stochastice devin mai potrivite decât formularea deterministă. Intervențiile precum vaccinarea, izolarea și tratamentul se pot introduce ca fluxuri suplimentare între compartimente.

2.5 Metode Numerice pentru Ecuații Diferențiale Ordinare

2.5.1 Clasificarea metodelor numerice

Soluția analitică a sistemului SIR nu există în formă închisă (cu excepția cazurilor degenerate), ceea ce face indispensabilă rezolvarea numerică. O metodă numerică pentru problema cu valori inițiale (PVI):

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y}), \quad \mathbf{y}(t_0) = \mathbf{y}_0, \tag{2.43}$$

generează o secvență de aproximații $\mathbf{y}_n \approx \mathbf{y}(t_n)$ la punctele $t_n = t_0 + nh$.

Metodele numerice pot fi privite din mai multe unghiuri. Metodele explicite calculează direct $\mathbf{y}_{n+1}$ din valorile anterioare, în timp ce metodele implicite includ necunoscuta și în membrul drept, necesitând rezolvarea unui sistem la fiecare pas. Metodele un-pas folosesc doar starea curentă, pe când metodele multi-pas folosesc și stări anterioare. Ordinul de convergență p determină comportamentul erorii globale, $\|\mathbf{y}(t_n) - \mathbf{y}_n\| = \mathcal{O}(h^p)$. În sfârșit, metodele cu pas adaptiv ajustează h pe baza unei estimări a erorii locale, spre deosebire de schemele cu pas fix.

2.5.2 Metoda Euler explicită

Metoda Euler (1768) este cea mai simplă metodă numerică un-pas, obținută prin trunchierea seriei Taylor la ordinul întâi:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h \cdot \mathbf{f}(t_n, \mathbf{y}_n). \tag{2.44}$$

Teoremă 2.20 (Eroarea globală a metodei Euler)**.** *Dacă* $\mathbf{f}$ *este Lipschitz în* $\mathbf{y}$ *cu constanta* L *și* $\mathbf{y} \in C^2([t_0, T])$*, eroarea globală a metodei Euler satisface:*

$$\begin{aligned} \max_{n:t_n \le T} \|\mathbf{y}(t_n) - \mathbf{y}_n\| \le \frac{h}{2L} \max_{t\in[t_0,T]} \|\mathbf{y}''(t)\| \\ \cdot \left(e^{L(T-t_0)} - 1\right) = \mathcal{O}(h). \end{aligned} \tag{2.45}$$

Eroarea locală de trunchiere (pe un singur pas) este $\mathcal{O}(h^2)$, deci metoda are ordinul local 2 și ordinul global 1.

Aplicată sistemului SIR cu $\mathbf{y}_n = (S_n, I_n, R_n)^\intercal$, schema Euler este o iterație simplă peste pașii temporali: la fiecare pas se evaluează rata de variație $(\dot{S}, \dot{I}, \dot{R})$ folosind starea curentă și se actualizează compartimentele prin $\mathbf{y}_{n+1} = \mathbf{y}_n + h\,\mathbf{f}(t_n, \mathbf{y}_n)$.

Stabilitate. Analiza stabilității metodei Euler pe ecuația test $y' = \lambda y$ ($\lambda \in \mathbb{C}$) conduce la factorul de amplificare $z = 1 + h\lambda$. Metoda este stabilă dacă $|1 + h\lambda| \le 1$, ceea ce definește domeniul de stabilitate (disc de rază 1 centrat în -1 în planul $h\lambda$). Această restricție limiteaza pasul de integrare h, în special pentru probleme *stiff*.

2.5.3 Metoda Runge-Kutta de ordinul 4 (RK4)

Metodele Runge-Kutta sunt metode un-pas explicite care realizează mai multe evaluări ale funcției $\mathbf{f}$ la fiecare pas, atingând ordine superioare fără a recurge la derivate de ordin înalt ale soluției.

2.5.3.1 Derivare și formulă

Metoda RK4 clasică (cu 4 stadii) este dată de:

$$\begin{aligned}
\mathbf{k}_1 &= h \cdot \mathbf{f}(t_n, \mathbf{y}_n), \\
\mathbf{k}_2 &= h \cdot \mathbf{f}\Big(t_n + \tfrac{h}{2},\ \mathbf{y}_n + \tfrac{\mathbf{k}_1}{2}\Big), \\
\mathbf{k}_3 &= h \cdot \mathbf{f}\Big(t_n + \tfrac{h}{2},\ \mathbf{y}_n + \tfrac{\mathbf{k}_2}{2}\Big), \\
\mathbf{k}_4 &= h \cdot \mathbf{f}(t_n + h,\ \mathbf{y}_n + \mathbf{k}_3), \\
\mathbf{y}_{n+1} &= \mathbf{y}_n + \frac{1}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right).
\end{aligned} \tag{2.46}$$

Coeficienții metodei sunt reprezentați compact prin *tabloul Butcher*:

$$\begin{array}{c|cccc}
0 & & & & \\
1/2 & 1/2 & & & \\
1/2 & 0 & 1/2 & & \\
1 & 0 & 0 & 1 & \\
\hline
 & 1/6 & 1/3 & 1/3 & 1/6
\end{array} \tag{2.47}$$

Teoremă 2.21 (Ordinul metodei RK4)**.** *Metoda* (2.46) *are ordinul de convergență* $p = 4$*: dacă* $\mathbf{f} \in C^4$ *și* $\mathbf{y} \in C^5([t_0, T])$*, atunci eroarea globală satisface:*

$$\max_n \|\mathbf{y}(t_n) - \mathbf{y}_n\| = \mathcal{O}(h^4). \tag{2.48}$$

2.5.3.2 Schiță de demonstrație.

Eroarea locală de trunchiere τ_n se obține substituind soluția exactă $\mathbf{y}(t_n)$ în schema (2.46) și expandând în serie Taylor. Primele patru condiții de ordin (condițiile de arbore Butcher pentru $p = 4$) sunt satisfăcute de coeficienții (2.47), deci $\tau_n = \mathcal{O}(h^5)$. Acumularea pe $\sim 1/h$ pași conduce la o eroare globală $\mathcal{O}(h^4)$. Tratare completă în [16].

2.5.4 Metode Runge-Kutta adaptive (RK45 / Dormand-Prince)

Metodele cu pas fix impun utilizatorului alegerea unui h adecvat, care este o sarcină dificilă în practică. Metodele *adaptive* ajustează automat pasul h pe baza unei estimări a erorii locale, asigurând că eroarea rămâne sub o toleranță prestabilită ε.

2.5.4.1 Metode Runge-Kutta încorporate

O *pereche RK încorporată* de ordine p și $p+1$ calculează doi estimatori pentru $\mathbf{y}_{n+1}$ folosind *același* set de evaluări:

$$\hat{\mathbf{y}}_{n+1} = \mathbf{y}_n + h \sum_{i=1}^{s} b_i \mathbf{k}_i \quad \text{(ordinul } p\text{)}, \tag{2.49}$$

$$\tilde{\mathbf{y}}_{n+1} = \mathbf{y}_n + h \sum_{i=1}^{s} \tilde{b}_i \mathbf{k}_i \quad \text{(ordinul } p+1\text{)}, \tag{2.50}$$

cu $\mathbf{k}_i = \mathbf{f}(t_n + c_i h, \mathbf{y}_n + h\sum_{j<i} a_{ij}\mathbf{k}_j)$. Estimatorul de eroare locală este:

$$\mathbf{e}_{n+1} = \tilde{\mathbf{y}}_{n+1} - \hat{\mathbf{y}}_{n+1} = h\sum_{i=1}^{s}(b_i - \tilde{b}_i)\mathbf{k}_i. \tag{2.51}$$

2.5.4.2 Metoda Dormand-Prince (DOPRI5 / RK45)

Perechea **Dormand-Prince** [17], implementată ca `ode45` în MATLAB, `RK45` în SciPy (Python), `Tsit5` în Julia și `lsoda`/`ode45` în R (pachetul `deSolve`), utilizează 6 evaluări de funcție și are ordinul principal 5 cu estimator de eroare de ordinul 4.

Tabloul Butcher complet al metodei Dormand-Prince este:

0							
1/5	1/5						
3/10	3/40	9/40					
4/5	44/45	−56/15	32/9				
8/9	19372/6561	−25360/2187	64448/6561	−212/729			
1	9017/3168	−355/33	46732/5247	49/176	−5103/18656		
1	35/384	0	500/1113	125/192	−2187/6784	11/84	
$\hat{b}$	35/384	0	500/1113	125/192	−2187/6784	11/84	0
$\tilde{b}$	5179/57600	0	7571/16695	393/640	−92097/339200	187/2100	1/40

(2.52)

2.5.4.3 Control adaptiv al pasului

Pasul h este actualizat după fiecare pas pe baza normei erorii:

$$\text{err} = \left\| \frac{\mathbf{e}_{n+1}}{\text{atol} + \text{rtol} \cdot |\mathbf{y}_{n+1}|} \right\|_{\infty}, \tag{2.53}$$

unde atol și rtol sunt toleranțe absolute și relative. Pasul următor este calculat prin formula:

$$h_{\text{nou}} = h \cdot \min\Big(f_{\max},\ \max\Big(f_{\min},\ S \cdot \text{err}^{-1/(p+1)}\Big)\Big), \tag{2.54}$$

cu factorul de siguranță $S = 0.9$ și limitele $f_{\min} = 0.1$, $f_{\max} = 10$.

2.5.5 Stabilitate numerică și probleme stiff

Definiție 2.22 (Problemă stiff)**.** O problemă cu valori inițiale (2.43) este numită *stiff* dacă valorile proprii ale Jacobianului $\partial\mathbf{f}/\partial\mathbf{y}$ au componente cu parte reală negativă de magnitudine mare, impunând pași de integrare h extrem de mici pentru metodele explicite din motive de stabilitate, chiar dacă soluția variază lent.

Aplicație la modelul SIR. Jacobianul sistemului SIR (2.25) este:

$$J(S, I) = \begin{pmatrix} -\beta I/N & -\beta S/N \\ \beta I/N & \beta S/N - \gamma \end{pmatrix}. \tag{2.55}$$

Valorile proprii depind de (S, I). La debutul epidemiei ($S \approx N$, I mic), valorile proprii sunt aproximativ 0 și $\beta - \gamma$, deci problema nu este stiff. Modelul SIR de bază nu este stiff pentru parametri tipici epidemiologici. Totuși, extensiile cu mai multe compartimente și rate foarte diferite pot produce probleme stiff, necesitând metode implicite (Runge-Kutta implicit, BDF).

Observație 2.23. Metodele implicite (de exemplu, metoda Euler implicită, Crank-Nicolson, Radau) au domenii de stabilitate nelimitate și sunt preferate pentru probleme stiff, în ciuda costului computațional superior (rezolvarea unui sistem nonlinear la fiecare pas).

2.6 Date Epidemiologice — Surse și Preprocesare

2.6.1 Surse de date epidemiologice

Implementarea și validarea modelelor epidemiologice computaționale necesită date empirice de calitate. Sursele folosite frecvent includ rapoartele Organizației Mondiale a Sănătății, seriile de supraveghere publicate de ECDC, setul de date COVID-19 al Johns Hopkins University CSSE și colecțiile armonizate de Our World in Data. Această lucrare folosește două seturi OWID complementare: setul COVID-19 (sursa principală, cu acoperire globală pe 2020–2023) și setul mpox (focar 2022–2024, cu o dinamică complet diferită ca scară, durată infecțioasă și raportare). Ambele oferă serii temporale publice, comparabile între țări și ușor de prelucrat într-un format comun.

2.6.2 Structura datelor și preprocesare

Datele epidemiologice brute conțin frecvent valori lipsă, raportări retrospective, efecte de weekend și schimbări administrative ale unităților geografice. Înainte de utilizarea lor într-un model compartimental, seriile trebuie ordonate temporal, corectate pentru valori imposibile și aduse la o granularitate compatibilă cu pasul de integrare ales. În caz contrar, erorile de raportare pot fi confundate cu variații reale ale dinamicii infecției.

2.6.3 Estimarea parametrilor din date reale

Parametrii β și γ ai modelului SIR pot fi estimați din date observate prin minimizarea erorii pătratice dintre seria raportată și soluția numerică, prin estimare de verosimilitate maximă cu un model de zgomot adecvat sau prin inferență bayesiană. Alegerea metodei depinde de natura datelor: pentru incidență zilnică, modelele Poisson sau binomial negativ sunt mai potrivite decât o eroare gaussiană simplă, în timp ce inferența bayesiană oferă o descriere explicită a incertitudinii.

Estimatorul $\hat{\gamma}$ poate fi obținut direct dacă durata medie a perioadei infecțioase este disponibilă din studii clinice: $\hat{\gamma} = 1/T_{\text{infecțios}}$. Parametrul β este mai dificil de estimat și depinde de comportamentul social, intervenții și variante ale agentului patogen.

Capitolul 3
Tehnologii și Instrumente Utilizate

3.1 Limbaje de programare pentru calcul științific

Alegerea limbajului de programare pentru implementarea modelelor matematice nu este neutră. Performanța, expresivitatea și ecosistemul de biblioteci influențează atât timpul de dezvoltare, cât și comportamentul numeric al aplicației. În această lucrare, R, Julia și Python sunt tratate ca ecosisteme de calcul științific, nu doar ca sintaxe diferite pentru aceleași formule.

3.1.1 R

3.1.1.1 Istoric și filosofie

R este un limbaj de programare și mediu software pentru calcul statistic și grafică, descendent direct al limbajului S (Bell Laboratories, 1976). A fost creat de Ross Ihaka și Robert Gentleman la Universitatea din Auckland în 1993, iar în 1997 a fost adoptat ca proiect open-source de *R Development Core Team* [18]. Filosofia R este centrată pe date: vectorii, matricile și cadrele de date (data frames) sunt cetățeni de primă clasă, iar gramatica limbajului este proiectată pentru manipularea și analiza datelor.

3.1.1.2 Caracteristici tehnice relevante

R este un limbaj *interpretat*, dinamic tipizat, cu suport pentru programare funcțională, procedurală și orientată pe obiecte prin sistemele S3, S4 și R6. Pentru calcul numeric, avantajul său principal vine din vectorizarea nativă: operațiile pe vectori și matrice sunt executate frecvent în cod C sau Fortran, evitând buclele interpretate. În același timp, modelul de memorie cu garbage collector și copiere la scriere poate introduce costuri vizibile atunci când sunt manipulate structuri mari. Posibilitatea de a apela cod nativ prin `.C()`, `.Call()` și `.Fortran()` explică de ce multe pachete R performante ascund implementări compilate în spatele unei interfețe statistice convenabile.

3.1.1.3 Pachete relevante pentru modele compartimentale

Pentru modelele compartimentale implementate, pachetul relevant este `deSolve` [4], care oferă funcția `ode()` și acces la metode consacrate precum `lsoda`, `ode45` și `rk4`. În jurul lui se pot folosi pachete precum `ggplot2` pentru vizualizare, `data.table` pentru prelucrare tabulară și `bench` sau `microbenchmark` pentru măsurarea timpilor de execuție. În implementarea sistemului, runner-ul R folosește `deSolve` pentru integrarea numerică, iar vizualizarea este lăsată aplicației web.

3.1.2 Julia

3.1.2.1 Istoric și filosofie

Julia a fost creat la MIT de Jeff Bezanson, Alan Edelman, Stefan Karpinski și Viral B. Shah, cu versiunea 1.0 lansată în august 2018 [19]. Motivația proiectului a fost depășirea *problemei celor două limbaje* (two-language problem): cercetătorii scriu prototipuri în R/Python (productivitate ridicată, viteză scăzută) și re-implementează codul critic în C/Fortran (viteză ridicată, productivitate scăzută). Julia vizează ambele obiective simultan.

3.1.2.2 Compilare Just-In-Time și sistemul de tipuri

Julia utilizează compilare **JIT** (Just-In-Time) bazată pe LLVM, cu o particularitate fundamentală: specializarea pe tipuri concrete. Când o funcție este apelată pentru prima dată cu argumente de tipuri specifice, compilatorul generează cod mașină optimizat pentru acele tipuri exacte. Apelurile ulterioare cu aceleași tipuri rulează direct codul compilat, apropiind performanța de C.

Mecanismul de *multiple dispatch* permite definirea mai multor metode pentru aceeași funcție, în funcție de tipurile argumentelor. Împreună cu sistemul de tipuri parametric, acesta permite compilatorului să genereze cod specializat pentru cazurile folosite efectiv. Macrocomenzile completează acest model prin posibilitatea de a transforma codul la momentul parsării, o tehnică utilizată frecvent în ecosistemul științific Julia.

3.1.2.3 DifferentialEquations.jl

Ecosistemul **SciML** (Scientific Machine Learning) al Julia conține pachetul `DifferentialEquations.jl` [5], un framework open-source matur pentru rezolvarea ecuațiilor diferențiale. Pachetul include solver-e pentru ecuații diferențiale ordinare, parțiale, stochastice și cu întârziere. Pentru probleme nestiff, metode precum `Tsit5` și `Vern7/9` sunt folosite frecvent, iar pentru probleme stiff există familii precum `Rodas`. Ecosistemul SciML integrează și diferențiere automată, ceea ce îl face relevant pentru estimarea parametrilor și identificarea sistemelor dinamice.

3.1.3 Python

3.1.3.1 Istoric și filosofie

Python a fost creat de Guido van Rossum și lansat în 1991 [20]. Filosofia sa, rezumată în *Zen of Python* (`PEP 20`), accentuează lizibilitatea și simplitatea codului. Python a devenit limbajul dominant în știința datelor și machine learning datorită ecosistemului său extins și comunității masive.

3.1.3.2 Caracteristici tehnice și ecosistem științific

Python este un limbaj interpretat, dinamic tipizat, multi-paradigmă. Principala limitare pentru calcul intensiv este **GIL** (Global Interpreter Lock), care previne execuția paralelă a thread-urilor Python pure. Performanța este recuperată prin biblioteci care rulează cod nativ C sau Fortran în afara GIL. NumPy oferă calcul vectorizat pe array-uri dense, SciPy [6] adaugă algoritmică științifică și modulul `scipy.integrate`, iar Pandas este folosit pe scară largă pentru date tabulare. Pentru sistemul dezvoltat aici, componenta esențială este `solve_ivp`, care expune metode precum RK45, DOP853, Radau, BDF și LSODA.

Tabela 3.1: Comparație a limbajelor de programare utilizate

Criteriu	**R**	**Julia**	**Python**
Licență	Open-source	Open-source	Open-source
Paradigmă	Funcț./OOP	Multi	Multi
Compilare	Interpretat	JIT (LLVM)	Interpretat
Solver EDO	deSolve	DiffEq.jl	SciPy
Performanță rel.	Medie	Înaltă	Medie
Maturitate ecos.	Înaltă	Medie	Înaltă
Utilizare acad.	Ridicată	Medie	Ridicată

3.1.3.3 Funcțiile de rezolvare a EDO în SciPy

Interfața `solve_ivp` din `scipy.integrate` este apelată printr-o funcție RHS care descrie modelul selectat. Exemplul următor folosește cazul SIR, deoarece are doar trei variabile de stare; pentru SIDARTHE, aceeași interfață primește un vector de opt compartimente și un dicționar mai mare de parametri.

```python
from scipy.integrate import solve_ivp
import numpy as np

def sir_model(t, y, beta, gamma, N):
    S, I, R = y
    dSdt = -beta * S * I / N
    dIdt =  beta * S * I / N - gamma * I
    dRdt =  gamma * I
    return [dSdt, dIdt, dRdt]

# Parametri
N = 1_000_000; beta = 0.3; gamma = 0.1
S0, I0, R0 = N - 1, 1, 0
t_span = (0, 160); t_eval = np.linspace(0, 160, 1601)

sol = solve_ivp(
    fun=lambda t, y: sir_model(t, y, beta, gamma, N),
    t_span=t_span,
    y0=[S0, I0, R0],
    method='RK45',
    t_eval=t_eval,
    rtol=1e-8,
    atol=1e-10
)
```

Listing 3.1: Interfața solve_ivp din SciPy pentru un sistem compartimental SIR

3.1.4 Analiză comparativă a limbajelor

Table 3.1 sintetizează principalele caracteristici ale celor trei limbaje din perspectiva utilizării pentru implementarea modelelor compartimentale implementate în sistem.

3.2 Baze de date distribuite — YugabyteDB

3.2.1 Concepte de baze de date distribuite

O *bază de date distribuită* este o colecție de date distribuite fizic pe mai multe noduri (mașini sau locații geografice), gestionată de un sistem software care oferă transparența localizării:

aplicațiile interacționează cu baza de date ca și cum ar fi centralizată.

Definiție 3.1 (Teorema CAP)**.** Teorema CAP (Brewer, 2000) afirmă că un sistem distribuit nu poate garanta simultan mai mult de două dintre următoarele trei proprietăți: *consistency*, adică nodurile observă aceeași stare logică a datelor; *availability*, adică cererile valide primesc răspuns chiar în condiții de degradare; și *partition tolerance*, adică sistemul continuă să funcționeze în prezența unor întreruperi de comunicare între noduri.

Bazele de date distribuite precum YugabyteDB, CockroachDB și Spanner optează pentru **CP** (consistency + partition tolerance), sacrificând disponibilitatea perfectă în favoarea coerenței datelor.

3.2.2 Arhitectura YugabyteDB

YugabyteDB [21] este o bază de date distribuită open-source, construită pentru a combina caracteristicile PostgreSQL cu scalabilitatea orizontală și toleranța la erori a sistemelor NoSQL. Arhitectura sa se bazează pe doi piloni principali:

1. Stratul de stocare DocDB. DocDB este motorul de stocare distribuit al YugabyteDB, bazat pe arhitectura **LSM-tree** (Log-Structured Merge-tree) derivată din **RocksDB**. Datele sunt stocate în *tablete* — unități de sharding distribuite pe noduri. Fiecare tabletă este replicată pe cel puțin trei noduri prin protocolul Raft. Scrierile trec printr-un lider, sunt replicate către nodurile următoare și sunt confirmate după ce o majoritate de replici a persistat modificarea.

2. Nivelurile de interfață API. Interfața **YSQL** este compatibilă cu PostgreSQL și suportă SQL, tranzacții ACID distribuite, join-uri și indecși. Interfața **YCQL** este compatibilă cu Apache Cassandra și este orientată spre workload-uri wide-column. În această lucrare este folosită interfața YSQL, pentru compatibilitatea cu driver-ele PostgreSQL disponibile în Go, R, Julia și Python.

3.2.3 Sharding și replicare

YugabyteDB suportă sharding prin hash și sharding pe intervale. Hash sharding distribuie uniform cheia primară pe tablete și reduce riscul de noduri fierbinți, dar este mai puțin potrivit pentru scanări ordonate pe intervale. Range sharding păstrează vecinătatea valorilor și este util pentru serii temporale, însă poate concentra scrierile dacă toate valorile noi ajung în același interval.

Observație 3.2. Pentru datele epidemiologice cu interogări frecvente pe intervale de date (serii temporale), range sharding pe coloana `timestamp` sau `date` este strategia recomandată, împreună cu un index secundar pe codul bolii și/sau regiune geografică.

3.3 Orchestrarea containerelor — Kubernetes

3.3.1 Containere și nevoia de orchestrare

Containerizarea (Docker) permite împachetarea unei aplicații împreună cu toate dependențele sale într-o unitate portabilă și reproductibilă. Pe măsură ce sistemele cresc în complexitate și numărul de containere depășește câteva zeci, gestionarea manuală devine infezabilă: unde rulează fiecare container? Cum se repornește dacă cade? Cum se scalează?

Kubernetes (K8s) [22, 23] este platforma open-source de orchestrare a containerelor, inițial dezvoltată la Google pe baza experienței cu sistemul intern Borg, și donată CNCF (Cloud Native Computing Foundation) în 2014.

3.3.2 Arhitectura Kubernetes

Un cluster Kubernetes este împărțit între planul de control și nodurile worker. Planul de control expune API-ul clusterului prin `kube-apiserver`, păstrează starea în `etcd`, programează pod-urile prin `kube-scheduler` și rulează controllerele care aduc starea curentă spre starea dorită. Pe nodurile worker, `kubelet` gestionează ciclul de viață al pod-urilor, `kube-proxy` configurează accesul de rețea la servicii, iar container runtime-ul pornește efectiv containerele.

3.3.3 Resurse Kubernetes relevante

Resursele folosite cel mai des sunt `Pod`, `Deployment`, `StatefulSet`, `Service`, `PersistentVolumeClaim`, `ConfigMap` și `Secret`. Pentru API și frontend-uri sunt suficiente Deployment-uri, deoarece instanțele nu păstrează stare locală. Pentru YugabyteDB este necesar un StatefulSet, care oferă identitate stabilă și volume persistente pentru fiecare replică.

3.4 Backend API — Go

3.4.1 Caracteristici și filosofia limbajului Go

Go (cunoscut și ca Golang) este un limbaj de programare open-source creat la Google de Robert Griesemer, Rob Pike și Ken Thompson, cu prima versiune stabilă în 2012 [24]. Go a fost proiectat pentru productivitate la scară mare, cu accent pe simplitate, compilare rapidă și concurență structurată.

Pentru un API backend, Go are câteva proprietăți utile: compilează rapid la cod mașină, produce binare statice ușor de containerizat, oferă concurență prin goroutine și canale, iar biblioteca standard include un pachet HTTP suficient de solid pentru servicii REST. Interfețele structurale reduc cuplarea dintre module, iar garbage collectorul concurrent menține latențe acceptabile pentru un API care nu face calcul numeric intensiv în procesul HTTP.

3.4.2 Ecosistem pentru API-uri REST

Deși `net/http` din biblioteca standard este suficient pentru API-uri simple, proiectul folosește routerul `chi`, o bibliotecă idiomatică și redusă ca suprafață, care oferă rutare ierarhică, middleware-uri compozabile și integrare directă cu tipurile standard `http.Handler`. Această alegere evită un framework mai greu și se potrivește cu dimensiunea backend-ului implementat.

3.5 Frontend — Vue.js

3.5.1 Arhitectura Vue 3 și Composition API

Vue.js [25] este un framework JavaScript progresiv pentru construirea interfețelor utilizator, creat de Evan You în 2014. Vue 3 (2020) introduce **Composition API**, o alternativă la Options API care permite organizarea logicii prin funcționalitate (nu prin tip — data, methods, computed) și reutilizarea logicii prin *composables*.

Arhitectura Vue 3 combină un sistem reactiv bazat pe `Proxy`, un Virtual DOM pentru reconcilierea arborelui de interfață și Single File Components, în care template-ul, logica TypeScript și stilurile CSS sunt grupate în fișiere `.vue`. Pentru aplicația de față, această structură permite păstrarea componentelor de formular, navigare și grafic în unități ușor de urmărit.

3.5.2 Ecosistem și biblioteci utilizate

În implementare sunt folosite Vue Router pentru rutare client-side, Pinia pentru starea globală, Axios pentru apelurile REST și Chart.js, prin `vue-chartjs`, pentru reprezentarea curbelor compartimentale ale modelului selectat. Componenta de grafic construiește dinamic seriile în funcție de cheile prezente în obiectul JSONB `compartments`, astfel încât același cod afișează atât traiectoriile $S(t)$, $I(t)$, $R(t)$ ale modelului SIR, cât și cele opt traiectorii ale modelului SIDARTHE. Aceste biblioteci acoperă nevoile aplicației fără a introduce un framework frontend suplimentar peste Vue.

Capitolul 4
Arhitectura Sistemului E-Medicine

4.1 Vedere de ansamblu

Sistemul e-medicine propus în această lucrare este o aplicație distribuită, organizată în componente independente, care permite utilizatorilor să organizeze date epidemiologice pe studii, să încarce seturi de date reale, să declanșeze simulări pentru SIDARTHE, SIR, SIS, SEIR și SIRS în trei limbaje de programare distincte — R, Julia și Python — și să compare rezultatele atât din punct de vedere numeric, cât și al performanței computaționale. Arhitectura globală este organizată în patru straturi funcționale, ilustrate în Fig. 4.1.

Stratul de prezentare constă din două aplicații web single-page (SPA) implementate în Vue.js și servite prin containere Nginx: aplicația principală pentru utilizatorii unui studiu și consola separată pentru super-admini. **Stratul API** este un server REST implementat în Go, responsabil cu autentificarea prin JWT, autorizarea pe baza rolurilor din studii, orchestrarea job-urilor de simulare și medierea accesului la baza de date. **Stratul de calcul** cuprinde runner-ele compartimentale R, Julia și Python, plus runner-ul LSTM dedicat predicției pe serii temporale; acestea comunică asincron cu API-ul prin cozi Redis separate. **Stratul de stocare** este asigurat de YugabyteDB, care persistă atât datele epidemiologice de intrare, cât și rezultatele simulărilor. În mediul local, componentele sunt pornite prin Docker Compose; pentru un deployment de producție, aceeași separare a responsabilităților permite descrierea resurselor în Kubernetes.

Fluxul principal de date începe cu autentificarea utilizatorului și obținerea unui token JWT. Utilizatorul selectează un studiu în care este membru, iar toate operațiile ulterioare sunt evaluate în contextul acelui `study_id`. În funcție de rolul său (`admin`, `researcher`, `contributor` sau `viewer`), el poate încărca un fișier CSV cu date epidemiologice, poate porni simulări sau poate administra membrii studiului. API-ul validează structura fișierului, stochează metadatele în `datasets`, înregistrările brute în `epidemiological_records` și păstrează toate resursele scope-uite pe studiu. La inițierea unei simulări, utilizatorul specifică modelul, limbajul dorit, parametrii, populația N și orizontul $t_{\max}$; API-ul creează o înregistrare în `simulation_runs` cu statusul `pending` și publică un mesaj JSON în coada Redis corespunzătoare limbajului ales. Runner-ul consumă job-ul, execută simularea cerută, scrie traiectoriile compartimentale în `simulation_results` și actualizează statusul la `completed`. Frontend-ul interoghează periodic API-ul și afișează graficele imediat ce simularea este finalizată.

Figura 4.2 prezintă aceeași secvență sub forma unei diagrame temporale, în care timpul curge de sus în jos, evidențiind decuplarea între răspunsul sincron al API-ului și execuția asincronă din runner.

4.2 Cerințe funcționale și non-funcționale

4.2.1 Cerințe funcționale

Prima cerință funcțională privește **autentificarea și autorizarea**: utilizatorii trebuie să se poată înregistra și autentifica, iar toate operațiile asupra datelor și simulărilor sunt protejate

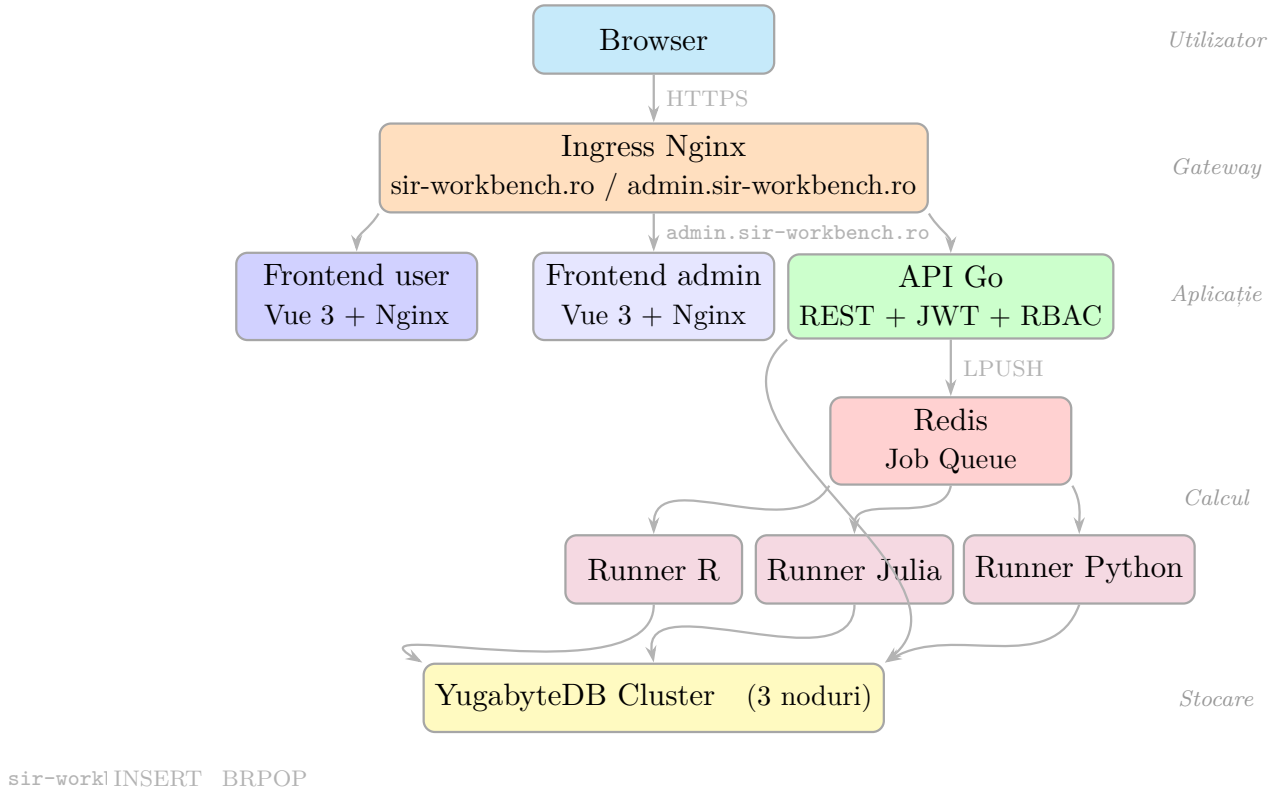


Figura 4.1: Arhitectura globală a sistemului e-medicine. Săgețile indică direcția de flux a datelor: browserul accesează aplicația prin Ingress, care rutează cererile către cele două SPA-uri sau API; API-ul publică job-uri în Redis și le consumă runner-ele dedicate; atât API-ul, cât și runner-ele accesează același cluster YugabyteDB.

prin token JWT cu durată de valabilitate configurabilă. Autorizarea nu este doar globală, ci și contextuală: resursele sunt grupate în studii, iar accesul depinde de rolul utilizatorului în studiul curent. Pe lângă aceste roluri există flag-ul `is_super_admin`, folosit pentru consola administrativă și pentru vizibilitate la nivelul tuturor studiilor.

Gestionarea studiilor epidemiologice permite crearea unui context de lucru cu nume, cod de boală și descriere. Creatorul devine automat `admin` al studiului și poate adăuga membri cu rolurile `researcher`, `contributor` sau `viewer`. Această separare permite ca aceiași utilizatori să participe în mai multe studii cu drepturi diferite.

Gestionarea seturilor de date epidemiologice presupune că un utilizator cu rol `admin` sau `contributor` poate încărca fișiere CSV ce conțin serii temporale epidemiologice — date calendaristice, numere de cazuri, populație, regiune geografică. Sistemul validează structura, calculează numărul de înregistrări și stochează datele persistent în baza de date, asociindu-le cu studiul curent.

Configurarea și declanșarea simulărilor permite utilizatorului cu rol `admin` sau `researcher` să selecteze un set de date din studiul curent, să aleagă limbajul de execuție (R, Julia sau Python) și să introducă tipul modelului, parametrii și dimensiunea populației N. Sistemul execută simularea asincron, printr-un runner dedicat, și notifică finalizarea fără a bloca interfața.

Vizualizarea rezultatelor presupune afișarea curbelor compartimentale ca grafice interactive, indiferent dacă simularea are două compartimente, ca SIS, trei compartimente, ca SIR/SIRS, patru compartimente, ca SEIR, sau opt compartimente, ca SIDARTHE.

Compararea inter-limbaj permite utilizatorului să selecteze mai multe simulări efectuate pe același set de date, cu aceiași parametri, în limbaje diferite, și să compare vizual atât forma

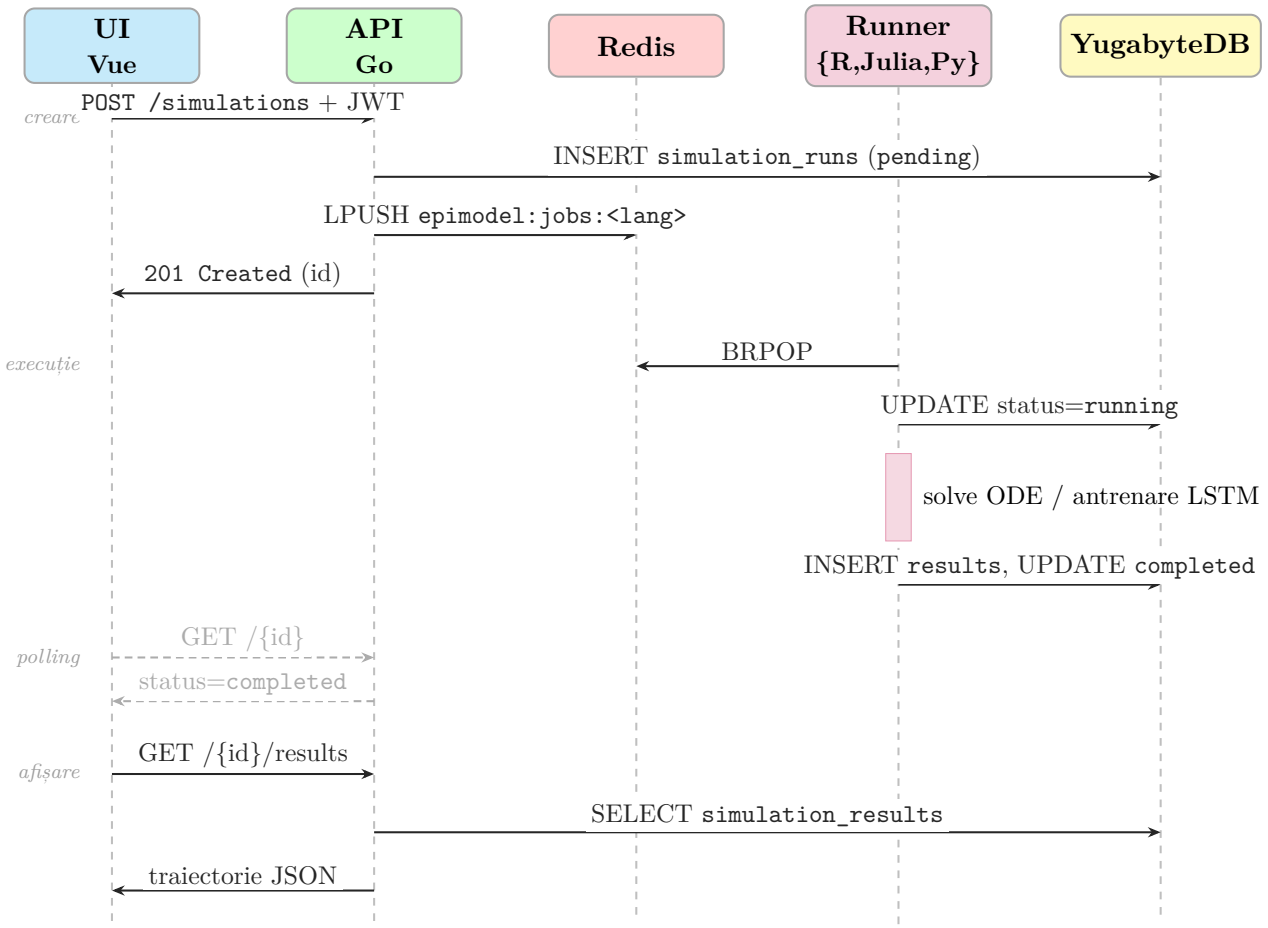


Figura 4.2: Ciclul de viață al unei simulări, de la cererea utilizatorului în UI până la afișarea rezultatului. Timpul curge de sus în jos. Săgețile pline indică mesaje sincrone HTTP/queue/SQL; săgețile întrerupte indică polling-ul repetat al statusului până la finalizare. API-ul răspunde `201 Created` *înainte* ca solver-ul să ruleze — întreaga execuție numerică se petrece asincron în runner, decuplată prin Redis. Aceasta este complementară Figurii 4.1, care arată topologia: cine vorbește cu cine; aici se vede *când*.

curbelor, cât și metricile de performanță (timp de execuție, memorie utilizată).

Exportul rezultatelor oferă posibilitatea de a descărca seria temporală a unei simulări în format CSV.

Monitorizarea benchmark-urilor înseamnă că sistemul colectează automat timpii de execuție (wall-clock și CPU) și consumul de memorie pentru fiecare rulare și le expune printr-un endpoint dedicat, accesibil în interfața de comparare.

Administrarea globală este separată de interfața principală. Utilizatorii marcați `is_super_admin=true` pot accesa consola `admin.sir-workbench.ro`, unde văd statistici globale, lista utilizatorilor cu promovarea sau revocarea drepturilor de super-admin, lista tuturor studiilor și pagina de detaliu pentru orice studiu, cu membrii, seturile de date, simulările (cu badge de status) și benchmark-urile agregate. Dashboard-ul include distribuțiile agregate între studii (simulări pe status, model și limbaj; seturi de date și studii pe boală; top studii după volum), iar o pagină dedicată afișează feed-ul de activitate recentă (simulări și încărcări de seturi de date, indiferent de studiu). Toate paginile administrative sunt strict de citire; mutațiile rămân în fluxul aplicației principale, controlate prin RBAC pe studiu.

4.2.2 Cerințe non-funcționale

Din punct de vedere al performanței, sistemul trebuie să păstreze scurte operațiile sincrone ale API-ului. Crearea unei simulări nu execută modelul în procesul HTTP, ci doar validează cererea, persistă metadatele și publică job-ul în Redis. Durata simulării este tratată separat, la nivel de runner.

Disponibilitatea depinde de modul de deployment. În varianta Docker Compose, sistemul este potrivit pentru dezvoltare și testare locală. Într-un deployment Kubernetes, API-ul poate fi replicat, iar YugabyteDB poate rula cu factor de replicare 3, tolerând pierderea unui nod fără pierdere de date confirmate.

Scalabilitatea este obținută prin design-ul stateless al API-ului și prin arhitectura bazată pe coadă: numărul de replici ale oricărui runner poate fi crescut orizontal prin simpla modificare a câmpului `replicas` în Deployment-ul Kubernetes, fără modificări de cod; YugabyteDB suportă adăugarea de noduri fără downtime.

Securitatea implică protejarea tuturor endpointurilor sensibile prin JWT, gestionarea secretelor (chei JWT, parole DB) prin obiecte Kubernetes `Secret` — montate ca variabile de mediu, nu incluse în imaginile Docker — și aplicarea principiului privilegiilor minime la nivelul ServiceAccount-urilor Kubernetes.

Extensibilitatea garantează că adăugarea unui nou limbaj sau a unui nou model compatibil necesită doar crearea unui nou container runner care respectă interfața de comunicare cu coada Redis, fără modificarea API-ului sau a schemei bazei de date.

Portabilitatea este asigurată prin containerizarea (Docker) a tuturor componentelor și descrierea lor declarativă prin manifeste Kubernetes, permițând deployment pe orice furnizor de cloud sau infrastructură on-premises care suportă Kubernetes 1.28+.

4.3 Arhitectura bazei de date distribuite

4.3.1 Schema relațională

Baza de date a sistemului este structurată în jurul conceptului de studiu. Schema include tabele pentru identitate, membership, resurse epidemiologice, simulări și benchmark-uri, iar relațiile principale sunt ilustrate în Fig. 4.3. În forma curentă, tabelele aplicative principale sunt: `users`, `studies`, `study_members`, `datasets`, `epidemiological_records`, `simulation_runs`, `simulation_results` și `benchmarks`. Resursele de lucru (`datasets` și `simulation_runs`) conțin explicit `study_id`, astfel încât autorizarea și interogările să fie făcute în contextul unui studiu.

Tabelul `users` stochează datele de autentificare ale utilizatorilor; parola nu este reținută în clar, ci ca hash bcrypt cu factor de cost 12. Câmpul `is_super_admin` acordă acces la consola administrativă și la vizibilitatea globală asupra studiilor. Tabelul `studies` definește unitatea de izolare logică a datelor, iar `study_members` leagă utilizatorii de studii prin unul dintre rolurile `admin`, `researcher`, `contributor` sau `viewer`. Tabelul `datasets` conține metadatele seturilor de date epidemiologice încărcate: autorul, studiul, codul bolii, data încărcării și numărul total de înregistrări. Tabelul `epidemiological_records` stochează valorile brute ale seriei temporale; câmpurile `susceptible`, `infected` și `recovered` conțin valorile observate sau valorile inițiale folosite la calibrarea parametrilor, iar `region_code` urmează standardul ISO 3166-1 alpha-3 în seturile de date preprocesate din OWID. Pe lângă acestea, schema include și 9 coloane epidemiologice opționale (`deaths`, `new_cases`, `new_deaths`, `hospitalized`, `icu`, `vaccinated`, `tested`, `stringency_index`, `reproduction_rate`), populate atunci când sursa de date le oferă; aceste metrici nu sunt folosite de runner-ele built-in, dar sunt expuse integral scripturilor user. Tabelul `simulation_runs` reprezintă job-ul de simulare și include

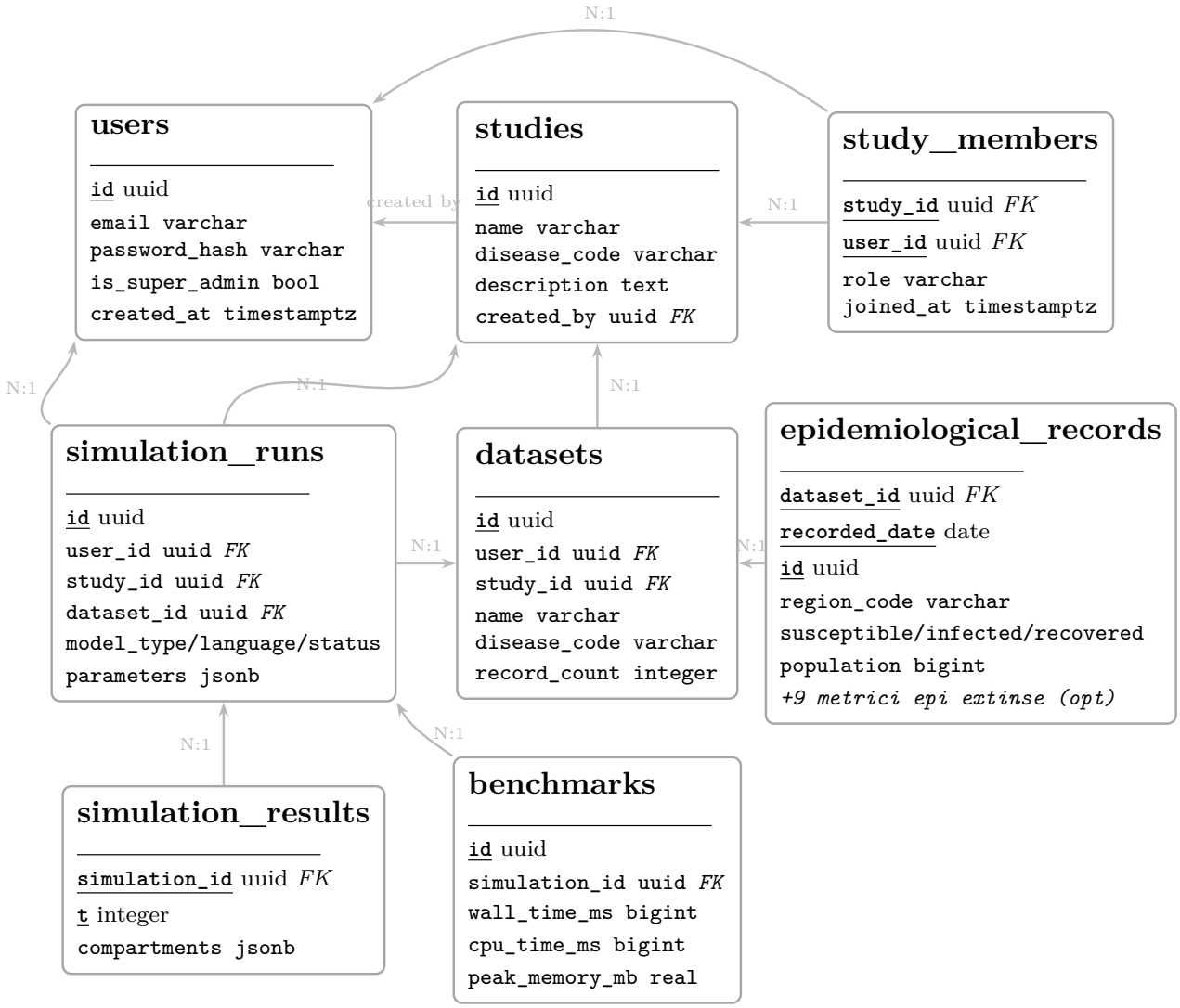


Figura 4.3: Diagrama entitate-relație a bazei de date curente. PK-urile sunt marcate cu aldine și subliniere; FK-urile sunt marcate cu *FK*. Tabelele `datasets` și `simulation_runs` sunt scope-uite pe `study_id`, iar tabela `study_members` definește rolul fiecărui utilizator într-un studiu.

studiul, setul de date, tipul modelului (`sidarthe`, `sir`, `sis`, `seir`, `sirs`, `lstm` sau `script`), parametrii în format JSONB, populația, orizontul temporal, limbajul ales, statusul execuției (`pending`, `running`, `completed`, `failed`), câmpul `error_message` și timestampurile de creare și finalizare. Tabelul `simulation_results` stochează seria temporală de ieșire: pentru fiecare pas de timp $t \in \{0, 1, \ldots, t_{\max}\}$, câmpul `compartments` conține un obiect JSONB cu valorile compartimentelor modelului. Tabelul `benchmarks` reține metricile de performanță colectate automat la finalul fiecărei rulări: timpul wall-clock, timpul CPU și consumul maxim de memorie.

4.3.2 Modelul datelor epidemiologice

Datele epidemiologice sunt ingerate în sistem prin fișiere CSV cu structura minimă prezentată în Table 4.1.

Câmpurile `susceptible` și `recovered` sunt opționale; dacă lipsesc, API-ul le estimează prin $S(0) = N - I(0)$ și $R(0) = 0$. Granularitatea temporală recomandată este zilnică; datele săptămânale sunt acceptate, dar reduc precizia estimării parametrilor. La încărcarea unui set de date, API-ul verifică că toate câmpurile obligatorii sunt prezente și au tipul corect, că relația $I(t) + R(t) \leq N$ este respectată pentru fiecare înregistrare, că seria temporală este ordonată

Tabela 4.1: Structura minimă a fișierului CSV de date epidemiologice.

Câmp	Tip	Descriere
`date`	`DATE`	Data înregistrării (format ISO 8601, obligatoriu)
`region`	`VARCHAR`	Cod regiune ISO 3166-2 (obligatoriu)
`infected`	`BIGINT`	Număr de cazuri active (obligatoriu)
`population`	`BIGINT`	Populația totală (obligatoriu)
`recovered`	`BIGINT`	Cazuri recuperate (opțional)
`susceptible`	`BIGINT`	Populația susceptibilă estimată (opțional)
Metrici epidemiologice extinse (toate opționale, propagate scripturilor user)		
`deaths`	`BIGINT`	Decese cumulate
`new_cases`	`BIGINT`	Cazuri noi în intervalul de raportare
`new_deaths`	`BIGINT`	Decese noi în intervalul de raportare
`hospitalized`	`BIGINT`	Pacienți spitalizați la moment
`icu`	`BIGINT`	Pacienți în terapie intensivă la moment
`vaccinated`	`BIGINT`	Persoane vaccinate (cel puțin o doză), cumulat
`tested`	`BIGINT`	Teste cumulate
`stringency_index`	`REAL`	Oxford Stringency Index, $0 \ldots 100$
`reproduction_rate`	`REAL`	R_t observat, când e disponibil

cronologic și că fiecare cod de regiune este nevid. Validarea formală față de un registru ISO și analiza golurilor calendaristice nu sunt incluse în implementarea curentă a API-ului; ele sunt tratate în etapa de preprocesare pentru seturile de date OWID incluse în repository.

Metricile epidemiologice extinse (decese, spitalizări, ATI, vaccinări, teste, indicele Oxford de stringență, R_t observat) sunt opționale la nivel de schemă și de upload: dacă header-ul CSV include coloana corespunzătoare iar valoarea este nevidă, parser-ul Go o convertește la tipul potrivit (`BIGINT` sau `REAL`) și o trece prin `COPY FROM` în `epidemiological_records`; altfel, câmpul rămâne `NULL`. Runner-ele built-in (SIR/SIDARTHE/LSTM) ignoră aceste coloane — ele consumă doar prima înregistrare pentru condiția inițială —, însă executorul de scripturi (Chapter 9, Section 9.5) le expune integral funcției `main` prin lista `records`, ceea ce permite cercetătorului să scrie modele mai bogate (calibrări care folosesc decesele, ajustări la R_t observat) fără modificări de schemă sau de cod în API.

4.3.3 Strategia de partiționare și replicare în YugabyteDB

YugabyteDB utilizează un model de stocare bazat pe *tablets* (echivalentul shard-urilor), gestionate prin protocolul de consens Raft [26]. Clusterul este configurat cu factor de replicare $RF = 3$, garantând că pierderea oricărui nod unic nu afectează disponibilitatea sau consistența datelor.

Tabelul `epidemiological_records` este proiectat pentru *range sharding* pe cheia compusă `(dataset_id, recorded_date)`. Această strategie optimizează interogările de tip *range scan* pe interval temporal, care reprezintă cel mai frecvent pattern de acces — de exemplu, extragerea seriei temporale complete a unui set de date. În mediul de producție, partiționarea inițială se declară explicit prin clauza `SPLIT AT VALUES`, prezentată mai jos; migrația de bază aplică schema fără presplitting, lăsând YugabyteDB să partitioneze automat pe măsura creșterii datelor.

```
CREATE TABLE epidemiological_records (
    id              UUID DEFAULT gen_random_uuid(),
    dataset_id      UUID NOT NULL REFERENCES datasets(id),
    recorded_date   DATE NOT NULL,
    region_code     VARCHAR(10) NOT NULL,
```

```
    susceptible       BIGINT,
    infected          BIGINT NOT NULL,
    recovered         BIGINT,
    population        BIGINT NOT NULL,
    -- Metrici epi extinse (opt; populate cand sursa le ofera).
    deaths            BIGINT,
    new_cases         BIGINT,
    new_deaths        BIGINT,
    hospitalized      BIGINT,
    icu               BIGINT,
    vaccinated        BIGINT,
    tested            BIGINT,
    stringency_index  REAL,
    reproduction_rate REAL,
    PRIMARY KEY (dataset_id, recorded_date, id)
) SPLIT AT VALUES (
    ('00000000-0000-0000-0000-000000000000', '2020-01-01'),
    ('00000000-0000-0000-0000-000000000000', '2021-01-01'),
    ('00000000-0000-0000-0000-000000000000', '2022-01-01')
);
```

Listing 4.1: DDL de producție al tabelului `epidemiological_records` cu range sharding presplit pe cheie temporală.

Tabelul `simulation_results` utilizează, în schimb, hash sharding pe `simulation_id`, distribuind uniform rândurile între tablets, deoarece accesul la rezultate este întotdeauna filtrat după un `simulation_id` specific și nu necesită range scans inter-simulări.

Fiecare tablet are un lider Raft ales dintre cele trei replici; scrierile trec prin lider (*strong consistency*) și sunt replicate sincron pe celelalte două noduri înainte de confirmare. Parametrul `ysql_num_shards_per_tserver` este setat la 4 (față de valoarea implicită 8) pentru a reduce supraîncărcarea managementului de tablets pe un cluster de dimensiuni mici.

4.4 Arhitectura API-ului backend Go

4.4.1 Structura modulară a proiectului

Proiectul backend este organizat conform convențiilor comunității Go (*Standard Go Project Layout*) [27]:

```
emedicine-api/
+-- cmd/
|   `-- server/
|       `-- main.go       # Entry point: initializare, injectie dependinte
+-- internal/
|   +-- api/
|   |   +-- handlers/     # Auth, users, studies, datasets, sims, admin
|   |   +-- middleware/   # JWT si RBAC pe studii
|   |   `-- router/       # Configurare rute (chi router)
|   +-- db/
|   |   `-- repository/   # Interfete si implementari repository pattern
|   +-- runner/
|   |   `-- dispatcher.go # Publicare job-uri in Redis
|   `-- models/           # Struct-uri de domeniu (User, Study, Dataset, Run)
```

Listing 4.2: Structura de directoare a proiectului backend Go.

Arhitectura internă folosește o separare pragmatică între handler-ele HTTP, middleware-uri și repository-uri. **Handler layer** parsează cererea, validează input-ul, apelează repository-urile și serializează răspunsul JSON. Regulile transversale sunt extrase în middleware-uri: validarea JWT introduce `user_id` în context, iar middleware-ul RBAC introduce `study_id` și rolul curent după verificarea membership-ului. **Repository layer** concentrează interogările SQL și operațiile tranzacționale peste `pgx`. Publicarea job-urilor în Redis este izolată în pachetul `runner`, printr-un dispatcher injectat explicit în handler-ul de simulări.

4.4.2 Mecanismul de autentificare JWT

Autentificarea se bazează pe JSON Web Tokens (RFC 7519) [28], utilizând algoritmul de semnătură `HS256` (HMAC-SHA256). Cheia secretă este generată la instalare și stocată ca Kubernetes Secret, montată în containerul API ca variabilă de mediu — nu este inclusă în imaginea Docker. La autentificare, clientul trimite credențialele la `POST /api/auth/login`; API-ul verifică hash-ul bcrypt al parolei din baza de date și, în caz de succes, returnează un JWT cu payload `{sub: user_id, exp: now+24h, iat: now}`. Clientul include token-ul în fiecare cerere ulterioară prin header-ul `Authorization: Bearer <token>`, iar middleware-ul JWT îl validează — verificând semnătura și expierea — fără a accesa baza de date. Absența oricărei stări pe server este esențială pentru scalabilitatea orizontală: oricare replică a API-ului poate valida orice token, fără sincronizare între instanțe.

4.4.3 RBAC pe studii

După autentificare, rutele de lucru sunt evaluate în contextul unui parametru `{studyId}`. Middleware-ul `RequireStudyMembership` verifică dacă utilizatorul curent este membru al studiului și dacă rolul său este inclus în lista permisă pentru operația respectivă. Pentru citire este suficient orice rol valid; pentru încărcarea sau ștergerea dataseturilor sunt acceptate `admin` și `contributor`; pentru lansarea simulărilor sunt acceptate `admin` și `researcher`; pentru administrarea studiului este necesar rolul `admin`. Utilizatorii cu `is_super_admin=true` trec automat verificarea de membership și sunt tratați ca administratori în orice studiu, ceea ce permite operații de suport și audit global.

4.4.4 Diagrama endpointurilor REST

Table 4.2 prezintă endpointurile REST expuse de API.

Răspunsurile API urmează o convenție simplificată inspirată din *JSON:API*: obiectul de nivel superior conține fie câmpul `data` (la succes), fie `error` cu un cod numeric și un mesaj uman-citibil (la eșec), permițând tratarea uniformă a erorilor în clientul frontend.

4.4.5 Integrarea cu runner-ele de algoritmi

Comunicarea dintre API-ul Go și runner-e este asincronă, mediată de Redis printr-un model *producer–consumer*. Când utilizatorul inițiază o simulare sau o predicție, API-ul publică un obiect JSON într-o listă Redis dedicată runner-ului. Pentru modelele compartimentale se folosește convenția `epimodel:jobs:<language>` (ex. `epimodel:jobs:r`), iar pentru LSTM se folosește coada separată `epimodel:jobs:lstm`:

```
{
  "simulation_id": "a3f9c2d1-...",
  "dataset_id":    "7b4e1a02-...",
  "model_type":    "sidarthe",
  "parameters":    {"alpha": 0.57, "beta": 0.011, "gamma": 0.456,
                    "sigma": 0.017, "tau": 0.010},
  "population_n":  1000000,
  "t_max":         365,
  "initial":       {"s": 998999.0, "i": 1001.0, "d": 0.0,
                    "a": 0.0, "r": 0.0, "t": 0.0, "h": 0.0, "e": 0.0},
  "db_dsn":        "postgres://emedicine:***@yb-tserver:5433/..."
}
```

Listing 4.3: Exemplu mesaj JSON publicat în coada Redis la inițierea unei simulări.

Fiecare runner execută o buclă `BRPOP` blocantă pe coada sa dedicată. La primirea unui job, runner-ul actualizează statusul simulării la `running`, execută modelul cerut prin `model_type`

Tabela 4.2: Endpointurile REST principale ale API-ului backend.

Metodă	Cale	Descriere	Auth
POST	`/api/auth/register`	Înregistrare utilizator nou	Nu
POST	`/api/auth/login`	Autentificare, returnează JWT	Nu
GET	`/api/health`	Health check pentru API, DB și Redis	Nu
GET	`/api/me`	Profilul curent și membershipurile în studii	DA
GET	`/api/studies`	Lista studiilor vizibile utilizatorului curent	DA
POST	`/api/studies`	Creare studiu; creatorul devine `admin`	DA
GET	`/api/studies/{studyId}`	Detalii studiu	DA
PATCH	`/api/studies/{studyId}`	Actualizare metadata studiu	Admin studiu
DELETE	`/api/studies/{studyId}`	Ștergere studiu și resurse asociate	Admin studiu
GET	`/api/studies/{studyId}/members`	Lista membrilor studiului	Membru
POST	`/api/studies/{studyId}/members`	Adăugare membru prin email și rol	Admin studiu
PATCH	`/api/studies/{studyId}/members/{userId}`	Modificare rol membru	Admin studiu
DELETE	`/api/studies/{studyId}/members/{userId}`	Eliminare membru	Admin studiu
GET	`/api/studies/{studyId}/datasets`	Lista dataseturilor din studiu	Membru
POST	`/api/studies/{studyId}/datasets`	Încărcare CSV nou	Admin/Contributor
GET	`/api/studies/{studyId}/datasets/{id}`	Detalii set de date	Membru
DELETE	`/api/studies/{studyId}/datasets/{id}`	Ștergere set de date	Admin/Contributor
GET	`/api/studies/{studyId}/simulations`	Lista simulărilor din studiu	Membru
POST	`/api/studies/{studyId}/simulations`	Inițiere simulare nouă	Admin/Researcher
GET	`/api/studies/{studyId}/simulations/{id}`	Status și metadate simulare	Membru
GET	`/api/studies/{studyId}/simulations/{id}/results`	Seria temporală	Membru
GET	`/api/studies/{studyId}/simulations/{id}/export`	Export CSV rezultate	Membru
GET	`/api/studies/{studyId}/benchmarks`	Benchmark-uri agregate per model și limbaj	Membru
GET	`/api/admin/stats`	Statistici globale pentru consola admin	Super-admin
GET	`/api/admin/users`	Lista utilizatorilor	Super-admin
PATCH	`/api/admin/users/{userId}/super-admin`	Promovare/revocare super-admin	Super-admin
GET	`/api/admin/studies`	Lista tuturor studiilor	Super-admin
GET	`/api/admin/breakdown`	Distribuții agregate între studii (status, model, limbaj, boală, top studii)	Super-admin
GET	`/api/admin/recent-simulations`	Ultimele N simulări cu studiu și user	Super-admin
GET	`/api/admin/recent-datasets`	Ultimele N seturi de date încărcate	Super-admin

(detaliat în Chapter 5), inserează rezultatele în `simulation_results` prin inserări batch de $t_{\max} + 1$ rânduri, înregistrează metricile de performanță în `benchmarks`, iar în final actualizează statusul la `completed` — sau la `failed`, cu mesajul de eroare stocat, dacă a apărut o excepție. Existența cozilor separate per executor (`epimodel:jobs:r`, `epimodel:jobs:julia`, `epimodel:jobs:python` și `epimodel:jobs:lstm`) permite scalarea independentă a fiecărui runner în funcție de cerere, fără a modifica logica principală a API-ului.

4.5 Arhitectura frontend Vue.js

4.5.1 Structura componentelor

Stratul frontend este împărțit în două aplicații Vue.js 3 cu Composition API, Vite și TypeScript. Aplicația `frontend` este interfața de lucru pentru studiile epidemiologice, iar `frontend-admin` este consola separată pentru super-admini. Cele două build-uri sunt independente, dar folosesc același API și același mecanism JWT.

```
1 frontend/
```

```
`-- src/
    +-- pages/
    |   +-- StudiesPage.vue
    |   +-- CreateStudyPage.vue
    |   +-- StudyManagementPage.vue
    |   +-- DashboardPage.vue
    |   +-- DatasetsPage.vue
    |   +-- NewSimulationPage.vue
    |   +-- SimulationResultPage.vue
    |   +-- ComparePage.vue
    |   `-- BenchmarksPage.vue
    +-- stores/
    |   +-- auth.store.ts
    |   +-- studies.store.ts
    |   +-- datasets.store.ts
    |   +-- simulations.store.ts
    |   `-- benchmarks.store.ts
    `-- api/client.ts

frontend-admin/
`-- src/
    +-- pages/
    |   +-- LoginPage.vue
    |   +-- DashboardPage.vue
    |   +-- UsersPage.vue
    |   +-- StudiesPage.vue
    |   +-- StudyDetailPage.vue
    |   `-- ActivityPage.vue
    +-- stores/
    |   +-- auth.store.ts
    |   `-- admin.store.ts
    `-- api/client.ts
```

Listing 4.4: Structura simplificată a celor două aplicații Vue.js.

4.5.2 Fluxul de date și state management

Gestionarea stării globale este realizată cu Pinia [29], biblioteca oficială de state management pentru Vue.js 3. Fiecare store encapsulează starea, acțiunile (apeluri API asincrone) și getters (date derivate) pentru un domeniu specific.

La autentificare, `authStore` primește JWT-ul de la API și îl stochează în `localStorage` pentru persistență între sesiuni. Aplicația principală folosește cheia `emedicine.token`, iar consola admin folosește `emedicine-admin.token`; astfel, sesiunile celor două interfețe rămân independente. Un interceptor Axios adaugă automat header-ul `Authorization: Bearer` la fiecare cerere ulterioară, iar navigation guards Vue Router redirecționează utilizatorii neautentificați spre pagina de login.

În aplicația principală, `studiesStore` păstrează lista de membershipuri, studiul curent și rolul utilizatorului. Rutele care necesită date epidemiologice au meta-câmpul `requiresStudy`; dacă nu există un studiu selectat, utilizatorul este trimis către pagina de studii. Drepturile derivate `canWriteDatasets`, `canWriteSimulations` și `canManageStudy` controlează vizibilitatea acțiunilor în interfață și oglindesc regulile aplicate de backend.

Fluxul de simulare începe din pagina `NewSimulationPage`, care apelează `simulationsStore.create()` cu parametrii formularului și cu `studyId`-ul curent inclus în URL-ul API. Store-ul primește înapoi `simulation_id` și componenta `SimulationStatus` pornește un polling la interval de 2 secunde pe ruta scope-uită pe studiu, oprindu-se când statusul devine `completed` sau `failed`. La finalizare, `SimulationResultPage` încarcă seria temporală și o pasează componentei `CompartmentChart`, care randează graficul interactiv pe baza cheilor prezente în obiectul `compartments`.

Pagina `ComparePage` permite selectarea a 2–3 simulări finalizate. Pentru fiecare rulare selectată, store-ul încarcă metadatele și seria temporală prin cereri paralele, iar `CompartmentChart` le suprapune în același grafic, diferențiindu-le prin stil de linie (continuu, întrerupt, punctat).

Consola `frontend-admin` folosește un store separat, `adminStore`, care apelează exclusiv rutele `/api/admin/*`. La login, aplicația refuză utilizatorii care nu au `is_super_admin=true`, chiar dacă autentificarea API ar fi validă. Pe lângă statisticile globale și lista utilizatorilor cu promovarea/revocarea drepturilor de super-admin, store-ul mai expune trei fluxuri de citire între studii: distribuțiile agregate pentru dashboard (`breakdown`), simulările recente (`recentSimulations`) și seturile de date recente (`recentDatasets`). Pagina de detaliu pentru un studiu (`StudyDetailPage`) reutilizează direct endpointurile `/api/studies/{studyId}/*`, deoarece super-adminul trece automat prin middleware-ul RBAC, fără a fi necesare endpointuri admin dedicate.

4.6 Infrastructura Kubernetes

Sistemul rulează pe un cluster Kubernetes versiunea v1.31, găzduit pe o platformă cloud privată; detaliile specifice provider-ului IaaS rămân în afara domeniului acestei lucrări. Ciclul de viață al nodurilor (creare, redimensionare, rolling updates) este gestionat declarativ prin Cluster API, care permite descrierea topologiei prin obiecte Kubernetes într-un cluster de management separat. Această abordare oferă reproducibilitate și uniformitate între medii: aceleași manifeste aplicative pot fi deployate pe orice cluster K8s conform standardelor CNCF, indiferent de provider-ul de infrastructură.

4.6.1 Topologia cluster-ului

Clusterul este compus din șase noduri (Fig. 4.4): trei noduri în rolul de control plane (2 vCPU, 4 GiB RAM fiecare) care găzduiesc componentele standard — `kube-apiserver`, `etcd` stacked, `kube-scheduler`, `kube-controller-manager` — și trei noduri *worker* (4 vCPU, 8 GiB RAM fiecare) pe care rulează pod-urile aplicației, baza de date și componentele add-on. Toate nodurile rulează Ubuntu 22.04 LTS cu `containerd` 1.7 ca runtime de container.

Interfața de rețea este implementată prin Cilium CNI, care oferă rețeaua de pod, DNS intern (prin `CoreDNS`) și posibilitatea de a aplica politici de rețea (NetworkPolicy, CiliumNetworkPolicy) la nivel de namespace. Stocarea persistentă este asigurată de un driver CSI de tip block storage cu `reclaimPolicy: Delete` și expansion de volum activat, ceea ce permite scalarea în sus a volumelor YugabyteDB fără migrare manuală. `metrics-server` este instalat pentru a permite atât utilitarul `kubectl top`, cât și autoscalarea bazată pe metrici CPU/memorie (Horizontal Pod Autoscaler).

Toate resursele aplicației sunt grupate în namespace-ul `epimodel`, ceea ce permite aplicarea politicilor de rețea (NetworkPolicy) și controlul accesului bazat pe roluri (RBAC) la nivel de namespace. Add-on-urile sunt separate în namespace-urile lor proprii (`ingress-nginx`, `kube-system`) conform convenției standard.

4.6.2 Deployment YugabyteDB pe Kubernetes

YugabyteDB este deploiat prin Helm chart-ul oficial. Cu chart-ul vine un set complet de manifeste — două `StatefulSet`-uri, două Service-uri *headless* pentru DNS intern, ConfigMap-uri pentru `gflags` și volumele persistente — pe care îl putem suprascrie printr-un singur fișier de `values` custom adaptat resurselor disponibile:

```
helm repo add yugabytedb https://charts.yugabyte.com
helm install yugabyte yugabytedb/yugabyte \
  --namespace epimodel \
  -f code/k8s/10-yugabyte-values.yaml \
  --wait --timeout 10m
```

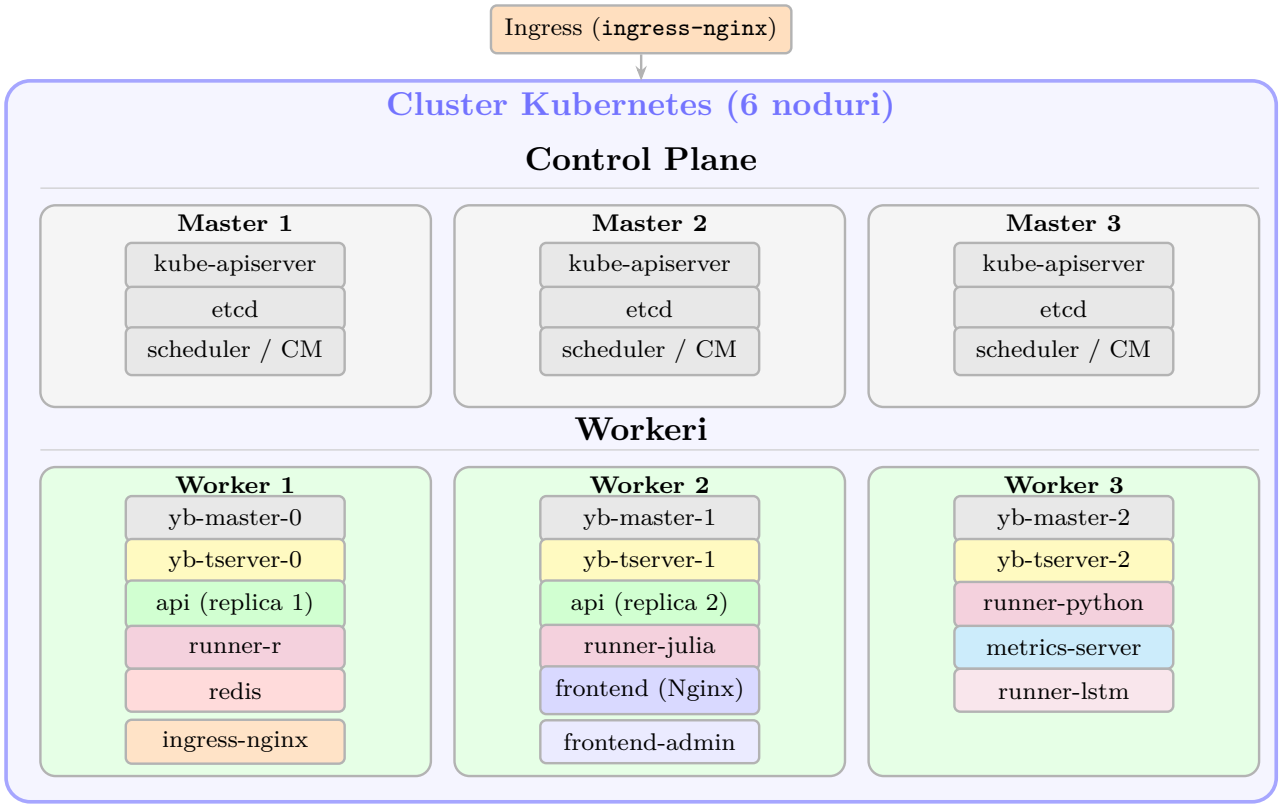


Figura 4.4: Topologia cluster-ului Kubernetes cu 3 noduri control plane HA și 3 noduri worker. Pod-urile YugabyteDB sunt distribuite uniform prin `podAntiAffinity`, garantând că pierderea unui worker afectează cel mult o replică `yb-master` și o replică `yb-tserver` — în limita toleranței quorum-ului Raft.

Listing 4.5: Instalarea YugabyteDB cu Helm și fișierul de `values` custom.

Fișierul `10-yugabyte-values.yaml` declară factor de replicare 3 pentru ambele componente, sizing adaptat workerilor (1 GiB request / 1.5 GiB limit pentru `yb-master`; 2 GiB request / 3 GiB limit pentru `yb-tserver`), un singur volum persistent per replică (5 GiB master, 10 GiB tserver), și gflag-uri specifice pentru clustere mici (`default_memory_limit_to_ram_ratio: 0.6`, `ysql_num_shards_per_tserver: 2`). Chart-ul creează:

- două `StatefulSet`-uri (`yb-master` și `yb-tserver`) cu câte 3 replici fiecare;
- șase `PersistentVolumeClaim`-uri (3×5 GiB pentru masteri + 3×10 GiB pentru tserveri = 45 GiB cumulat);
- două `Service`-uri headless (`yb-masters` și `yb-tservers`) pentru descoperirea peer-urilor prin DNS intern stabil (de exemplu `yb-master-0.yb-masters.epimodel.svc.cluster.local`), necesar protocolului Raft;
- un `Service ClusterIP` numit `yb-tserver-service` pe portul 5433 (YSQL), folosit de API-ul Go și de runner-e pentru conexiunea la baza de date.

4.6.3 Stratul aplicativ: imagini container și registru

Cele șapte componente proprii ale sistemului — API Go, frontend-ul principal, frontend-ul admin, cele trei runner-e compartimentale R / Python / Julia și runner-ul LSTM — sunt împachetate ca imagini Docker construite pentru arhitectura `linux/amd64` și publicate într-un

registru de imagini compatibil OCI (GitHub Container Registry în cazul de față). Pod-urile cluster-ului fac `pull` autentificat prin `imagePullSecrets` de tip `docker-registry`, configurat o singură dată per namespace.

Imaginea API-ului este construită cu un *multi-stage build*: primul stage compilează binarul Go static (`CGO_ENABLED=0`), iar al doilea copiază doar binarul rezultat într-o imagine `distroless/static` (`nonroot`), obținând o imagine finală de aproximativ 18 MiB, fără shell și fără pachete de sistem inutile. Cele două frontend-uri Vue sunt compilate în fișiere statice (`npm run build`) și servite prin containere Nginx configurate ca SPA: toate rutele necunoscute sunt redirecționate la `index.html`, permițând Vue Router să gestioneze navigarea pe partea de client. Runner-ele utilizează imagini de bază specifice fiecărui limbaj (Python 3.12 slim, R 4.4 base, Julia 1.11), cu dependențele instalate la build time pentru a evita orice descărcare la pornire.

API-ul este deployat ca `Deployment` cu 2 replici și un `Service ClusterIP` pe portul 8080; probele `readinessProbe` și `livenessProbe` interoghează `GET /api/health`, care verifică intern conexiunile către YugabyteDB și Redis. Runner-ele funcționează ca `Deployment`-uri cu o replică fiecare, fără Service expus — comunicarea se face exclusiv prin coada Redis, ceea ce le face independente de adresele lor de rețea și ușor de scalat orizontal prin `kubectl scale` sau HPA pe metrici personalizate (de exemplu lungimea cozii Redis).

4.6.4 Coadă de mesaje și migrațiile bazei de date

Redis este deployat ca `Deployment` cu o singură replică și volum efemer (`emptyDir`); datele cozilor nu sunt persistente între restarturi, ceea ce este acceptabil pentru job-uri de simulare care durează secunde sau minute, nu zile. În scenarii cu cerințe stricte de durabilitate (de exemplu retenția istorică a job-urilor pentru audit), Redis poate fi convertit într-un `StatefulSet` cu `PersistentVolumeClaim` fără modificări la nivelul aplicației.

Migrațiile SQL sunt aplicate printr-un `Job` Kubernetes care folosește clientul `psql` în locul utilitarului `golang-migrate`. Această decizie este motivată de faptul că YugabyteDB nu implementează încă *advisory locks* PostgreSQL, pe care multe utilitare de migrare le folosesc pentru a se asigura că o singură instanță aplică schema la un moment dat. Job-ul nostru iterează fișierele `*.up.sql` în ordine alfabetică și le aplică cu `psql -v ON_ERROR_STOP=1`, ceea ce oferă același efect de *all-or-nothing* fără a depinde de blocaje distribuite. Prima migrare creează extensia `pgcrypto`, necesară pentru funcția `gen_random_uuid()` pe baza PostgreSQL 11 a YugabyteDB 2024.1.

4.6.5 Expunere externă și ingress

Ingress controller-ul `ingress-nginx` este instalat în namespace-ul `ingress-nginx` prin Helm chart-ul oficial, înregistrând un singur `IngressClass` (`nginx`) marcat ca *default* pe cluster. Un singur obiect `Ingress` (`epimodel-ingress`) rutează cererile externe pentru două host-uri. Pe `sir-workbench.ro`, prefixul `/api` ajunge la `Service api`, iar restul traficului ajunge la `Service frontend`. Pe `admin.sir-workbench.ro`, același prefix `/api` ajunge la API, iar restul traficului ajunge la `Service frontend-admin`. Această separare păstrează consola administrativă într-un build distinct, fără a duplica backend-ul.

Modul de expunere a ingress-controller-ului depinde de capabilitățile provider-ului cloud. Pe un cluster cu `LoadBalancer Service` funcțional, controller-ul primește un IP public alocat automat și poate fi accesat direct prin DNS. Pe medii fără provider de `LoadBalancer` disponibil — sau pentru sesiuni de demonstrație controlată — ingress-controller-ul poate fi expus indirect prin `kubectl port-forward` la portul 80 intern al Service-ului. În mediul de producție, terminarea TLS se configurează la nivel de Ingress prin `cert-manager` cu certificate Let's Encrypt; pentru

disertație, traficul rămâne necriptat pe legătura locală `port-forward`, urmând ca activarea TLS să fie un pas operațional ulterior.

4.7 Sistem de plug-in-uri pentru algoritmi

Runner-ele R, Julia și Python descrise până acum acoperă familia modelelor compartimentale (SIR/SIS/SEIR/SIRS/SIDARTHE) și componenta LSTM, dar lista algoritmilor relevanți pentru analiza unei serii epidemiologice este deschisă: un cercetător poate dori să compare cu o regresie statistică, cu un model AR/ARIMA, cu o variantă bayesiană sau cu un model agent-based implementat în orice limbaj. Pentru a evita modificarea API-ului, a schemei de bază de date și a deployment-urilor Kubernetes la fiecare astfel de adăugare, sistemul oferă două căi distincte și complementare de extensibilitate, separate explicit ca responsabilități și permisiuni:

- **Runner-e bazate pe imagine container** (această secțiune): un super-admin înregistrează în studiu o referință către o imagine construită extern, iar API-ul provisionează automat un Deployment Kubernetes dedicat. Algoritmul este *persistent* (pod always-on, coadă proprie) și poate fi rulat de orice membru al studiului. Potrivit pentru algoritmi "oficiali", reutilizabili, în orice limbaj.
- **Editor de cod în browser pentru cercetători** (descris în capitolul 9, secțiunea 9.5): un cercetător scrie un script Python într-un editor Monaco, îl salvează ca draft personal și îl rulează ad-hoc peste un dataset. Algoritmul este *efemer* (execuție per cerere, fără pod dedicat) și este executat de un executor partajat `runner-script`. Potrivit pentru explorare, prototipare și analize one-off, fără ceremonia înregistrării unui runner.

Această diviziune are o motivație practică: înregistrarea unui runner-container implică provisionarea unei resurse Kubernetes permanente și consumă o slot de pod în cluster, ceea ce justifică restricția la super-admin. Scrierea unui script de exploare nu trebuie să aibă același cost organizațional, deci este liberă oricărui cercetător cu drept de scriere în studiu. Cele două căi se folosesc de aceleași tabele `simulation_runs`/`simulation_results` și de același grafic în interfață — diferă doar lifecycle-ul resursei.

4.7.1 Forma standard: *runner protocol*

Forma pe care trebuie să o respecte o imagine container pentru a se comporta ca runner valid este deliberat compactă, ca să poată fi implementată în orice limbaj:

1. La pornire, imaginea citește variabilele de mediu `RUNNER_QUEUE` (coada Redis de ascultat), `DATABASE_URL` (conexiunea la YugabyteDB) și `REDIS_URL`.
2. Pentru fiecare mesaj scos cu `BRPOP` din `RUNNER_QUEUE`, se așteaptă un JSON cu cheile `simulation_id`, `dataset_id`, `parameters`, `population_n` și `t_max`.
3. Imaginea citește înregistrările dataset-ului din tabela `epidemiological_records` (`WHERE dataset_id = $1`), execută algoritmul și scrie rezultatele în `simulation_results` (câte un rând per pas temporal, cu `compartments` ca JSONB) și metricile în `benchmarks`.
4. În final, marchează `simulation_runs.status = 'completed'` (sau `'failed'` cu `error_message`).

Tipul rezultatului așteptat per pas temporal este același ca pentru runner-ele built-in (`{"t": int, "compartments": {...}}`); aceasta înseamnă că graficul de traiectorii compartimentale poate afișa rezultatele oricărui plug-in fără modificări de cod. Imaginea `runner-custom` furnizată cu sistemul (folosită și ca executor pentru editor) respectă acest protocol și poate servi atât ca implementare de referință, cât și ca bază pentru imagini derivate (`FROM ghcr.io/.../emedicine-runner-custom`). Validarea referinței de imagine la upload este o verificare de sanitate (`^[a-zA-Z0-9][a-zA-Z0-9._/:@-]{1,254}$`); Kubernetes respinge la momentul `pull` referințele care nu pot fi rezolvate de nodurile cluster-ului.

4.7.2 Provisioning Kubernetes

La înregistrarea unui runner, API-ul folosește clientul oficial `client-go` pentru a crea un Deployment cu o replică, imagine egală cu referința dată de super-admin, variabilele de mediu populate din Secret/ConfigMap (`DATABASE_URL`, `REDIS_URL`) și `RUNNER_QUEUE` setat la o coadă unică derivată din numele runner-ului (`epimodel:jobs:custom-<nume>`). Un `ServiceAccount` dedicat (`api`) este legat de un `Role` care permite verbele `get`, `list`, `create` și `delete` doar pe resursa `deployments` din namespace-ul aplicației — nimic mai mult, ceea ce respectă principiul privilegiului minim. Tabelul 4.3 rezumă tranzițiile de stare ale unui runner pe acest traseu.

Tabela 4.3: Ciclul de viață al unui runner plug-în, de la upload până la oprire. Stările sunt persistate în coloana `status` a tabelei `runners`; tranzițiile sunt declanșate de API (creare/ștergere) și de wrapper-ul din pod (marcarea `ready`).

Stare	Trigger / descriere
`pending`	INSERT inițial în DB; răspunsul HTTP returnează imediat.
`building`	API a creat Deployment-ul K8s prin `client-go`; pod-ul este *Pending*.
`ready`	Wrapper-ul din pod a citit manifestul din DB, a compilat scriptul și ascultă pe coadă.
`failed`	Eroare la crearea Deployment-ului sau la compilarea scriptului; `status_message` conține detalii.
`stopped`	Oprire explicită; Deployment-ul este șters, dar rândul rămâne pentru audit (opțional).

Imaginea fiind construită extern (în CI-ul propriu al cercetătorului sau local cu `docker build`), API-ul nu execută nicio operație de build in-cluster. O extensie naturală ar fi un pipeline opțional bazat pe `Kaniko` care ar putea construi imagini din scripturi cu `requirements.txt` declarat, dar pentru scopul tezei această responsabilitate este lăsată în sarcina cercetătorului, exact așa cum este lăsată construirea oricărei imagini reutilizabile.

4.7.3 Exemplu concret de runner plug-in

Pentru a face palpabilă diviziunea descrisă, Listing 4.6 prezintă un runner plug-in complet care implementează un model SIR cu rată explicită de mortalitate (β, γ, μ – parametri liberi, compartiment suplimentar D pentru decese). Codul folosește `redis-py` și `psycopg` ca runner-ele built-in, dar trăiește într-un repository extern, este publicat ca imagine container pe un registru accesibil de cluster și este înregistrat în studiu printr-un singur apel API. Restul mecanicii (provisionare Deployment, ascultare `BRPOP`, afișare a traiectoriei în graficul UI) este moștenită *gratis* de la contractul standard.

```
import json, os, time
import redis, psycopg
from psycopg.types.json import Json
from scipy.integrate import solve_ivp

QUEUE = os.environ["RUNNER_QUEUE"]   # epimodel:jobs:custom-sir-d
```

```python
DB = os.environ["DATABASE_URL"]
RD = os.environ["REDIS_URL"]

def sir_d(t, y, beta, gamma, mu, N):
    S, I, R, D = y
    dS = -beta * S * I / N
    dI =  beta * S * I / N - gamma * I - mu * I
    dR =  gamma * I
    dD =  mu * I
    return [dS, dI, dR, dD]

def main():
    r = redis.from_url(RD)
    with psycopg.connect(DB) as cn:
        while True:
            _, raw = r.brpop(QUEUE)
            job = json.loads(raw)
            sid, did = job["simulation_id"], job["dataset_id"]
            p = job["parameters"]
            N = job["population_n"]
            tmax = job["t_max"]
            # citeste prima inregistrare ca conditie initiala
            with cn.cursor() as cur:
                cur.execute(
                    "SELECT susceptible, infected, recovered, deaths "
                    "FROM epidemiological_records "
                    "WHERE dataset_id=%s ORDER BY recorded_date LIMIT 1",
                    (did,))
                S0, I0, R0, D0 = cur.fetchone()
            sol = solve_ivp(sir_d, (0, tmax), [S0, I0, R0 or 0, D0 or 0],
                            args=(p["beta"], p["gamma"], p["mu"], N),
                            t_eval=range(tmax+1), method="RK45")
            rows = [(sid, int(t), Json({"S": float(sol.y[0][t]),
                                        "I": float(sol.y[1][t]),
                                        "R": float(sol.y[2][t]),
                                        "D": float(sol.y[3][t])}))
                    for t in range(tmax+1)]
            with cn.cursor() as cur:
                cur.executemany(
                    "INSERT INTO simulation_results "
                    "(simulation_id, t, compartments) VALUES (%s,%s,%s)", rows)
                cur.execute("UPDATE simulation_runs SET status='completed' "
                            "WHERE id=%s", (sid,))
            cn.commit()

if __name__ == "__main__":
    main()
```

Listing 4.6: Runner plug-in minimal pentru SIR-D (compartiment de decese), care respectă *runner protocol*-ul (Section 4.7.1) și se integrează fără modificări în API.

După publicarea imaginii (de exemplu pe `ghcr.io`), super-adminul o înregistrează în studiu printr-un `POST /api/studies/{sid}/runners` cu corpul `{"name": "sir-d", "image": "ghcr.io/.../sir-d:1.0"}`; API-ul creează Deployment-ul cu `RUNNER_QUEUE=epimodel:jobs:custom-sir-d` și runner-ul devine selectabil în interfața de creare a simulărilor pentru orice membru al studiului. Niciun cod din API, frontend sau runner-ele built-in nu este modificat. Acest exemplu acoperă cele trei elemente cheie ale contractului: consumul cozii Redis, citirea dataset-ului prin `epidemiological_records` (folosind, opțional, coloanele extinse precum `deaths`) și scrierea rezultatelor în formatul JSONB `compartments`, identic cu runner-ele built-in.

4.7.4 Securitate și limitări

Runner-ele plug-in rulează cod arbitrar furnizat de utilizator, deci modelul de securitate este conservator. **Acces restrâns la creare:** numai un super-admin poate înregistra sau șterge un runner — API-ul aplică această restricție prin lanțul de middleware `RequireStudyMembership` + `RequireSuperAdmin`, iar pentru un utilizator obișnuit cu rol în studiu cererea returnează `403`

`super admin required`. **Scope-uire la studiu:** fiecare runner are coloana `study_id` NOT NULL cu cheie străină la `studies`, iar atât listarea cât și ștergerea sunt scope-uite (`Get(sid, id)`), exact ca pentru simulări și seturi de date. Un runner înregistrat într-un studiu nu apare în alt studiu și nu poate fi referit cross-studiu. **Nume DNS-safe:** numele runner-ului este validat prin expresia regulată `^[a-z][a-z0-9-]{1,62}[a-z0-9]$`, deoarece ajunge atât în numele cozii Redis cât și în numele Deployment-ului K8s. **Limite de resurse:** fiecare Deployment generat are atribuite explicit `requests` (`100m` CPU, `128Mi` memorie) și `limits` (`500m` CPU, `512Mi` memorie), prevenind ca o imagine prost configurată să afecteze celelalte runner-e sau API-ul.

Întrucât imaginea provine dintr-un registry arbitrar, super-adminul își asumă conținutul ei, exact cum își asumă orice altă imagine din infrastructură. Nu există încă un mecanism de *allowlist* de registre permise; o îmbunătățire naturală ar fi restricționarea la registre cunoscute (de exemplu doar `ghcr.io/organizatia/...`) și verificarea opțională a semnăturilor prin `cosign`. De asemenea, nu există un mecanism formal de sandboxing suplimentar (de exemplu `gVisor` sau `Kata Containers`) pentru a izola și mai puternic runner-ele față de nodul Kubernetes; containerul are acces la rețeaua cluster-ului (inclusiv DB-ul, prin DSN-ul moștenit), ceea ce este intenționat pentru a putea citi dataset-ul, dar înseamnă că nu este o sandboxă universală. În scenariul vizat de această lucrare — platformă internă pentru cercetători cunoscuți, super-admin curatând înregistrările — modelul este adecvat; pentru deployment public ar fi necesare politici de rețea (`NetworkPolicy`) restrictive și posibil un sandbox runtime.

Modelul de securitate al *editor-ului din browser* este complementar și descris la momentul potrivit (secțiunea 9.5): scripturile rulează într-un executor partajat `runner-script` cu aceleași limite de resurse, dar permisiunea de a rula scripturi este deschisă oricărui cercetător cu drept de scriere de simulări în studiu, fără gating super-admin — compromisul fiind natura ad-hoc a execuției și absența unei resurse Kubernetes permanente create per script.

Capitolul 5
Implementarea modelelor compartimentale

5.1 Datele de intrare și inițializarea compartimentelor

Datele epidemiologice utilizate de sistem provin din setul public *Our World in Data* (OWID) COVID-19 [30]. Fișierele brute nu conțin direct toate compartimentele necesare modelului SIDARTHE, ci serii temporale agregate, cum ar fi cazurile noi, cazurile cumulate, decesele cumulate și populația. Din acest motiv, etapa de preprocesare nu pretinde că recuperează starea epidemiologică completă a populației. Ea construiește o stare inițială compatibilă cu sistemul software, folosind datele observabile drept punct de pornire pentru explorarea numerică a scenariilor.

Scriptul `scripts/preprocess_owid.py` reindexează fiecare țară pe un calendar zilnic continuu, completează cumulativii prin forward-fill și înlocuiește valorile negative din `new_cases` cu zero, deoarece acestea apar de obicei ca ajustări retrospective ale raportărilor. Numărul de infectați activi este aproximat printr-o fereastră mobilă de 14 zile:

$$I(t) = \sum_{\tau=t-\Delta}^{t} \text{new_cases}(\tau), \qquad \Delta = 14. \tag{5.1}$$

Compartimentul *removed* este aproximat prin

$$R_{\text{removed}}(t) = \text{total_cases}(t) - I(t), \tag{5.2}$$

iar susceptibilii se obțin din bilanțul populației,

$$S(t) = N - I(t) - R_{\text{removed}}(t). \tag{5.3}$$

Pentru modelele SIR, SIS, SEIR și SIRS, aceste trei valori sunt mapate direct sau printr-o transformare simplă, în funcție de existența compartimentului *removed*. Pentru SIDARTHE, aceeași informație este distribuită într-o stare inițială clinică mai bogată: cazurile active observabile sunt plasate în compartimentul I, persoanele ieșite din infecție sunt plasate în H, iar compartimentele clinice neobservabile pornesc de la zero. Această alegere este deliberat conservatoare. Ea nu afirmă că distribuția clinică reală este cunoscută, ci oferă o condiție inițială reproductibilă, potrivită pentru compararea implementărilor și pentru analiza sensibilității la parametri.

Pentru ca benchmark-urile să fie evaluate pe regimuri dinamice variate, sistemul suportă trei familii de seturi de date care produc toate aceeași schemă CSV de bază (`date, region, infected, population, recovered, susceptible`). La aceasta se adaugă, opțional, nouă coloane epidemiologice extinse propagate de `preprocess_owid.py` direct din sursa OWID atunci când există în fișierul brut: `deaths`, `new_cases`, `new_deaths`, `hospitalized`, `icu`, `vaccinated`, `tested`, `stringency_index` (Oxford COVID-19 Government Response Tracker) și `reproduction_rate` (R_t observat). Aceste coloane sunt ignorate de runner-ele built-in (care nu au nevoie decât de prima înregistrare pentru condiția inițială), dar sunt expuse integral scripturilor scrise în editorul din UI prin lista `records`, ceea ce permite scenarii mai bogate de

Tabela 5.1: Maparea datelor observabile în condiții inițiale pentru modelele suportate.

Model	Compartimente inițiale	Interpretare
SIS	$S_0 + R_{\text{removed},0}$, I_0	recuperații redevin susceptibili
SIR și SIRS	S_0, I_0, $R_{\text{removed},0}$	stare agregată cu imunitate permanentă sau temporară
SEIR	S_0, $E_0 = 0$, I_0, $R_{\text{removed},0}$	latența pornește neobservată
SIDARTHE	S_0, I_0, $D_0 = A_0 = R_0 = T_0 = 0$, H_0, $E_0 = 0$	model clinic cu decese explicite

analiză (calibrări care fitează simultan cazurile și decesele, factoring în R_t observat, ajustări la intervențiile non-farmaceutice prin Stringency Index) fără modificări de schemă sau de cod în API.

Pentru a concretiza această capabilitate, Listing 5.1 prezintă un script user scurt care raportează simultan numărul de cazuri active, decesele cumulate și R_t observat la fiecare pas de timp — asemenea unei „simulări" identitate care nu mai necesită nici parametri, nici condiție inițială explicită, doar accesul la coloanele extinse. Listing-ul ilustrează că aceeași infrastructură care execută SIDARTHE prin runner-ele built-in poate găzdui și analize descriptive sau modele ad-hoc direct pe datele extinse, fără tabele suplimentare și fără cod nou în API.

```python
def main(records, parameters, context):
    """Returneaza I, decese cumulate si Rt observat
    ca pseudo-compartimente pentru fiecare zi cu date.
    Nu cere parametri; demonstreaza accesul direct
    la coloanele epi extinse din records.
    """
    out = []
    for t, r in enumerate(records):
        if r.get("deaths") is None:
            continue  # zile fara raportare de decese
        out.append({
            "t": t,
            "compartments": {
                "I":  float(r["infected"]),
                "D":  float(r["deaths"]),
                "Rt": float(r.get("reproduction_rate")
                            or 0.0),
            },
        })
    return out
```

Listing 5.1: Script user (executor partajat `runner-script`) care folosește coloanele epi extinse: raportează I, decese cumulate și R_t observat ca pseudo-compartimente.

OWID COVID-19 extins. Lista de țări procesate prin `preprocess_owid.py` a fost extinsă de la nucleul european la 64 de țări de pe toate continentele (Europa, America, Asia, Africa, Oceania), pentru a acoperi scări de populație de la ∼5 M (Noua Zeelandă) la ∼1.4 B (China, India) și tipare epidemice diferite (suprimare timpurie, valuri masive necontrolate, ieșire abruptă din zero-COVID).

OWID mpox. O a doua boală reală este integrată prin `preprocess_mpox.py`, care consumă setul oficial OWID mpox [30]. Spre deosebire de COVID, fișierul brut nu raportează populația direct; aceasta este dedusă ca mediană a raportului total_cases/total_cases_per_million peste rândurile cu raportare nenulă. Fereastra infecțioasă folosită este de 28 de zile (perioada infecțioasă a mpox este semnificativ mai lungă decât la COVID), iar țările sunt selectate automat după un prag minim de cazuri totale, rezultând 27 de țări care acoperă atât focarul european/american din 2022, cât și focarul din Africa Centrală din 2024.

Generator sintetic de stres. Pentru a împinge limitele sistemului — în special antrenarea LSTM pe serii lungi și volumul de inserări în baza de date — scriptul `generate_synthetic.py`

simulează înainte un model SIRS cu forțare sezonieră:

$$\beta(t) = \beta_0 \left(1 + a \sin(2\pi t/T)\right), \tag{5.4}$$

producând valuri recurente pe orizonturi configurabile (implicit 20 000 de zile, ∼55 de ani simulați). Generatorul oferă trei profile predefinite (`multiwave`, `sharp`, `slow`) cu valori distincte pentru $\mathcal{R}_0$ de bază, durata infecțioasă și amplitudinea sezonieră, iar conservarea $S + I + R = N$ este impusă exact după rotunjire. Logica comună de derivare, validare și scriere CSV este extrasă în modulul `epi_common.py`, partajat între cele trei surse.

5.2 Contractul comun de simulare

Extinderea aplicației la SIDARTHE și la modelele SIR/SIS/SEIR/SIRS a impus generalizarea contractului intern dintre API și runner-e. Versiunea inițială a sistemului transmitea doar parametrii β și γ și presupunea implicit că rezultatul conține curbele S, I și R. Versiunea curentă folosește câmpul `model_type`, un obiect `parameters` și un obiect `initial`. În acest fel, același flux poate executa toate modelele suportate fără a modifica interfața de cozi sau mecanismul de benchmark.

```
{
  "simulation_id": "a3f9c2d1-...",
  "dataset_id": "7b4e1a02-...",
  "model_type": "sidarthe",
  "parameters": {
    "alpha": 0.57, "beta": 0.011, "gamma": 0.456, "delta": 0.011,
    "epsilon": 0.171, "zeta": 0.125, "lambda": 0.034,
    "eta": 0.125, "rho": 0.034, "theta": 0.370,
    "mu": 0.017, "kappa": 0.017, "nu": 0.027,
    "xi": 0.017, "sigma": 0.017, "tau": 0.010
  },
  "initial": {
    "s": 19000000, "i": 1200, "d": 0, "a": 0,
    "r": 0, "t": 0, "h": 18000, "e": 0
  },
  "population_n": 19019200,
  "t_max": 160,
  "db_dsn": "postgres://..."
}
```

Listing 5.2: Structura job-ului publicat în Redis pentru o simulare SIDARTHE.

Coada Redis a fost redenumită din `sir:jobs:<language>` în `epimodel:jobs:<language>`. Schimbarea este mică în cod, dar importantă conceptual: runner-ul nu mai este o implementare exclusivă SIR, ci un executor pentru modele compartimentale compatibile cu schema comună. Pentru LSTM există o coadă separată, `epimodel:jobs:lstm`, consumată de runner-ul `runner-lstm`. Limbajul nu trebuie transmis în payload pentru modelele compartimentale, deoarece este deja codificat în numele cozii Redis (`epimodel:jobs:r`, `epimodel:jobs:julia` sau `epimodel:jobs:python`); pentru LSTM, rutarea este determinată de `model_type`. Rezultatele sunt stocate în `simulation_results` ca obiect JSONB `compartments`, nu ca trei coloane fixe. Pentru o rulare SIDARTHE, un punct temporal conține cheile `s`, `i`, `d`, `a`, `r`, `t`, `h` și `e`; pentru SIR, același câmp conține doar `s`, `i` și `r`; pentru SEIR, ordinea este `s`, `e`, `i`, `r`; pentru LSTM, câmpul conține serii predictive precum `observed`, `fitted` și `predicted`.

5.3 Implementarea modelului SIDARTHE

Sistemul include varianta clinică `sidarthe`. Modelul folosește compartimentele S, I, D, A, R, T, H și E, iar ieșirea din T este împărțită între vindecare (σT) și deces (τT). Această alegere

elimină ambiguitatea dintre vindecări și decese și păstrează explicit bilanțul populației în toate rezultatele.

În Python, nucleul modelului este o funcție RHS compatibilă cu `scipy.integrate.solve_ivp`. Ea primește vectorul de stare, dicționarul de parametri și populația totală:

```python
def sidarthe_rhs(t, y, params, n):
    s, i, d, a, r, threatened, h, extinct = y
    force = (
        params["alpha"] * i +
        params["beta"] * d +
        params["gamma"] * a +
        params["delta"] * r
    ) / n
    new_infections = s * force

    return [
        -new_infections,
        new_infections - (params["epsilon"] + params["zeta"] + params["lambda"]) * i,
        params["epsilon"] * i - (params["eta"] + params["rho"]) * d,
        params["zeta"] * i - (params["theta"] + params["mu"] + params["kappa"]) * a,
        params["eta"] * d + params["theta"] * a - (params["nu"] + params["xi"]) * r,
        params["mu"] * a + params["nu"] * r - (params["sigma"] + params["tau"]) * threatened,
        params["lambda"] * i + params["rho"] * d + params["kappa"] * a
            + params["xi"] * r + params["sigma"] * threatened,
        params["tau"] * threatened,
    ]
```

Listing 5.3: Funcția RHS SIDARTHE în runner-ul Python.

În ultimele trei ecuații se vede separarea cazurilor severe în vindecare și deces:

```python
dT = params["mu"] * a + params["nu"] * r \
     - (params["sigma"] + params["tau"]) * threatened
dH = params["lambda"] * i + params["rho"] * d + params["kappa"] * a \
     + params["xi"] * r + params["sigma"] * threatened
dE = params["tau"] * threatened
```

Listing 5.4: Ieșirea din compartimentul sever în SIDARTHE.

Aceeași dinamică este exprimată în R și în Julia, cu idiomurile specifice fiecărui ecosistem. În R, funcția returnează o listă de derivate, iar variabilele sunt aduse în scop prin `with(as.list(...))`, ceea ce face codul foarte apropiat de notația matematică:

```r
sidarthe_rhs <- function(t, state, parms) {
  with(as.list(c(state, parms)), {
    force <- (alpha*S_I + beta*D + gamma*A + delta*R) / N
    new_infections <- S * force
    dS <- -new_infections
    dI <- new_infections - (epsilon + zeta + lambda) * S_I
    dD <- epsilon * S_I - (eta + rho) * D
    dA <- zeta * S_I - (theta + mu + kappa) * A
    dR <- eta * D + theta * A - (nu + xi) * R
    dT <- mu * A + nu * R - (sigma + tau) * T
    dH <- lambda*S_I + rho*D + kappa*A + xi*R + sigma*T
    dE <- tau * T
    list(c(dS, dI, dD, dA, dR, dT, dH, dE))
  })
}
```

Listing 5.5: Funcția RHS SIDARTHE în runner-ul R (`deSolve`).

În Julia, convenția *in-place* din `DifferentialEquations.jl` cere ca funcția să scrie derivatele direct într-un vector `du` pre-alocat, ceea ce permite solverului să evite alocările repetate de memorie:

```julia
function sidarthe_rhs!(du, u, p, t)
    S, I, D, A, R, T, H, E = u
    N = p["N"]
    force = (p["alpha"]*I + p["beta"]*D
           + p["gamma"]*A + p["delta"]*R) / N
    new_inf = S * force
```

```
    du[1] = -new_inf
    du[2] = new_inf - (p["epsilon"] + p["zeta"]
                     + p["lambda"]) * I
    du[3] = p["epsilon"]*I - (p["eta"] + p["rho"]) * D
    du[4] = p["zeta"]*I - (p["theta"] + p["mu"]
                         + p["kappa"]) * A
    du[5] = p["eta"]*D + p["theta"]*A
            - (p["nu"] + p["xi"]) * R
    du[6] = p["mu"]*A + p["nu"]*R
            - (p["sigma"] + p["tau"]) * T
    du[7] = p["lambda"]*I + p["rho"]*D + p["kappa"]*A
            + p["xi"]*R + p["sigma"]*T
    du[8] = p["tau"] * T
end
```

Listing 5.6: Funcția RHS SIDARTHE în runner-ul Julia (`DifferentialEquations.jl`, convenție in-place).

Cele trei snippet-uri reprezintă exact *același* sistem ODE; diferențele țin de ecosistemul numeric: R oferă o sintaxă apropiată de notația matematică, Julia urmărește performanța prin alocare zero a vectorului de derivate, iar Python oferă integrare directă cu infrastructura software și cu bibliotecile științifice uzuale. Tabelul 5.2 sintetizează diferențele de implementare pentru cele trei runner-e ODE.

Tabela 5.2: Comparație side-by-side a celor trei runner-e compartimentale, ca instanțe ale aceluiași contract de simulare. Diferențele țin de ecosistemul numeric (biblioteca solver dominantă, convenția de control al pasului), nu de modelul matematic implementat. Numărul de linii include parsarea mesajului, dispecerizarea pe model, scrierea în baza de date și colectarea metricilor.

Limbaj	**Bibliotecă & metodă**	**Pas**	**Linii**
`r`	`deSolve::ode`, RK4 clasic	fix	274
`julia`	`DiffEq.jl`, `Tsit5` (Tsitouras 5(4))	adaptiv	236
`python`	`scipy.solve_ivp`, `RK45` (Dormand-Prince)	adaptiv	259

Trei observații merită notate. Prima: pentru R s-a ales `rk4` cu pas fix $h = 1$ în loc de un solver adaptiv ca `lsoda`, pentru a păstra predictibilitatea timpului de execuție în benchmark-uri — compromisul este o precizie mai mică decât în Julia sau Python, vizibilă în regresia cross-limbaj (Tabelul 8.10), dar funcțional acceptabilă. A doua: Julia și Python folosesc toleranțe identice (rtol $= 10^{-8}$, atol $= 10^{-10}$), ceea ce face traiectoriile lor numerice quasi-identice. A treia: lungimea similară a celor trei runner-e (sub 300 de linii fiecare) arată că abstractizarea contractului comun — mesaj din Redis, dispecerizare pe model, scriere în DB — este suficient de simplă încât adăugarea unui al patrulea limbaj să fie un efort redus.

5.4 Variantele SIR, SIS, SEIR și SIRS

Modelele din familia SIR rămân în sistem pentru comparații, verificări de regresie și explorarea ipotezelor epidemiologice de bază. Ele folosesc același câmp `parameters`, dar cu seturi diferite de rate: `sir` și `sis` folosesc `beta`, `gamma`; `seir` adaugă `sigma` pentru tranziția expuși–infecțioși; iar `sirs` adaugă `xi` pentru pierderea imunității. Condițiile inițiale sunt transmise prin același obiect `initial`, iar rezultatele sunt stocate tot în `compartments`. Această simetrie simplifică interfața web: graficul citește cheile disponibile în fiecare punct temporal și construiește curbele fără să presupună apriori numărul de compartimente.

5.5 Persistența rezultatelor și exportul

Tabela `simulation_runs` reține metadatele execuției: utilizatorul, studiul, setul de date, modelul, limbajul, populația, orizontul temporal și parametrii sub formă JSONB. Câmpul `study_id` păstrează izolarea logică dintre studiile epidemiologice și permite ca aceeași tabelă să conțină simulări aparținând unor echipe diferite. Tabela `simulation_results` reține câte un rând pentru fiecare zi simulată, identificat prin perechea `(simulation_id, t)`. În locul coloanelor fixe `s`, `i`, `r`, rândul conține obiectul JSONB `compartments`. Această decizie evită migrații repetate ale schemei atunci când se adaugă un model cu altă structură de stare.

```
CREATE TABLE simulation_results (
    simulation_id UUID NOT NULL REFERENCES simulation_runs(id) ON DELETE CASCADE,
    t             INTEGER NOT NULL,
    compartments  JSONB NOT NULL,
    PRIMARY KEY (simulation_id, t),
    CHECK (t >= 0),
    CHECK (jsonb_typeof(compartments) = 'object')
);
```

Listing 5.7: Schema generică pentru rezultatele simulărilor.

Exportul CSV este generat dinamic. API-ul inspectează cheile prezente în rezultate, le ordonează în secvența epidemiologică potrivită și scrie doar coloanele existente pentru modelul respectiv. Prin urmare, o simulare SIR se exportă cu coloanele `t,s,i,r`, o simulare SEIR cu `t,s,e,i,r`, iar o simulare SIDARTHE cu `t,s,i,d,a,r,t,h,e`. O predicție LSTM se exportă cu coloanele `t,observed,fitted,predicted`. Această convenție păstrează fișierele lizibile și evită completarea artificială a unor compartimente care nu există în model.

5.6 Metrici de performanță

Toate runnerele colectează aceleași trei metrici: timpul wall-clock, timpul CPU și vârful de memorie rezidentă. Python folosește `time.perf_counter()`, `time.process_time()` și `resource.getrusage`; R folosește `proc.time()` și `/proc/self/status`; Julia folosește `time_ns()` și `Sys.maxrss()`. Valorile sunt scrise în tabela `benchmarks` după inserarea traiectoriilor, în aceeași tranzacție logică în care rularea este marcată ca `completed`. Dacă integrarea numerică sau persistarea eșuează, runner-ul marchează rularea ca `failed` și păstrează mesajul de eroare, astfel încât interfața web să poată diferenția între o simulare aflată încă în lucru și una care a eșuat definitiv.

Capitolul 6
Predicție Epidemiologică folosind Rețele LSTM

6.1 Rolul predicției statistice în sistem

Modelele compartimentale prezentate în capitolul 5 oferă interpretabilitate: fiecare compartiment are o semnificație epidemiologică, iar parametrii descriu rate de tranziție între stări. Această proprietate este esențială pentru o lucrare centrată pe familia SIR și pe extensia SIDARTHE. Seriile epidemiologice reale conțin însă tipare pe care ecuațiile diferențiale nu le modelează explicit: întârzieri de raportare, efecte de weekend, ajustări administrative ale datelor și schimbări bruște de comportament. Aceste tipare apar ca zgomot structurat peste dinamica de transmitere.

De aceea, sistemul include o componentă bazată pe o rețea LSTM (*Long Short-Term Memory*), tratată ca instrument de predicție statistică pe serii temporale. Ea nu înlocuiește modelele SIR, SIS, SEIR, SIRS sau SIDARTHE; adaugă o linie de comparație complet empirică: în loc să presupună o structură de compartimente, învață direct dependența dintre valorile recente ale seriei și valoarea următoare. Capitolul descrie modelul implementat efectiv în runner-ul `runner-lstm`, nu o variantă idealizată. Implementarea curentă este *univariată* (folosește o singură serie, numărul de infectați activi) și *independentă* (nu este cuplată cu un model compartimental). Extensiile naturale spre variante multivariate și hibride sunt discutate separat, în secțiunea 6.9, și sunt marcate explicit ca neimplementate.

6.2 Formularea problemei de predicție

Fie $y_0, y_1, \ldots, y_{n-1}$ seria observată, unde y_t este numărul de infectați activi în ziua t, derivat din datele OWID conform preprocesării descrise în secțiunea 5.1. Predicția este formulată ca regresie pe ferestre glisante: dintr-o fereastră de lungime ℓ (parametrul `lookback`), rețeaua estimează valoarea imediat următoare,

$$\hat{y}_t = f_\theta(y_{t-\ell}, y_{t-\ell+1}, \ldots, y_{t-1}), \tag{6.1}$$

unde θ reunește toți parametrii rețelei. Funcția f_θ produce o predicție pe un singur pas. Predicția pe orizont mai lung se obține *autoregresiv*: ieșirea $\hat{y}_t$ este reintrodusă în fereastră, iar procedeul se repetă pentru fiecare zi din orizontul cerut h (parametrul `forecast_horizon`, egal cu `t_max`).

Această formulare cere două precauții. Ordinea temporală trebuie păstrată strict: o împărțire aleatoare a exemplelor ar permite ca informație din viitor să ajungă în antrenare, producând o evaluare optimistă nerealistă. Apoi, normalizarea seriei trebuie estimată numai pe porțiunea de antrenare, pentru a nu introduce, prin medie și abatere standard, informație din fereastra de test. Ambele cerințe sunt tratate explicit în secțiunile 6.4 și 6.6.

6.3 Arhitectura celulei LSTM

O rețea recurentă simplă suferă de atenuarea gradientului pe secvențe lungi, ceea ce o împiedică să învețe dependențe pe termen lung. Celula LSTM rezolvă această limitare introducând o stare internă c_k protejată de trei porți multiplicative [31]. Implementarea din `runner-lstm`

Tabela 6.1: Corespondența dintre formularea matematică a celulei LSTM și implementarea din `runner-lstm`.

Element	**Notație**	**Reprezentare în cod**
Porțile și candidatul de stare	W_f, W_i, W_o, W_g	matrice `wf, wi, wo, wg` de formă $(H, H{+}1)$
Bias-urile porților	b_f, b_i, b_o, b_g	vectori `bf, bi, bo, bg`; `bf` inițializat la 0,5
Stratul de ieșire	W_y, b_y	`wy` de formă $(1, H)$ și scalarul `by`
Vectorul concatenat	z_k	`z = vstack([h_prev, x])`
Starea celulei și starea ascunsă	c_k, h_k	variabilele `c` și `h`, reportate între pași
Memoria pasului înainte	—	dicționarul `cache`, folosit la backpropagation

construiește celula manual, în NumPy, fără bibliotecă de învățare automată, astfel încât fiecare ecuație să fie vizibilă și verificabilă în cod.

Se notează cu H dimensiunea stării ascunse (parametrul `hidden_size`). La fiecare pas k din fereastră, $1 \le k \le \ell$, intrarea scalară x_k este concatenată cu starea ascunsă anterioară $h_{k-1} \in \mathbb{R}^H$ într-un vector

$$z_k = \begin{bmatrix} h_{k-1} \\ x_k \end{bmatrix} \in \mathbb{R}^{H+1}. \tag{6.2}$$

Din z_k se calculează cele trei porți și candidatul de stare:

$$f_k = \sigma(W_f\, z_k + b_f)\,, \qquad \text{poarta de uitare} \tag{6.3}$$
$$i_k = \sigma(W_i\, z_k + b_i)\,, \qquad \text{poarta de intrare} \tag{6.4}$$
$$o_k = \sigma(W_o\, z_k + b_o)\,, \qquad \text{poarta de ieșire} \tag{6.5}$$
$$\tilde{c}_k = \tanh(W_g\, z_k + b_g)\,, \qquad \text{candidat de stare} \tag{6.6}$$

unde σ este funcția sigmoidă, iar $W_f, W_i, W_o, W_g \in \mathbb{R}^{H\times(H+1)}$ și $b_f, b_i, b_o, b_g \in \mathbb{R}^H$. Starea celulei și starea ascunsă se actualizează prin

$$c_k = f_k \odot c_{k-1} + i_k \odot \tilde{c}_k, \tag{6.7}$$
$$h_k = o_k \odot \tanh(c_k), \tag{6.8}$$

unde $\odot$ este produsul Hadamard. Stările inițiale sunt nule, $h_0 = c_0 = \mathbf{0}$. Poarta de uitare f_k controlează cât din starea anterioară se păstrează, poarta de intrare i_k controlează cât din candidatul $\tilde{c}_k$ se adaugă, iar poarta de ieșire o_k filtrează starea expusă ca h_k. Bias-ul b_f este inițializat la o valoare pozitivă mică, ceea ce menține poarta de uitare aproape deschisă la începutul antrenării și ușurează propagarea gradientului.

După parcurgerea întregii ferestre, predicția pe un pas se obține printr-un strat liniar de ieșire aplicat ultimei stări ascunse:

$$\hat{y} = W_y\, h_\ell + b_y, \qquad W_y \in \mathbb{R}^{1\times H},\ b_y \in \mathbb{R}. \tag{6.9}$$

Tabelul 6.1 pune în corespondență fiecare element matematic cu reprezentarea sa în clasa `LSTMRegressor`. Listing-ul 6.1 reproduce pasul înainte din implementare, în care se recunosc direct ecuațiile (6.7)–(6.9).

```
for value in seq:
    x = np.array([[float(value)]])
```

```
    z = np.vstack([h_prev, x])
    f = _sigmoid(self.wf @ z + self.bf)
    i = _sigmoid(self.wi @ z + self.bi)
    o = _sigmoid(self.wo @ z + self.bo)
    g = np.tanh(self.wg @ z + self.bg)
    c = f * c_prev + i * g
    h = o * np.tanh(c)
y_hat = float((self.wy @ h + self.by)[0, 0])
```

Listing 6.1: Pasul înainte al celulei LSTM în `runner-lstm`.

6.4 Normalizarea și construcția exemplelor

Seriile epidemiologice au scări foarte diferite între regiuni, iar valorile absolute pot ajunge la zeci sau sute de mii de cazuri. Pentru ca funcțiile σ și tanh să lucreze într-un domeniu util, seria este standardizată prin transformarea z-score:

$$\tilde{y}_t = \frac{y_t - \mu}{\varsigma}, \tag{6.10}$$

unde media μ și abaterea standard ς se calculează *exclusiv* pe porțiunea de antrenare a seriei. Dacă abaterea standard este sub un prag numeric, este înlocuită cu 1, pentru a evita împărțirea la zero pe serii aproape constante. Predicțiile rețelei sunt readuse la scara originală prin transformarea inversă $\hat{y}_t = \tilde{y}_t\,\varsigma + \mu$, urmată de o trunchiere la valori nenegative, deoarece numărul de infectați nu poate fi negativ.

Din seria normalizată se construiesc exemple de antrenare prin ferestre glisante: pentru fiecare indice $t \geq \ell$ din porțiunea de antrenare, intrarea este vectorul $(\tilde{y}_{t-\ell}, \ldots, \tilde{y}_{t-1})$, iar ținta este $\tilde{y}_t$. Se obține astfel un set de perechi intrare–țintă pe care rețeaua le folosește la antrenare.

6.5 Antrenarea prin backpropagation through time

Antrenarea minimizează eroarea pătratică dintre predicție și țintă. Pentru un singur exemplu cu ținta y, funcția de pierdere este

$$L = (\hat{y} - y)^2\,, \qquad \frac{\partial L}{\partial \hat{y}} = 2\,(\hat{y} - y)\,. \tag{6.11}$$

Deoarece predicția depinde de toți pașii din fereastră, gradientul trebuie propagat înapoi prin întreaga secvență recurentă. Acest procedeu este cunoscut ca *backpropagation through time* (BPTT) [32].

Gradientul stratului de ieșire se obține direct din ecuația (6.9):

$$\frac{\partial L}{\partial W_y} = 2(\hat{y} - y)\,h_\ell^\top\,, \qquad \frac{\partial L}{\partial b_y} = 2(\hat{y} - y). \tag{6.12}$$

De aici pornește recurența înapoi. Se notează cu δh_k gradientul acumulat al stării ascunse la pasul k și cu δc_k gradientul stării celulei. La pasul final, $\delta h_\ell = W_y^\top\, 2(\hat{y} - y)$. Pentru

$k = \ell, \ell-1, \dots, 1$ se aplică, în ordine,

$$\delta o_k = \delta h_k \odot \tanh(c_k) \odot o_k \odot (1 - o_k), \tag{6.13}$$

$$\delta c_k = \delta h_k \odot o_k \odot \big(1 - \tanh^2(c_k)\big) + \delta c_k^{+}, \tag{6.14}$$

$$\delta f_k = \delta c_k \odot c_{k-1} \odot f_k \odot (1 - f_k), \tag{6.15}$$

$$\delta i_k = \delta c_k \odot \tilde{c}_k \odot i_k \odot (1 - i_k), \tag{6.16}$$

$$\delta \tilde{c}_k = \delta c_k \odot i_k \odot \big(1 - \tilde{c}_k^2\big), \tag{6.17}$$

unde δc_k^{+} este contribuția propagată de la pasul $k+1$ și este nulă la pasul final. Termenii δo_k, δf_k, δi_k și $\delta \tilde{c}_k$ includ deja derivata neliniarității corespunzătoare. Cu aceștia se acumulează gradienții ponderilor,

$$\frac{\partial L}{\partial W_\bullet} \mathrel{+}= \delta_{\bullet,k}\, z_k^{\top}, \qquad \frac{\partial L}{\partial b_\bullet} \mathrel{+}= \delta_{\bullet,k}, \tag{6.18}$$

pentru $\bullet \in \{f, i, o, g\}$, iar gradientul se propagă mai departe către pasul anterior:

$$\delta z_k = W_f^{\top} \delta f_k + W_i^{\top} \delta i_k + W_o^{\top} \delta o_k + W_g^{\top} \delta \tilde{c}_k, \tag{6.19}$$

$$\delta h_{k-1} = [\delta z_k]_{1:H}\,, \qquad \delta c_{k-1}^{+} = \delta c_k \odot f_k. \tag{6.20}$$

Componenta $[\delta z_k]_{1:H}$ reprezintă primele H elemente ale vectorului δz_k, adică partea care corespunde stării ascunse din concatenarea z_k; ultima componentă, asociată intrării x_k, nu este propagată deoarece intrarea este un dat al problemei.

Înainte de actualizare, fiecare gradient este limitat element cu element la intervalul $[-5, 5]$. Această tăiere previne exploziile de gradient, frecvente la rețelele recurente antrenate pe secvențe cu variații mari. Actualizarea parametrilor folosește coborârea pe gradient stochastic, cu rata de învățare η (parametrul `learning_rate`):

$$\theta \leftarrow \theta - \eta\, \nabla_\theta L. \tag{6.21}$$

Antrenarea parcurge setul de exemple de un număr fixat de ori (parametrul `epochs`); în fiecare epocă, exemplele sunt prezentate într-o ordine permutată aleator, astfel încât actualizările succesive să nu fie corelate cu ordinea calendaristică.

6.6 Împărțirea cronologică și evaluarea

Pentru ca rezultatele să măsoare capacitatea de generalizare, seria este împărțită cronologic: ultima fracțiune din puncte (în implementare, 20%) formează fereastra de test, iar restul formează porțiunea de antrenare. Împărțirea respectă o constrângere de mărime — porțiunea de antrenare trebuie să conțină cel puțin un exemplu complet de lungime ℓ — altfel rularea eșuează cu un mesaj explicit. Normalizarea (6.10) și exemplele de antrenare se construiesc numai din porțiunea de antrenare.

După antrenare, rețeaua produce predicții pe un pas pentru întreaga serie. În interiorul porțiunii de antrenare acestea sunt predicții *in-sample*; pe fereastra de test sunt predicții *out-of-sample*, deoarece țintele respective nu au fost văzute la antrenare. Pe fereastra de test se calculează metricile de eroare definite în secțiunea 7.4: eroarea medie absolută MAE, eroarea medie pătratică radicală RMSE și varianta normalizată la populația regiunii nRMSE $=$ RMSE $/N$. Folosirea aceleiași definiții ca în analiza comparativă din capitolul 7 permite așezarea predicției LSTM pe aceeași scară cu modelele compartimentale. Listing-ul 6.2 arată orchestrarea acestor pași în funcția `train_model`.

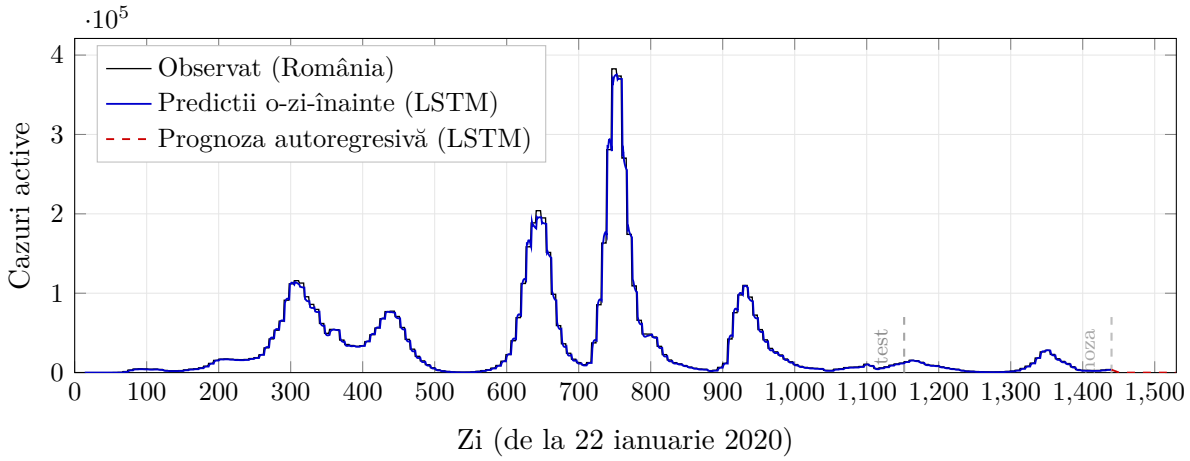


Figura 6.1: Ieșirile runner-ului LSTM dedicat pe seria de cazuri active din România (4 ani, 22 ian. 2020 – 31 dec. 2023), cu antrenare pe primele 80% din puncte și evaluare pe ultimele 20%. Predicțiile o-zi-înainte ($\hat{y}_t = f_\theta(y_{t-14:t-1})$) urmăresc seria observată atât pe fereastra de antrenare cât și pe cea de test, dar prognoza autoregresivă care reintroduce ieșirile în fereastra (dincolo de ziua 1440) tinde rapid către zero — o limitare cunoscută a LSTM-urilor compacte fără mecanisme explicite pentru sezonalitate, asupra căreia un strat ulterior cu informație exogenă (intervalul de raportare, calendarul școlar) ar putea ameliora.

```
n = len(series)
train_end = split_index(n, lookback)
# normalizarea se calibreaza doar pe antrenare
mean, std = fit_scaler(series[:train_end])
normalized = (series - mean) / std
samples = make_samples(normalized[:train_end], lookback)
# ... antrenare pe epoci ...
mae, rmse, nrmse = evaluate(series, fitted, train_end, population)
```

Listing 6.2: Împărțirea cronologică, normalizarea pe antrenare și evaluarea pe test.

În final, rețeaua produce o prognoză autoregresivă pe orizontul cerut, pornind de la ultima fereastră observată: fiecare predicție este reintrodusă în fereastră pentru pasul următor.

Aplicarea concretă a procedurii pe seria România OWID (4 ani, 22 ianuarie 2020 – 31 decembrie 2023) este ilustrată în Figura 6.1. Predicțiile o-zi-înainte urmăresc rezonabil seria observată atât pe fereastra de antrenare cât și pe cea de test, dar prognoza autoregresivă dincolo de ultima zi observată se aplatizează rapid către zero — o limitare cunoscută a LSTM-urilor compacte univariate, care nu au mecanisme explicite pentru tendințele și sezonalitățile pe termen lung. Acest comportament confirmă poziționarea LSTM-ului în această lucrare ca instrument de prognoză pe termen scurt și ca posibil corector de reziduuri, nu ca model autonom de extrapolare epidemiologică pe orizonturi mari.

6.7 Integrarea în arhitectura aplicației

Componenta LSTM este integrată printr-un runner separat, `runner-lstm`, nu prin extinderea runner-ului Python folosit pentru modelele compartimentale. Separarea este justificată: predicția LSTM are alt profil de calcul, alți parametri, altă logică de citire a datelor și altă semantică a rezultatelor. Modelele compartimentale rezolvă un sistem de ecuații diferențiale, în timp ce LSTM antrenează o rețea; amestecarea celor două în același executor ar fi îngreunat și codul, și interpretarea rezultatelor.

Fluxul rămâne compatibil cu arhitectura descrisă în capitolul 5:

- API-ul primește o cerere de predicție cu `model_type = lstm` și rutează jobul către coada Redis dedicată `epimodel:jobs:lstm`;
- `runner-lstm` consumă jobul, citește seria `infected` din setul de date și marchează rularea ca `running`;
- rețeaua este antrenată prin BPTT cu Adam pe porțiunea de antrenare, conform fluxului descris mai sus;
- predicțiile și prognoza sunt persistate în `simulation_results` ca obiecte JSONB, folosind cheile `observed`, `fitted` și `predicted`, iar metricile de calcul (timp, CPU, memorie) sunt scrise în `benchmarks`;
- interfața web afișează cele trei serii suprapuse, distingând vizual porțiunea observată de prognoză.

Parametrii rețelei — `lookback`, `hidden_size`, `epochs` și `learning_rate` — sunt transmiși prin același câmp `parameters` ca pentru modelele compartimentale, ceea ce păstrează contractul comun de simulare. Codul complet al runner-ului este inclus în Anexa D.

Această separare păstrează clară distincția dintre simularea epidemiologică și predicția statistică. Modelul compartimental rămâne responsabil pentru interpretarea mecanismului de transmitere, iar LSTM este folosit pentru prognoză pe termen scurt și ca reper empiric.

6.8 Linii de bază și limitări

O rețea LSTM trebuie comparată cu repere simple, altfel o eroare mică poate fi interpretată greșit drept performanță. Reperele relevante pentru o serie epidemiologică sunt predicția prin persistență, $\hat{y}_t = y_{t-1}$, media mobilă pe ultimele k zile și extrapolarea prin ultima diferență (drift). O componentă LSTM este justificată doar dacă reduce eroarea pe fereastra de test față de aceste repere, evaluată cu protocolul comparativ din capitolul 7.

Această comparație a fost realizată direct, cu eroare o-zi-înainte pe fereastra de test din seria României (ultimele 20% din puncte), prin scriptul `scripts/lstm_baselines.py`. Toate metodele folosesc strict valori observate trecute. Rezultatele, în Tabelul 6.2, sunt clarificatoare: LSTM-ul compact *nu* bate persistența — dimpotrivă, are o eroare MAE de aproximativ trei ori mai mare (755 față de 251 de cazuri), iar toate cele trei repere triviale au erori sub cea a rețelei pe MAE.

Tabela 6.2: Validarea LSTM față de predictori naivi, eroare o-zi-înainte pe fereastra de test (ultimele 20% din seria România, 288 zile). Toate metodele folosesc doar valori observate trecute. *Skill* este reducerea relativă a MAE față de persistența (valori negative = mai slab decât persistența). Pe această serie, 85% din zilele de test sunt identice cu ziua precedentă (raportare săptămânală), ceea ce face persistența extrem de greu de bătut.

Metodă	**MAE**	**RMSE**	**Skill**
LSTM (o-zi-înainte)	755	1114	-200%
Persistență	251	963	—
Drift (ultima diferență)	501	1362	-99%
Medie mobilă (7 zile)	1002	1628	-298%

Explicația nu este un defect de antrenare, ci o proprietate a datelor: 85% din zilele ferestrei de test sunt identice cu ziua precedentă, deoarece seria OWID are o cadență de raportare

săptamânală (actualizări la interval mediu de 7,0 zile, cu valori constante între ele). Pe o astfel de serie, persistența are eroare *exact zero* în 85% din zile, iar un model care prezice o traiectorie zilnică netedă este penalizat tocmai pe zilele plate. Acest rezultat este consistent cu literatura de prognoză a seriilor temporale, în care persistența este un reper notoriu de greu de bătut pentru serii netede sau cu raportare agregată. Interpretarea corectă nu este că LSTM-ul "nu funcționează" — predicțiile o-zi-înainte urmăresc vizual seria, conform Figurii 6.1 — ci că pe acest tip de date, la acest orizont, valoarea adăugată față de un reper trivial este negativă. Aceasta justifică tratarea LSTM-ului ca extensie complementară (corecție de reziduuri, intrare multivariată), nu ca înlocuitor, și motivează direcțiile din secțiunea 6.9.

Limitările sunt intrinsece abordării. LSTM necesită mai multe date decât modelele compartimentale simple, iar pentru regiuni cu serii scurte sau raportări instabile rețeaua poate învăța artefacte administrative în locul dinamicii epidemiologice. Riscul de supraînvățare este ridicat tocmai pentru că seriile sunt scurte și zgomotoase; de aceea evaluarea se face strict pe fereastra de test, niciodată pe cea de antrenare. În plus, rezultatul este puțin interpretabil: o predicție bună numeric nu explică prin ce tranziții epidemiologice se produce evoluția. Din aceste motive, LSTM este prezentat ca extensie complementară, nu ca înlocuitor al analizei modelelor SIR/SIS/SEIR/SIRS/SIDARTHE și nici al calibrării parametrilor epidemiologici.

6.9 Direcții de extindere

Implementarea curentă este deliberat minimală: univariată și independentă de modelele compartimentale. Două extensii sunt naturale, dar *nu* sunt implementate în sistemul descris de această lucrare.

Prima este intrarea *multivariată*. În locul unei singure serii, rețeaua ar putea primi mai multe canale: cazuri noi zilnice, decese, populația regiunii sau ziua săptămânii, pentru a capta efectele de raportare. Acest lucru ar cere extinderea dimensiunii intrării x_k de la un scalar la un vector și ajustarea corespunzătoare a lățimii matricelor de poartă.

A doua este modelul *hibrid compartimental–LSTM*. Modelul compartimental produce o predicție interpretabilă $\hat{y}_t^{(c)}$, iar LSTM ar putea învăța reziduul $r_t = y_t - \hat{y}_t^{(c)}$ în loc de seria brută. Predicția finală ar fi

$$\hat{y}_t^{(\text{hibrid})} = \hat{y}_t^{(c)} + f_\theta(r_{t-\ell}, \ldots, r_{t-1})\,. \tag{6.22}$$

Această variantă ar păstra rolul central al modelului epidemiologic, lăsând rețeaua să corecteze doar deviațiile statistice rămase după simularea compartimentală. Dacă reziduurile au un tipar stabil, modelul hibrid ar putea îmbunătăți predicția pe termen scurt fără a pierde interpretabilitatea.

Capitolul prezintă rezultate operaționale pe România (Figura 6.1), cost de execuție concret în Tabelul 8.8 din capitolul 8 și o validare predictivă față de linii de bază în Tabelul 6.2. Această validare, realizată pe seria României, arată că reperul de persistență rămâne superior la orizontul de o zi. Extinderea naturală constă în repetarea comparației pe mai multe regiuni, cu modelul compartimental calibrat ca reper suplimentar și cu varierea hiperparametrilor (ℓ, H, η, E), precum și în evaluarea variantei hibride din ecuația (6.22), în care rețeaua corectează reziduuri în loc să prezică seria brută — singurul regim în care, plauzibil, ar putea depăși un reper trivial.

Capitolul 7
Comparația Modelelor, Analiză Numerică și Calibrare

7.1 Rolul comparației între modele

Modelele compartimentale discutate în lucrare nu sunt variante interschimbabile ale aceleiași ecuații. Fiecare introduce o anumită ipoteză epidemiologică și elimină alte detalii pentru a păstra sistemul analizabil. De aceea, comparația trebuie făcută pe două planuri: structural (al ipotezelor) și numeric (al traiectoriilor obținute pe același set de date).

Comparația structurală răspunde la întrebarea „ce fenomen poate reprezenta modelul?”. Comparația numerică răspunde la întrebarea „cât de bine reproduce modelul datele observate în condiții experimentale identice?”. Niciuna dintre ele nu este suficientă singură: un model simplu poate avea eroare mică pe o fereastră scurtă, dar poate ascunde mecanisme importante, iar un model complex poate reprezenta mai fidel fenomenul, dar poate fi dificil de calibrat din date puține.

7.2 Comparație structurală

Tabelul 7.1 sintetizează diferențele principale dintre modelele suportate de aplicație. Criteriile alese sunt cele relevante pentru implementarea curentă: compartimentele, parametrii necesari, ipoteza centrală și tipul de date observabile cu care modelul poate fi comparat direct.

SIS este potrivit pentru boli unde reinfectarea rapidă sau lipsa imunității este o ipoteză acceptabilă. SIR este util ca reper matematic, deoarece păstrează dinamica minimală a unei epidemii cu imunitate. SEIR introduce latența și devine mai potrivit atunci când există o perioadă de incubație importantă. SIRS relaxează ipoteza imunității permanente. SIDARTHE mută accentul de la agregarea biologică la observabilitatea clinică: infecțiile nedepistate, cazurile diagnosticate, formele simptomatice, formele severe, vindecările și decesele sunt tratate separat.

7.3 Operatorul de observație

Datele reale nu conțin de obicei toate compartimentele modelului. Pentru a compara un model cu o serie observată este necesar un operator de observație $g_m(\mathbf{x}(t))$, dependent de modelul m, care transformă starea internă a modelului într-o mărime comparabilă cu datele.

Pentru o serie de cazuri active, operatorul poate fi definit astfel:

$$g_{\mathrm{SIR}}(\mathbf{x}) = I, \qquad g_{\mathrm{SEIR}}(\mathbf{x}) = I, \qquad g_{\mathrm{SIRS}}(\mathbf{x}) = I. \tag{7.1}$$

Pentru SIDARTHE, dacă seria observată reprezintă cazuri diagnosticate active, o alegere mai realistă este

$$g_{\mathrm{SIDARTHE}}(\mathbf{x}) = D + R + T, \tag{7.2}$$

deoarece compartimentul I reprezintă infectați nedepistați, iar A reprezintă persoane simptomatice nedepistate. Dacă setul de date conține doar o aproximare generală a cazurilor active, aplicația poate folosi o proiecție mai simplă, dar interpretarea trebuie notată explicit.

Tabela 7.1: Comparație structurală între modelele utilizate în lucrare.

Model	**Compartimente**	**Parametri**	**Ipoteză principală**	**Observabil comparabil**
SIS	S, I	β, γ	infecția nu produce imunitate durabilă	cazuri active aproximabile prin I
SIR	S, I, R	β, γ	recuperarea produce imunitate permanentă	active prin I, cumulate prin $I + R$
SEIR	S, E, I, R	β, σ, γ	există o fază latentă neinfecțioasă sau slab observabilă	active prin I, latența rămâne neobservată
SIRS	S, I, R	β, γ, ξ	imunitatea se pierde în timp	active prin I, revenire din R în S
SIDARTHE	S, I, D, A, R, T, H, E	16 rate	diagnosticul, simptomatologia și severitatea clinică modifică dinamica observată	active diagnosticate prin $D + R + T$, decese prin E

Pentru decese cumulative, operatorul SIDARTHE este direct:

$$g_{\text{decese}}(\mathbf{x}) = E. \tag{7.3}$$

Această separare este un avantaj al modelului SIDARTHE față de modelele SIR, SEIR sau SIRS, în care decesele sunt incluse implicit în compartimentul *removed*.

7.4 Metrici de eroare numerică

Fie y_t seria observată la ziua t, iar $\hat{y}_{m,t}$ seria produsă de modelul m după aplicarea operatorului de observație. Pentru o fereastră de evaluare cu T puncte, eroarea medie absolută este

$$\text{MAE}_m = \frac{1}{T} \sum_{t=1}^{T} \left| y_t - \hat{y}_{m,t} \right|. \tag{7.4}$$

Eroarea medie pătratică radicală este

$$\text{RMSE}_m = \sqrt{\frac{1}{T} \sum_{t=1}^{T} \left(y_t - \hat{y}_{m,t} \right)^2}. \tag{7.5}$$

Pentru a compara regiuni cu populații diferite, este utilă normalizarea la populația totală N:

$$\text{nRMSE}_m = \frac{\text{RMSE}_m}{N}. \tag{7.6}$$

În date epidemiologice, MAPE poate fi instabilă atunci când y_t este aproape de zero. Din acest motiv, lucrarea preferă MAE, RMSE și nRMSE. Dacă se compară mai multe serii

observabile, de exemplu cazuri active și decese, se folosește o funcție de pierdere ponderată:

$$\mathcal{L}_m(\theta) = \sum_{k \in \mathcal{K}} w_k \frac{1}{T} \sum_{t=1}^{T} \left(y_{k,t} - g_{m,k}(\mathbf{x}_\theta(t)) \right)^2, \tag{7.7}$$

unde k indexează tipul de observație, iar w_k controlează importanța fiecărei serii. Ponderile sunt necesare deoarece seriile pot avea scări diferite.

Aplicarea concretă a acestor metrici pe rezultate de calibrare pe România și pe alte trei țări europene este prezentată în subsecțiunile 7.6.1 și 7.7.2. Tabelul 7.4 raportează MAE pe ferestrele de antrenare și test pentru cele patru regiuni, iar extinderea aceleiași proceduri la o matrice completă model–limbaj–fereastră este o continuare directă a campaniei de benchmark din capitolul 8.

7.5 Protocol de analiză comparativă

Pentru ca analiza comparativă să fie reproductibilă, toate modelele trebuie rulate în condiții identice. Protocolul propus are următorii pași:

1. se alege același set de date, aceeași regiune și aceeași fereastră temporală pentru toate modelele;
2. se aplică aceeași regulă de inițializare a compartimentelor, conform Tabelului 5.1;
3. se calibrează fiecare model pe aceeași fereastră de antrenare;
4. se evaluează eroarea pe o fereastră separată de test;
5. se raportează MAE, RMSE, nRMSE și timpul de execuție;
6. se păstrează separat interpretarea epidemiologică și costul software.

Separarea între antrenare și test este importantă. Un model cu mulți parametri, cum este SIDARTHE, poate potrivi mai bine fereastra de antrenare, dar aceasta nu garantează o predicție mai bună pe zile viitoare. Prin urmare, analiza comparativă trebuie să raporteze atât eroarea de calibrare, cât și eroarea de generalizare.

7.6 Calibrarea parametrilor

Calibrarea parametrilor transformă modelul dintr-un instrument de simulare exploratorie într-un instrument ajustat la date observate. Pentru un model m cu vector de parametri θ, calibrarea poate fi formulată ca problemă de optimizare:

$$\hat{\theta}_m = \arg\min_{\theta \in \Omega_m} \mathcal{L}_m(\theta), \tag{7.8}$$

unde Ω_m este domeniul admisibil al parametrilor. Constrângerile sunt esențiale: ratele de tranziție trebuie să fie nenegative, iar intervalele lor trebuie alese în funcție de scară temporală și de cunoștințe epidemiologice.

Pentru modelele SIR/SIS/SEIR/SIRS, calibrarea poate fi făcută cu metode clasice de optimizare neliniară, de exemplu Nelder–Mead [33], L-BFGS-B sau căutare pe grilă pentru intervale mici. Pentru SIDARTHE, spațiul de parametri este mai mare, iar optimizarea devine mai fragilă. În acest caz sunt utile:

Tabela 7.2: Locurile din analiza comparativă care necesită testare pentru obținerea rezultatelor.

Secțiune	Test necesar	Rezultat de raportat
Operatorul de observație	alegerea proiecției potrivite pentru fiecare model și set de date	seria comparabilă cu datele reale: active, diagnosticate sau decese
Metrici de eroare	rularea modelelor pe aceeași fereastră temporală	MAE, RMSE, nRMSE pe fiecare model și limbaj
Protocol comparativ	împărțirea cronologică train/-test și rulări identice pentru toate modelele	tabel agregat cu eroare pe train și test
Calibrare parametri	optimizarea parametrilor în limite admisibile	parametrii calibrați, eroarea finală și numărul de evaluări ale solverului
Identificabilitate	repetarea calibrării din puncte inițiale diferite	stabilitatea parametrilor și variația erorii între rulări

- fixarea unor parametri din literatură sau din ipoteze clinice;
- calibrarea pe grupuri de parametri, nu a tuturor ratelor simultan;
- regularizarea față de valori de referință;
- validarea pe o fereastră temporală nefolosită la optimizare.

7.6.1 Aplicare pe România, valul 1 din 2020

Procedura este aplicată concret modelului SIR pe seria de cazuri active din România (sursa OWID, vezi capitolul 5), pe valul epidemic de primăvară-vară 2020. Implementarea este în `scripts/calibrate.py` și folosește `scipy.optimize.minimize` cu metoda Nelder–Mead, restartată din opt puncte inițiale aleatoare pentru a reduce sensibilitatea la condiția inițială. Fereastra este împărțită cronologic: 15 martie – 15 august 2020 (154 zile) pentru antrenare, 16 august – 15 septembrie 2020 (31 zile) pentru test. Funcția de pierdere este eroarea pătratică medie pe seria de cazuri active, iar operatorul de observație este $g_{\mathrm{SIR}}(\mathbf{x}) = I$.

Calibrarea este executată în două variante:

- *liber*: optimizare simultană a (β, γ) în $\Omega = (0, 5] \times (0, 5]$;
- *constrâns*: γ fixat la $1/14$ zi^{-1}, valoarea de referință din literatura clinică pentru COVID-19 (durata medie a perioadei infecțioase ~ 14 zile), iar optimizarea se face doar pe β.

Comparația este prezentată în Tabelul 7.3 și Figura 7.1.

Rezultatele evidențiază două aspecte caracteristice acestei clase de probleme. Pe de o parte, eroarea pe antrenare este aproape identică între cele două scenarii (MAE $\approx 1\,600$ cazuri pentru ambele), deși parametrii estimați diferă substanțial: $\hat{\gamma}_{\mathrm{liber}} = 0{,}335$ corespunde unei durate infecțioase medii de 3 zile, valoare care nu se susține clinic pentru COVID-19. Constrângerea $\gamma = 1/14$ produce un $\hat{\beta}$ de $\sim 3{,}5$ ori mai mic și un $\hat{\mathcal{R}}_0$ de 1,45, mai apropiat de intervalul de literatură 2–3 pentru tulpina originală, dar tot sub această limită din cauza datelor agregate ale unei singure regiuni într-o singură fereastră.

Pe de altă parte, eroarea pe fereastra de test crește de aproximativ un ordin de mărime față de antrenare (13 031 cazuri pentru fitul liber, 16 727 pentru cel constrâns). Modelul SIR

Tabela 7.3: Calibrare SIR pe România, valul 1 (15 martie – 15 august 2020 antrenare, 16 august – 15 septembrie 2020 test). Două scenarii: *liber* (optimizează ambele rate, (β, γ)), *constrâns* (folosește $\gamma = 1/14$ din durata clinică medie a infecției COVID-19, optimizează doar β). Comparația ilustrează tensiunea dintre minimizarea pierderii și interpretabilitatea epidemiologică a parametrilor.

Marime	**Liber**	**Constrâns**	**Observatii**
$\hat{\beta}$ (zi^{-1})	0.3677	0.1039	rata de transmitere
$\hat{\gamma}$ (zi^{-1})	0.3346	0.0714 (*fix*)	$1/\hat{\gamma}$ liber = 3.0 zile (non-fizic), constrâns = 14 zile
$\hat{\mathcal{R}}_0$	1.10	1.45	valori de literatură pentru tulpina originala $\sim$ 2–3
MAE antrenare	1630	1612	cazuri
RMSE antrenare	2199	2203	cazuri
MAE test	13031	16727	extrapolare 1 luna
RMSE test	14889	19371	cazuri

cu coeficienți constanți nu poate extrapola al doilea val epidemic, care apare în septembrie-octombrie 2020 cu β efectiv mai mare decât în vară. Aceasta este o limitare structurală a SIR clasic, nu o problemă de calibrare.

7.7 Identificabilitate și limitări ale calibrării

O eroare mică nu implică automat parametri epidemiologic corecți. În modelele neliniare, mai multe combinații de parametri pot produce traiectorii asemănătoare pe aceeași fereastră de timp. Această problemă se numește neidentificabilitate practică și este accentuată atunci când datele observate sunt agregate.

Pentru SIDARTHE, neidentificabilitatea este mai importantă decât pentru SIR. Dacă setul de date nu conține date despre testare, spitalizare, terapie intensivă și vindecări, parametri precum ϵ, ζ, μ, ν, σ sau τ nu pot fi estimați robust doar din cazuri active. Într-o lucrare aplicată, acest lucru trebuie declarat explicit. Rolul calibrării nu este să ascundă incertitudinea, ci să arate ce ipoteze sunt necesare pentru a obține o potrivire numerică.

7.7.1 Vizualizarea identifiabilității practice pentru SIR

Pentru a face explicit acest fenomen, peisajul de pierdere MSE(β, γ) a fost calculat pe o grilă 56×36 în domeniul $\beta \in [0{,}05, 0{,}60]$, $\gamma \in [0{,}05, 0{,}40]$, pentru fereastra de antrenare de mai sus (Figura 7.2). Reprezentarea în logaritm relevă o vale alungită, aproape liniară în planul (β, γ): combinații cu raport β/γ aproximativ constant produc traiectorii similare pe fereastra observată. În consecință, calibrarea liberă nu distinge între $(0{,}368, 0{,}335)$ și $(0{,}104, 0{,}0714)$ pe baza datelor din valul 1 — informația lipsește în date, nu în algoritm.

Această observație justifică restricționarea γ din literatura clinică ca o practică recomandabilă, nu ca o limitare arbitrară. Mai mult, rezolvarea unei astfel de identifiabilitati slabe pentru SIDARTHE necesită informații suplimentare în date — testare, spitalizare, terapie intensivă, decese — care permit fixarea sau constrângerea unor parametri prin operatori de observație multipli, nu doar serie singulară de cazuri active.

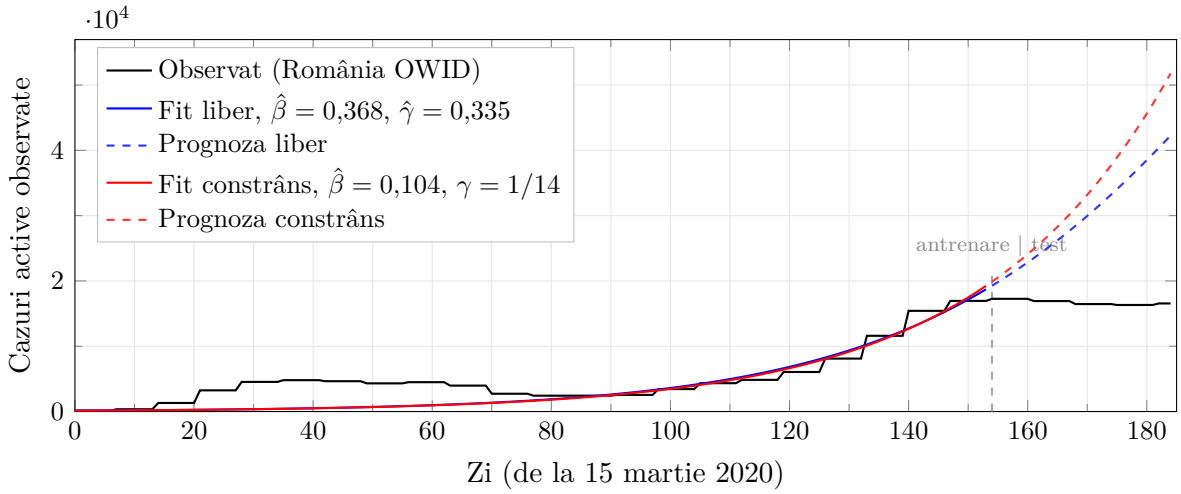


Figura 7.1: Calibrare SIR pe România, valul 1 din 2020. Linia neagră arată seria observată; cele două scenarii produc fituri vizual aproape identice pe fereastra de antrenare (MAE $\approx 1\,600$ cazuri pentru ambele), dar prognoza pe luna următoare diverge (MAE 13 031 pentru fitul liber vs 16 727 pentru cel constrâns). Parametrii estimați diferă substantial, ilustrând identifiabilitatea practică slaba: aceleași date sunt explicate aproape la fel de bine de combinații (β, γ) foarte diferite.

O ieșire concretă: calibrare bivariată folosind decesele

Îmbogățirea schemei datelor cu metrici epidemiologice suplimentare (decese cumulate, R_t observat, indici de stringență; Section 5.1) oferă levierul direct pentru a rupe valea identifiabilității slabe descrise mai sus. Pentru SIR cu rată de mortalitate μ, o calibrare bivariată în care funcția de pierdere agregă MAE pe seria $I(t)$ și pe seria $D(t)$:

$$\mathcal{L}(\beta, \gamma, \mu) = w_I \cdot \mathrm{MAE}_I + w_D \cdot \mathrm{MAE}_D, \tag{7.9}$$

selectează combinațiile (β, γ) consistente cu *ambele* serii simultan. Geometric, vorbim despre intersecția a două văi în spațiul de pierdere — una pentru fiecare serie observată — care, în general, se reduce la o vecinătate strâmtă, identifiabilă, dacă seriile sunt informative independent. Aceeași logică se aplică la SIDARTHE: cu seriile D (diagnosticate), T (severe spitalizate) și E (decese) disponibile prin coloanele `deaths` și (în lucrări viitoare) `icu` și `hospitalized`, mai mulți parametri se pot estima simultan, fără a se sprijini integral pe constrângeri din literatura clinică. Această direcție este menționată în limitările capitolului 10 și a devenit tehnic accesibilă odată cu îmbogățirea schemei de date din Section 5.1; implementarea concretă rămâne o extensie naturală a scriptului `calibrate.py`.

7.7.2 Generalizare pe mai multe regiuni

Pentru a verifica dacă framework-ul de calibrare funcționează cross-regiune și pentru a contextualiza valorile estimate pentru România, aceeași procedură constrânsă (cu $\gamma = 1/14$) a fost aplicată pe patru țări europene cu scări de populație și timing-uri ale valului 1 distincte: România, Italia, Germania și Spania. Rezultatele sunt prezentate în Tabelul 7.4 și Figura 7.3. Implementarea este în `scripts/calibrate_multiregion.py`.

$\hat{\mathcal{R}}_0$ estimat variază între 0,94 (Italia) și 1,45 (România), reflectând atât timing-ul diferit al începerii valului în fiecare țară (Italia este deja în faza de declin la începutul ferestrei, România abia începe), cât și intensitatea măsurilor non-farmaceutice deja în vigoare. Toate cele patru valori sunt sub intervalul de literatură 2–3 pentru tulpina originală, ceea ce este consistent

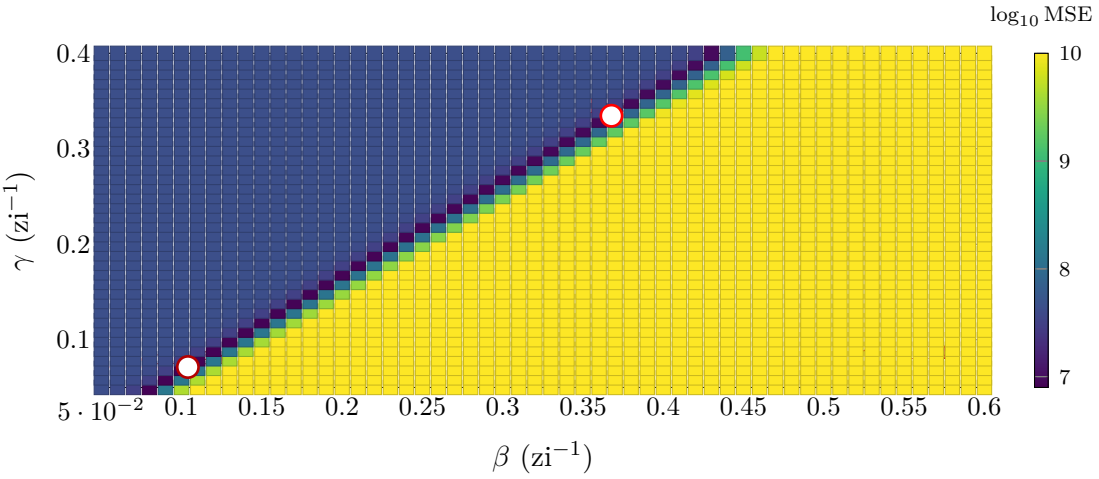


Figura 7.2: Peisajul de pierdere $\log_{10}\mathrm{MSE}(\beta,\gamma)$ pentru modelul SIR pe fereastra de antrenare. Punctele roșii marchează soluția liberă $(0{,}368, 0{,}335)$ și soluția constrânsă $(0{,}104, 0{,}0714)$; linia roșie punctata indica restricția clinică $\gamma = 1/14$. Valea de pierdere mica este alungită și aproape liniara, ceea ce înseamnă ca multe combinații (β,γ) produc o potrivire similara cu datele observate din valul 1 — semnătura tipică a identifiabilității practice slabe pentru SIR cu o singură serie observată.

Tabela 7.4: Calibrare SIR cross-regiune pe valul 1 al pandemiei COVID-19 (15 martie – 15 august 2020 antrenare, 16 august – 15 septembrie 2020 test). În toate cele patru cazuri γ este fixat la valoarea de literatură 1/14 zile, iar β este optimizat prin Nelder-Mead. Valorile $\hat{\beta}$ și $\hat{\mathcal{R}}_0$ sunt comparabile direct între țări, dar erorile absolute reflectă scările diferite de populație (coloana N) și de I_0.

Tara	N (mil.)	I_0	$\hat{\beta}$	$\hat{\mathcal{R}}_0$	Observatii	MAE train	MAE test
România	19.7	135	0.1039	1.45	infectie inca incipienta la inceputul ferestrei	1612	16727
Italia	59.0	20029	0.0673	0.94	deja în plin val 1 ($I_0 \sim 20\,000$)	16243	5318
Germania	83.4	7346	0.0750	1.05	raspuns puternic la masuri	14118	3616
Spania	47.6	13026	0.0759	1.06	val 2 începe sa crească la finalul ferestrei	22214	77036

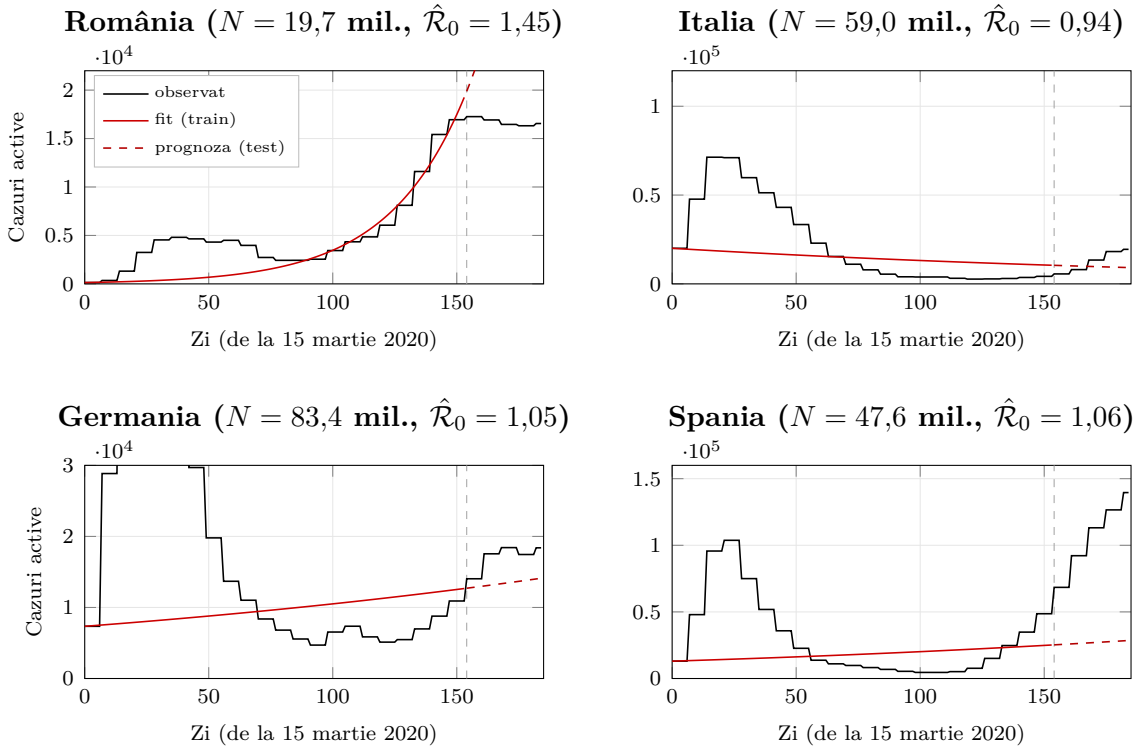


Figura 7.3: Calibrare SIR cu $\gamma = 1/14$ pe valul 1 al pandemiei COVID-19, în patru țări europene cu scări de populație diferite (atenție la scările y distincte). Linia neagră este seria observată, linia roșie continuă este fitul pe fereastra de antrenare (15 mar. – 15 aug. 2020), iar linia roșie întreruptă este prognoza pe luna următoare. Linia verticală marchează granița antrenare/test. Modelul reproduce calitativ valul 1 în toate cele patru cazuri; divergența pe test este vizibilă în Spania, unde valul 2 începe sa crească rapid în septembrie, evidențiind limitarea structurală a SIR cu coeficienți constanți.

cu calibrarea pe o fereastră în care intervențiile au acționat deja asupra ratei de transmitere efective. Procedura funcționează uniform și fără ajustări per caz: aceeași optimizare Nelder-Mead converge pentru toate cele patru țări. Spania rămâne cazul cu cea mai mare eroare pe test ($\sim$77 000 cazuri MAE), direct atribuibilă începerii valului doi la finalul ferestrei de test, limitare structurală a SIR cu coeficienți constanți, nu deficiență de calibrare. Modelele cu rate variabile în timp (sau modelele compartimentale care includ vaccinare sau imunitate care se pierde) ar fi necesare pentru a extrapola peste un schimb de regim epidemic.

7.8 Integrarea în sistemul software

Arhitectura curentă este pregătită pentru calibrare deoarece separă trei elemente: datele observate, parametrii modelului și rezultatele simulării. Extinderea naturală constă într-un tip suplimentar de job, de exemplu `calibration`, care rulează mai multe simulări candidate și salvează parametrii rezultați într-o tabelă dedicată. Simularea finală ar putea fi apoi creată automat folosind $\hat{\theta}$.

Această extensie trebuie totuși tratată diferit de o simulare obișnuită. Calibrarea poate necesita zeci sau sute de integrări numerice, deci timpul de execuție, costul CPU și consumul de memorie devin parte a rezultatului. Din acest motiv, evaluarea numerică și evaluarea software trebuie raportate împreună: modelul cu eroare mai mică poate fi mai costisitor, iar modelul mai rapid poate fi insuficient pentru datele disponibile.

7.8.1 Costul software al calibrării

Pentru a cuantifica acest cost s-a măsurat direct calibrarea SIR pe România (scenariile liber și constrâns împreună, cu restarturi multiple Nelder–Mead), prin scriptul `scripts/bench_calibration_cost.py`. Calibrarea a necesitat 1 377 de simulări candidate — adică 1 377 de evaluări ale funcției obiectiv, fiecare o integrare numerică completă pe fereastra de 154 de zile — realizate *in-process* în 17,2 secunde, cu un consum de memorie de vârf sub 1 MB. Acesta este numărul real de integrări, nu o estimare.

Întrebarea de proiectare este cum se traduce acest cost într-un job al aplicației. Tabelul 7.5 compară două arhitecturi. În varianta *naivă*, fiecare simulare candidată ar fi dispecerizată ca job separat prin aplicație (Redis → runner → scriere în YugabyteDB). Folosind latența end-to-end măsurată a unui job SIR real la acest orizont (∼1 047 ms/job, mediană pe 8 rulări), cele 1 377 de candidate ar costa ∼1 442 de secunde (aproape 24 de minute) și ar scrie peste 210 000 de rânduri în baza de date — de circa 84 de ori mai scump decât calibrarea in-process, fără niciun câștig de acuratețe.

Tabela 7.5: Costul software al calibrării SIR (România, valul 1; fereastra de 154 zile, restart-uri multiple Nelder–Mead pentru scenariile liber și constrâns). Coloana *în-process* măsoară direct costul procedurii din `calibrate.py`; coloana *naiv (ca job-uri)* proiectează costul dacă fiecare simulare candidată ar fi un job separat prin aplicație, folosind latența end-to-end măsurată a unui job SIR real (1047 ms/job); coloana *recomandat* corespunde calibrării într-un singur job, cu o singură simulare finală scrisă în baza de date. Numărul de simulări candidate este numărul real de evaluări ale funcției obiectiv.

Marime	**În-process**	**Naiv (ca job-uri)**	**Recomandat**
Simulări candidate	1377	1377	1377
Job-uri prin aplicație	1	1377	1
Randuri scrise în DB	0	213435	155
Timp total wall (s)	17.24	1441.8*	17.24
Timp CPU (s)	17.23	—	17.23
Memorie de varf (MB)	0.7	—	0.7

* Proiectat: 1377×1047 ms $= 1442$ s; de ∼84× mai scump decât calibrarea în-process, fără castig de acuratete — overhead-ul de coada și persistare ar fi platit pentru fiecare integrare candidată.

În varianta *recomandată*, calibrarea rulează in-process într-un singur job de tip `calibration`, iar în baza de date se scrie doar simularea finală obținută cu $\hat{\theta}$ (un singur set de rezultate). Costul de calcul rămâne cel măsurat (17,2 s), dar overhead-ul de coadă și de persistare — care în varianta naivă ar fi plătit de 1 377 de ori — este amortizat o singură dată. Concluzia de proiectare este clară: calibrarea nu trebuie modelată ca N simulări independente, ci ca un singur job care încapsulează bucla de optimizare și persistă numai rezultatul final. Această observație justifică tratarea calibrării ca tip de job distinct, cu propriile metrici de cost raportate alături de eroarea numerică, exact cum recomandă secțiunea anterioară.

Capitolul 8
Evaluarea Software, Performanței și Scalabilității

8.1 Scopul evaluării

Evaluarea software completează analiza numerică din Capitolul 7. Dacă analiza numerică măsoară cât de bine se potrivește un model cu datele, evaluarea software măsoară cât costă obținerea acelui rezultat într-un sistem real. În aplicația implementată, simularea nu este un apel local către o funcție, ci un flux distribuit: API-ul creează o rulare, Redis transportă job-ul, runner-ul execută solver-ul, YugabyteDB persistă rezultatele, iar frontend-ul afișează statusul și graficul.

Prin urmare, evaluarea trebuie să separe patru componente:

- costul numeric al solverului;
- costul de serializare și persistare a rezultatelor;
- costul operațional introdus de coadă și de baza de date;
- costul de utilizare perceput în interfața web.

Această separare este importantă deoarece optimizarea unei singure componente nu garantează îmbunătățirea întregului sistem. Un solver rapid poate fi mascat de scrieri lente în baza de date, iar un model cu puține compartimente poate deveni costisitor dacă este rulat de multe ori în cadrul calibrării.

8.2 Metrici colectate

Tabela 8.1: Metricile colectate sau derivabile pentru evaluarea software.

Metrică	**Nivel**	**Interpretare**
`wall_time_ms`	runner	durata observată între începutul execuției și finalizarea persistării rezultatelor
`cpu_time_ms`	runner	timpul de procesor consumat de procesul numeric
`peak_memory_mb`	runner	vârful de memorie rezidentă raportat de runtime
`queue_wait_ms`	sistem	timpul dintre crearea rulării și preluarea job-ului de către runner
`result_rows`	bază de date	numărul de puncte temporale persistate pentru rulare
`compartment_count`	model	dimensiunea vectorului de stare salvat la fiecare pas
`api_latency_ms`	API	timpul de răspuns pentru crearea unei simulări și citirea rezultatului

Primele trei metrici (`wall_time_ms`, `cpu_time_ms`, `peak_memory_mb`) sunt scrise de fiecare runner în tabela `benchmarks` și sunt disponibile pentru toate rulările din campania descrisă în secțiunea 8.7.1. Metricile derivate (`queue_wait_ms`, latențe API, dimensiunea răspunsului JSON) ar necesita o instrumentare suplimentară și nu fac parte din rezultatele actuale; ele apar în Tabelul 8.1 pentru completitudine a cadrului de evaluare.

8.3 Matricea experimentală

O evaluare completă trebuie să acopere toate combinațiile relevante dintre model, limbaj și orizont temporal. Matricea minimă recomandată este prezentată în Tabelul 8.2.

Tabela 8.2: Matrice minimă pentru campania de benchmark.

Dimensiune	**Valori**	**Scop**	**Observații**
Model	SIS, SIR, SEIR, SIRS, SIDARTHE	cost în funcție de complexitate	două până la opt compartimente
Limbaj	R, Julia, Python	comparație între ecosisteme	prima rulare Julia trebuie separată de rulările calde
Orizont $t_{\max}$	365, 2 000, 10 000, 50 000 zile	scalare cu numărul de pași de integrare și de puncte salvate	singurul lever real pentru rulările ODE; lungimea setului de date nu afectează durata solverului
Repetări	minim 10	robustețe statistică	raportare prin mediană și IQR
Concurență	1, 3, 5 job-uri	comportament operațional	măsoară coadă, DB și scheduling

Această matrice permite două tipuri de întrebări. Prima este numerică: cât crește durata când modelul trece de la două compartimente la opt? A doua este operațională: cum se schimbă timpul total când mai multe simulări sunt lansate simultan?

Două precizări sunt importante pentru interpretarea matricei. (i) Pentru runnerele compartimentale (R, Julia, Python), durata unei rulări nu depinde de mărimea setului de date — runnerul citește doar prima înregistrare pentru condițiile inițiale și integrează $t_{\max}$ pași. Levierul real pentru rulări lungi este, prin urmare, $t_{\max}$, complexitatea modelului și toleranțele solverului. (ii) Pentru runnerul LSTM, dimpotrivă, costul de antrenare scalează cu lungimea seriei (numărul de eșantioane glisante), cu numărul de epoci și cu dimensiunea stării ascunse; în această privință, seturile sintetice de zeci de mii de zile sunt esențiale pentru evaluarea regimului saturat al rețelei.

Execuția campaniei este automatizată prin scriptul `scripts/run_benchmark.py`, care primește credențialele super-admin, identificatorii unui studiu și ai unuia sau mai multor seturi de date, lista modelelor, lista limbajelor, valorile $t_{\max}$ de baleiat și numărul de repetări. Driverul lansează combinațiile prin API (`POST /api/studies/{id}/simulations`), așteaptă finalizarea fiecărei rulări prin polling și măsoară latența end-to-end pe partea de client (de la

crearea simulării până la statusul `completed`), care include coada Redis, solverul și persistarea — exact descompunerea propusă mai sus. Rezultatele sunt scrise într-un CSV per campanie, iar metricile interne ale solverului (`wall_time_ms`, `cpu_time_ms`, `peak_memory_mb`) rămân disponibile în tabela `benchmarks`, agregabile prin `GET /api/studies/{id}/benchmarks`.

Tabela 8.3: Teste necesare pentru obținerea rezultatelor de evaluare software.

Zonă evaluată	**Test necesar**	**Rezultat de raportat**
Runner numeric	rulări repetate pentru fiecare model, limbaj și orizont temporal	mediană, IQR, minim și maxim pentru timp și memorie
Cold-start/Warm-start	separarea primei rulări de rulările ulterioare	costul de inițializare al runtime-ului, în special pentru Julia
Concurență	lansarea simultană a 1, 3 și 5 job-uri	timp total, timp de așteptare în coadă și degradarea latenței
Persistență rezultate	rulări cu 30, 90, 180 și 365 puncte temporale	timp de scriere, număr de rânduri și dimensiunea JSONB
API și frontend	citirea rezultatelor și exportul CSV pentru fiecare model	latență endpoint, dimensiune răspuns și timp de export

8.4 Protocol de rulare

Pentru rezultate reproductibile, campania de benchmark trebuie să respecte un protocol fix:

1. se pornește aceeași versiune de cod și aceeași configurație Docker;
2. se rulează migrațiile bazei de date și se curăță datele experimentale vechi;
3. se execută rulări de încălzire pentru fiecare limbaj și model;
4. se rulează combinațiile din matricea experimentală;
5. se exportă tabela `benchmarks` și metadatele din `simulation_runs`;
6. se agregă rezultatele prin mediană, IQR, minim și maxim;
7. se păstrează separat rezultatele cold-start și warm-start.

Separarea cold-start/warm-start este necesară în special pentru Julia, unde prima rulare poate include cost de compilare JIT. Pentru Python și R, costurile mari apar de obicei la pornirea containerului și la importul bibliotecilor, nu la fiecare job.

Campania de bază a fost executată pe clusterul `kube-4kdhb` (3 worker-i × 4 vCPU / 8 GiB pe OpenStack), folosind imaginile `emedicine-*:latest` de pe GHCR, construite din branch-ul principal. Rezultatele sunt prezentate în secțiunea 8.7.1; reproducerea pe alte configurații hardware (de exemplu o mașină locală sau un nod cu GPU) este planificată ca script separat, în afara aplicației, și permite plasarea acelorași combinații $\mathit{model} \times \mathit{limbaj} \times t_{\max}$ pe scări de hardware diferite.

8.5 Scalabilitate și complexitatea modelului

În modelele compartimentale agregate, mărimea populației N nu crește direct dimensiunea sistemului diferențial. O simulare SIDARTHE pentru $N = 10^3$ și una pentru $N = 10^7$ au același număr de ecuații; se schimbă doar scara valorilor. Costul crește în schimb cu numărul de compartimente, cu orizontul temporal $t_{\max}$, cu numărul de puncte salvate și cu numărul de fluxuri calculate în funcția RHS.

SIDARTHE este mai costisitor decât SIR/SIS/SEIR/SIRS nu pentru că populația este mai mare, ci pentru că fiecare evaluare RHS calculează mai multe tranziții clinice și fiecare punct salvat conține mai multe valori. În plus, persistarea în JSONB introduce un cost de serializare care nu exista într-o schemă rigidă dedicată unui singur model, dar acest cost cumpără flexibilitate: aceeași tabelă poate stoca modele cu structuri diferite de stare.

8.6 Compararea implementărilor R, Julia și Python

Cele trei implementări nu folosesc exact același algoritm numeric. Runner-ul R folosește RK4 cu pas fix $h = 1$, Python folosește `solve_ivp` cu RK45, iar Julia folosește `Tsit5`. Comparația trebuie interpretată ca o comparație între implementări practice echivalente funcțional, nu ca un experiment izolat pe același solver. Pentru o comparație strictă ar fi necesară implementarea manuală a aceleiași metode în toate cele trei limbaje.

Avantajul abordării curente este că reflectă utilizarea naturală a fiecărui ecosistem: R prin `deSolve`, Python prin SciPy, Julia prin SciML. Rezultatele benchmark-ului trebuie deci citite ca o comparație între ecosisteme de calcul științific integrate într-un sistem software, nu doar între limbaje de programare.

8.7 Indicatori agregați

Pentru fiecare combinație model–limbaj se raportează mediana timpului wall-clock:

$$\widetilde{T}_{m,l} = \text{median}\left(T_{m,l}^{(1)}, T_{m,l}^{(2)}, \ldots, T_{m,l}^{(r)}\right), \tag{8.1}$$

unde r este numărul de repetări. Variabilitatea este exprimată prin intervalul intercuartilic:

$$\text{IQR}_{m,l} = Q_{3,m,l} - Q_{1,m,l}. \tag{8.2}$$

Pentru comparații între limbaje se poate raporta raportul de viteză față de Python:

$$\text{speedup}_l = \frac{\widetilde{T}_{m,\text{Python}}}{\widetilde{T}_{m,l}}. \tag{8.3}$$

Această valoare trebuie interpretată per model și per orizont temporal. Un limbaj poate fi mai rapid pentru SIDARTHE pe 365 de zile și mai lent pentru SIR pe 30 de zile dacă overheadul runtime-ului domină calculul numeric.

8.7.1 Rezultate măsurate

A fost executată o campanie completă prin scriptul `run_benchmark.py`, conform protocolului din secțiunea 8.4: cinci modele compartimentale, trei limbaje, patru orizonturi temporale ($t_{\max} \in \{90, 365, 2\,000, 10\,000\}$), 3 repetări per combinație, pe setul de date România OWID. Toate cele 180 de rulări s-au finalizat fără erori. Datele sunt extrase direct din tabela `benchmarks`

a aplicației deployate, valorile reflectând timpul măsurat de către runner între începutul integrării ODE și finalul scrierii rezultatelor.

Tabela 8.4: Wall-time median al solverului (ms), pe model și limbaj, pentru patru orizonturi $t_{\max}$. Rezultate din campania de benchmark prin aplicație (cluster `kube-4kdhb`, 3 worker x 4 vCPU / 8 GiB), 3 repetări per combinație, dataset România OWID, prima înregistrare ca condiție inițială.

Model	**Limbaj**	**Wall-time median (ms)**			
		$t_{\max} = 90$	$t_{\max} = 365$	$t_{\max} = 2000$	$t_{\max} = 10000$
`sis`	`r`	12.0	41.0	149.0	738.0
`sis`	`julia`	0.0	0.0	1.0	5.0
`sis`	`python`	4.0	5.0	4.0	6.0
`sir`	`r`	12.0	35.0	146.0	807.0
`sir`	`julia`	0.0	1.0	1.0	5.0
`sir`	`python`	4.0	4.0	5.0	5.0
`seir`	`r`	19.0	35.0	161.0	673.0
`seir`	`julia`	1.0	0.0	1.0	5.0
`seir`	`python`	4.0	3.0	5.0	9.0
`sirs`	`r`	10.0	40.0	154.0	665.0
`sirs`	`julia`	0.0	1.0	1.0	5.0
`sirs`	`python`	5.0	4.0	5.0	5.0
`sidarthe`	`r`	16.0	62.0	274.0	1327.0
`sidarthe`	`julia`	2.0	1.0	2.0	6.0
`sidarthe`	`python`	5.0	5.0	5.0	8.0

Vizual, scalarea wall-time-ului în funcție de $t_{\max}$ (pentru modelul cel mai complex, SIDARTHE) este prezentată în Figura 8.1; cele trei limbaje produc trei regimuri calitativ diferite, identificabile dintr-o privire pe o scară log-log.

Din date reies trei tipare dominante.

Julia este cel mai rapid solver, dar are un cost de pornire JIT semnificativ. La $t_{\max} = 10\,000$, mediana wall-time-ului Julia este de 5–6 ms pentru orice model, inclusiv SIDARTHE cu opt compartimente. Python rămâne în intervalul 5–9 ms, iar R urcă de la 665 la 1 327 ms, de două până la trei ordine de mărime peste celelalte. Pentru orizonturi mici ($t_{\max} = 90$), prima rulare Julia plătește însă costul compilării *just-in-time*, vizibil în diferența dintre repetarea 1 și repetările 2–3 (Tabelul 8.7): ∼900–1 500 ms doar pentru încărcarea și compilarea funcțiilor RHS și a solverului `Tsit5`.

Python (SciPy `solve_ivp`) este cel mai stabil ca performanță: wall-time-ul rămâne aproape constant în jurul a 4–9 ms indiferent de model sau orizont, fără cold-start vizibil. Acest comportament îl face limbajul implicit recomandabil pentru utilizare interactivă, unde predictibilitatea contează la fel de mult ca viteza brută.

Costul mare al runner-ului R la orizonturi mari nu vine de la `deSolve`, ci de la inserările individuale în baza de date. Cele 10 001 inserții rând-cu-rând prin `dbExecute()` domină cele ∼1 secundă măsurată la `sidarthe`/`r`/$t_{\max} = 10\,000$; integrarea numerică propriu-zisă cu `deSolve::ode` este sub-secundară. Avem astfel un punct concret de îmbunătățire ulterioară (de exemplu prin `COPY FROM` sau prin tranzacții batch), nu o limitare intrinsecă a ecosistemului R.

În ce privește memoria reziduală peak (Tabelul 8.6), Julia plătește un overhead constant de ∼940 MB pentru runtime-ul SciML, în timp ce Python rămâne sub 100 MB iar R sub 135 MB.

Tabela 8.5: Raport de viteză al solverului față de Python (mediana wall-time Python / mediana limbaj). Valori > 1 înseamnă mai rapid decât Python; valori < 1 înseamnă mai lent. Pragul '$> x$' apare când mediana sub-milisecundară face raportul exact instabil.

Model	**Limbaj**	**Speedup vs Python**			
		$t_{\max} = 90$	$t_{\max} = 365$	$t_{\max} = 2000$	$t_{\max} = 10000$
`sis`	`r`	0.33	0.12	0.03	0.01
`sis`	`julia`	> 8	> 10	4.00	1.20
`sir`	`r`	0.33	0.11	0.03	0.01
`sir`	`julia`	> 8	4.00	5.00	1.00
`seir`	`r`	0.21	0.09	0.03	0.01
`seir`	`julia`	4.00	> 6	5.00	1.80
`sirs`	`r`	0.50	0.10	0.03	0.01
`sirs`	`julia`	> 10	4.00	5.00	1.00
`sidarthe`	`r`	0.31	0.08	0.02	0.01
`sidarthe`	`julia`	2.50	5.00	2.50	1.33

Tabela 8.6: Memorie reziduală peak (MB), mediana pe toate combinațiile model x $t_{\max}$.

Limbaj	**Runtime**	**Memorie (MB)**
`r`	deSolve + RPostgres	129
`julia`	DifferentialEquations.jl (JIT)	940
`python`	NumPy + SciPy	95

Tabela 8.7: Costul de pornire JIT pentru runner-ul Julia: prima rulare include compilarea, repetările ulterioare rulează din cod cache-uit. Valori la $t_{\max} = 90$.

Model	**Rep 1 (ms)**	**Mediana rep 2-3 (ms)**	**Cost JIT (ms)**
`sis`	1435	0	1435
`sir`	891	0	891
`seir`	887	0	886
`sirs`	948	0	948
`sidarthe`	2	1	1

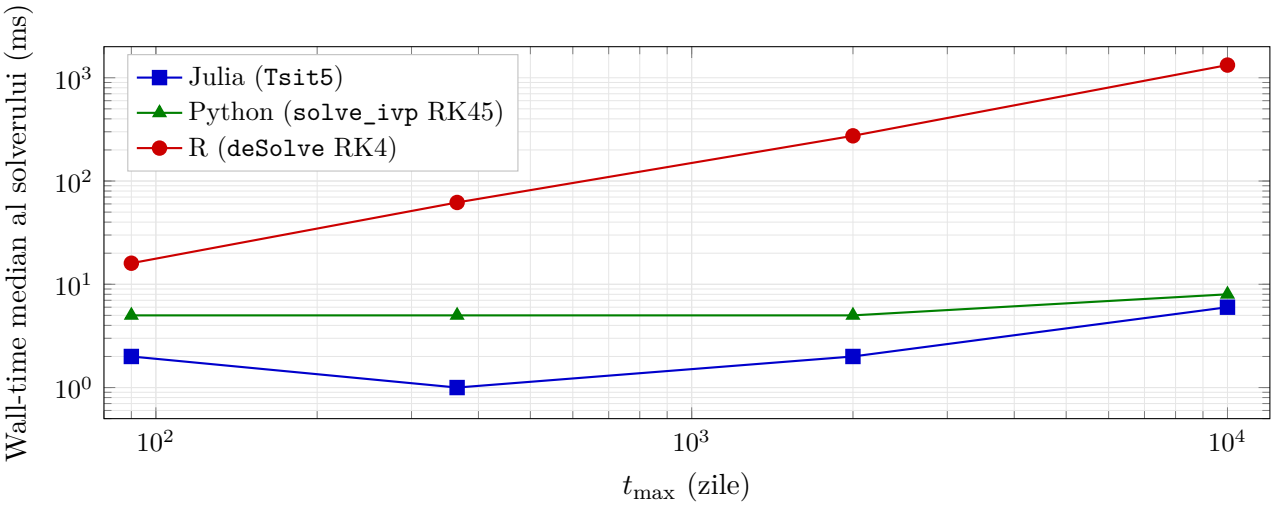


Figura 8.1: Scalarea wall-time-ului median al solverului SIDARTHE în funcție de orizontul $t_{\max}$, pentru cele trei limbaje. R prezintă o creștere aproape liniara cu $t_{\max}$ (∼130× între 90 și 10 000 de pași), dominată de inserările rând-cu-rând în baza de date prin `dbExecute()`; integrarea propriu-zisă cu `deSolve::ode` este sub-secundara. Python rămâne stabil (4–8 ms), iar Julia este mai rapid în regimul amortizat după compilarea JIT, atingând 6 ms la 10 000 de pași.

Acest cost este independent de complexitatea modelului și de orizont, deci nu se amortizează la rulări scurte; pentru job-uri foarte multe și mici, Python și R sunt opțiunile mai economicoase.

8.7.2 Costul componentei LSTM

Runner-ul LSTM are un profil de cost diferit calitativ de runnerele compartimentale. Costul ODE depinde de orizontul de integrare $t_{\max}$ (număr de pași și de puncte salvate), în timp ce costul LSTM este dominat de *antrenare* — numărul de epoci × numărul de eșantioane glisante — iar $t_{\max}$ controlează doar orizontul prognozei autoregresive (neglijabil în comparație).

Tabelul 8.8 arată rezultatele pe setul România OWID (1 440 zile, 80% antrenare / 20% test cronologic) cu parametrii impliciti din API. Mediana wall-time-ului este ∼290 s, aproape identică între $t_{\max} = 30$ și $t_{\max} = 90$, ceea ce confirmă empiric că forecast-ul nu contribuie semnificativ. Utilizarea CPU este peste 98% din wall-time (LSTM hand-rolled în NumPy fără paralelism), iar memoria peak este de doar ∼56 MB, mai mică decât a runnerelor ODE în Python — nu se construiesc matrice de stare comparabile ca dimensiune.

Tabela 8.8: Costul rulării pentru runner-ul LSTM dedicat (Python, hand-rolled NumPy), parametri impliciți (`lookback=14`, `hidden_size=12`, `epochs=150`, `learning_rate=0.01`), dataset România OWID (1 440 zile, 80% antrenare / 20% test cronologic). Forecast-ul autoregresiv pe orizontul $t_{\max}$ are cost neglijabil față de antrenare.

$t_{\max}$	n	Wall median (s)	CPU median (s)	Memorie (MB)
30	2	296.8	295.3	55.6
90	2	286.2	285.0	56.3

Pentru contextul tezei, această valoare absolută este mai puțin importantă decât observația de scalare: pe seriile sintetice de 20 000 de zile, antrenarea ar crește la ∼70 minute cu aceiași

parametri, ceea ce plasează LSTM ca operație asincronă lungă — exact rolul pentru care `runner-lstm` este izolat într-o coadă Redis dedicată.

8.7.3 Variabilitatea cross-hardware

Toate rezultatele de mai sus provin de pe un singur profil hardware (clusterul `kube-4kdhb`). Pentru a separa contribuția hardware-ului de cea a stack-ului software, aceeași logică de solver a fost rulată ca script standalone (`scripts/bench_local_solver.py`) pe o mașină locală complet diferită — Apple M1 Pro arm64, nativ, fără container și fără persistare în baza de date — folosind exact aceleași tolerante numerice (`RK45`, rtol $= 10^{-8}$, atol $= 10^{-10}$), aceiași parametri și aceleași condiții inițiale ca rulările prin aplicație. Așa cum este indicat explicit în secțiunea 8.4, această masurare se face în afara aplicației pentru a evita amestecarea variațiilor de hardware cu deciziile de ortografie ale containerului și ale bazei de date distribuite.

Tabelul 8.9 compară median wall-time pentru SIR și SIDARTHE la trei orizonturi temporale, pe workload-ul sintetic non-trivial (setul `synthetic_multiwave`, $I_0 = 100$ pentru a forța solverul să parcurgă efectiv pași de integrare). Datele de cluster sunt re-rulate în mod identic prin API (între 1 și 3 repetări per combinație, în funcție de campania anterioară), iar datele M1 provin din scriptul local (3 repetări per combinație, cu o rulare de încălzire ignorată).

Tabela 8.9: Comparația timpilor de execuție pentru solverul Python (`scipy.solve_ivp` RK45) pe două profile hardware distincte: cluster `kube-4kdhb` (worker amd64, OpenStack VM, 4 vCPU / 8 GiB, runner în container, wall-time include și scrierea rezultatelor în YugabyteDB) vs M1 Pro arm64 nativ (script standalone, fără container, fără DB). Workload non-trivial: set sintetic `multiwave` cu $I_0 = 100$, $N = 5 \cdot 10^7$. Coloana *speedup M1* este raportul medianelor; valori > 1 înseamnă M1 mai rapid.

Model	$t_{\max}$	n_{cluster}	**Cluster wall (ms)**	n_{M1}	**M1 wall (ms)**	**Speedup M1**
`sidarthe`	365	1	86.0	3	16.44	5.2×
	2000	3	127.0	3	31.46	4.0×
	10000	3	345.0	3	90.48	3.8×
`sir`	365	1	73.0	3	12.25	6.0×
	2000	3	96.0	3	16.07	6.0×
	10000	3	100.0	3	22.98	4.4×

Pentru același cod Python și aceeași încărcare numerică, M1 Pro nativ este de ∼4×–6× mai rapid decât worker-ul amd64 al clusterului. Acest raport include diferența de microarhitectură (Apple Silicon vs. CPU x86 virtualizat în KVM), diferența de mediu (nativ vs. container) și costul redus al stratului de persistare (M1 nu scrie rezultatele într-o bază de date distribuită cu factor de replicare 3). Raportul este relativ stabil între modele și orizonturi, deci marea parte a diferenței vine din execuția solver-ului propriu-zis, nu din overhead-uri fixe.

Pentru workload-ul trivial (setul România OWID cu $I_0 = 0$, ca în campania prin aplicație), M1 nativ rulează în 0,4–0,8 ms per solver, iar cluster-ul în 3–9 ms (Tabelul 8.4); diferența crește la 10× sau mai mult tocmai pentru că, atunci când solver-ul nu are dinamică reală, *toată* cheltuiala wall-time pe cluster vine din scrierea celor $t_{\max} + 1$ rânduri în `simulation_results`, cost care nu există în scriptul local.

Concluzia metodologică este că un raport *absolut* între cele două hardware-uri depinde semnificativ de natura workload-ului (solver-light vs. solver-heavy) și de stack-ul software inclus în măsurare. Recomandarea de limbaj din secțiunea 8.10 rămâne valabilă pe clusterul testat; pentru o decizie cantitativă pe un alt profil hardware — în special cu accelerare GPU pentru LSTM sau cu solvere implementate în Fortran/C++ legate static — aceeași campanie trebuie

re-rulată local, iar scriptul `bench_local_solver.py` este punctul de pornire pentru această replicare.

8.8 Acuratețe numerică și verificări de regresie

Corectitudinea rezultatelor este verificată prin conservarea populației. Pentru SIDARTHE, suma $S+I+D+A+R+T+H+E$ trebuie să rămână egală cu N la fiecare pas salvat; pentru SEIR, $S+E+I+R=N$; iar pentru modelele SIR/SIS/SIRS se verifică suma compartimentelor definite de fiecare model.

Un al doilea criteriu este apropierea traiectoriilor obținute în cele trei limbaje pentru aceeași configurație. Deoarece solverele diferă, egalitatea exactă nu este realistă. Se verifică în schimb abaterea maximă normalizată:

$$\Delta_{l_1,l_2} = \max_{t,c} \frac{|x_{l_1,c}(t) - x_{l_2,c}(t)|}{N}. \tag{8.4}$$

Dacă această abatere rămâne sub un prag ales, implementările sunt considerate compatibile funcțional pentru scopul aplicației.

Verificarea a fost rulată concret pe setul de date sintetic `multiwave` (populație $N = 50 \cdot 10^6$), pentru care condițiile inițiale produc dinamică ne-trivială (spre deosebire de România OWID, unde prima zi din serie are $I = 0$). Rezultatele sunt sintetizate în Tabelul 8.10.

Tabela 8.10: Verificare numerică de regresie cross-limbaj: abaterea maximă absolută între traiectoriile compartimentale produse în R, Julia și Python pentru aceiași parametri și aceleași condiții initiale, pe setul sintetic `multiwave` (populația $N = 50 \cdot 10^6$, $t_{\max} = 365$ pași zilnici). Modelele folosesc solvere diferite (R cu RK4 pas fix $h = 1$, Python cu RK45 adaptiv, Julia cu Tsit5), deci abaterile reflectă toleranțele numerice cumulate, nu erori de implementare.

Model	**Pereche**	**Max $\lvert\Delta\rvert$**	**Normalizat ($/N$)**
`sir`	`julia` vs `python`	0.5	$1{,}01 \cdot 10^{-8}$
	`julia` vs `r`	2185.8	$4{,}37 \cdot 10^{-5}$
	`python` vs `r`	2186.2	$4{,}37 \cdot 10^{-5}$
`sidarthe`	`julia` vs `python`	0.2	$3{,}72 \cdot 10^{-9}$
	`julia` vs `r`	6570.9	$1{,}31 \cdot 10^{-4}$
	`python` vs `r`	6571.1	$1{,}31 \cdot 10^{-4}$

Implementările Julia și Python produc traiectorii practic identice ($\sim 10^{-8} \cdot N$, la limita preciziei flotante), pentru că ambele folosesc solvere Runge–Kutta adaptive cu toleranțe similare (`Tsit5` și respectiv `RK45`). Implementarea R, care folosește RK4 cu pas fix $h = 1$ zi, deviază de la celelalte două cu $\sim 10^{-4} \cdot N$ ($\sim$2 200 cazuri pentru SIR, $\sim$6 600 pentru SIDARTHE). Pentru o populație de 50 de milioane, abaterea reprezintă 0,013% din populație, sub-vizuală pe o reprezentare grafică tipică, compatibilă funcțional cu scopul aplicației, dar deliberat mai puțin precisă decât metodele adaptive.

8.9 Evaluarea API-ului și a bazei de date

Performanța percepută de utilizator depinde și de API și baza de date. Pentru crearea unei simulări, latența API include validarea inputului, citirea primului rând epidemiologic, inserarea în `simulation_runs` și publicarea în Redis. Pentru vizualizarea rezultatului, latența include citirea rândurilor din `simulation_results`, deserializarea JSONB și construirea răspunsului.

Din acest motiv, benchmark-ul numeric a fost completat cu o campanie de teste de API, realizată printr-un script reproductibil (`scripts/bench_api.py`). Pentru fiecare model s-au măsurat cele patru operații expuse de API și percepute direct de utilizator: *create* (crearea unei simulări), *status* (interogarea stării), *results* (citirea traiectoriei complete în format JSON) și *export* (exportul CSV al aceleiași traiectorii). Latențele *create* au fost colectate din 12 repetări, iar cele de citire din 30 de repetări per combinație; măsurătorile au fost făcute prin `port-forward` către serviciul `api` din clusterul `kube-4kdhb`, deci includ tot lanțul HTTP → API → YugabyteDB → HTTP, plus variabilitatea de rețea introdusă de tunel.

Tabela 8.11: Latența API pe model, la orizontul de 365 puncte (1 an, pas zilnic). Pentru fiecare endpoint se raportează mediana (p_{50}); pentru *results*, endpointul cel mai greu, se raportează și percentila 95 (p_{95}) și dimensiunea răspunsului JSON. *create* răspunde asincron (înainte de rularea solver-ului); *results* și *export* includ deserializarea din YugabyteDB. Măsurat prin port-forward la clusterul `kube-4kdhb`.

Model	**Create** (ms)	**Status** (ms)	**Results** p_{50} (ms)	**Results** p_{95} (ms)	**JSON** (KB)	**Export** (ms)
`sir`	46.4	23.9	48.5	139.8	38.1	48.3
`sis`	70.6	31.5	45.5	123.9	36.0	39.6
`seir`	49.9	27.0	53.8	155.6	40.3	36.6
`sirs`	50.0	28.3	46.9	124.0	38.1	37.3
`sidarthe`	44.2	29.7	47.1	126.3	48.9	39.3

Tabelul 8.11 prezintă rezultatele la orizontul de un an (365 puncte). Latențele mediane sunt omogene între modele și se situează în intervalul 24–71 ms pentru toate endpointurile; modelul nu influențează semnificativ latența, deoarece costul este dominat de drumul dus-întors prin baza de date, nu de structura compartimentală. Endpointul *create* răspunde asincron: doar validează inputul, citește primul rând epidemiologic, inserează în `simulation_runs` și publică în coada Redis, înainte ca solverul să ruleze, deci latența sa nu reflectă costul de integrare numerică, ci doar pe cel de inițiere a unui job. Dimensiunea răspunsului JSON depinde de model: SIDARTHE, cu opt compartimente, produce ∼49 KB față de ∼36 KB pentru SIS cu două compartimente, la același orizont. Percentila 95 este de 2,5–3 ori mai mare decât mediana (124–156 ms), reflectând cozile de latență tipice unei măsurători prin tunel; pe un acces direct în cluster acestea ar fi mai mici, dar raportarea lor este corectă pentru profilul de testare folosit.

Tabelul 8.12 izolează efectul orizontului pentru modelul cel mai greu (SIDARTHE), variind numărul de puncte de la 30 la 365. Dimensiunea răspunsului JSON crește aproape liniar cu numărul de puncte (de la ∼4 KB la ∼49 KB), în timp ce latența mediană *results* crește mult mai lent (de la ∼30 la ∼47 ms): la aceste dimensiuni, costul fix al interogării domină costul de serializare. Exportul CSV produce constant răspunsuri de 5–10 ori mai mici decât JSON-ul echivalent (text plat, fără chei repetate per rând), confirmând că formatul de transport contează pentru volumul transferat, chiar dacă latența rămâne comparabilă.

Concluzia practică este că, pentru orizonturile uzuale (≤ 365 puncte) și modelele studiate, limita sistemului nu este nici solverul (sub 10 ms pe cluster, conform Tabelului 8.4), nici transportul JSON, ci costul fix al drumului prin baza de date, de ordinul a câtorva zeci de

Tabela 8.12: Scalarea latenței de citire cu orizontul, pentru modelul `sidarthe` (8 compartimente). Pe măsură ce numărul de puncte crește, latența *results* și dimensiunea JSON cresc aproximativ liniar, în timp ce *export* CSV rămâne mai ieftin (text plat, fără structură JSON).

Puncte	**Results** p_{50} (ms)	**Results** p_{95} (ms)	**JSON** (KB)	**Export** (ms)
30	30.5	135.6	4.1	31.1
90	34.3	129.7	12.1	35.5
180	40.0	113.4	24.1	34.7
365	47.1	126.3	48.9	39.3

milisecunde per cerere. Aceasta este o latență confortabilă pentru o interfață interactivă și indică faptul că optimizările viitoare ar trebui să vizeze reducerea numărului de cereri (de exemplu prin agregarea rezultatelor mai multor rulări într-un singur răspuns), nu accelerarea unei singure cereri.

8.10 Interpretarea rezultatelor

Evaluarea corectă trebuie să combine trei axe: eroarea numerică, timpul de execuție și complexitatea modelului. Un model nu este automat superior doar pentru că are eroare mai mică. Dacă obține această eroare cu un număr mare de parametri și cu timp de calibrare ridicat, poate fi mai puțin potrivit pentru analize rapide. În sens invers, un model simplu și rapid poate fi foarte util pentru explorare, dar insuficient pentru întrebări clinice.

Recomandarea practică derivată din această campanie urmează profilul cazului de utilizare. Pentru rulări numeroase și scurte (de exemplu explorarea sensibilității parametrilor), Python este alegerea implicită cea mai bună: timpii sunt sub 10 ms și predictibili pentru toate modelele și orizonturile testate, fără cost de pornire. Pentru rulări puține și foarte lungi (de exemplu integrare pe orizonturi $t_{\max} \gg 10\,000$ sau calibrare cu mii de iterații în care Julia se compilează o singură dată), Julia oferă un solver cu un ordin de mărime mai rapid, cu condiția ca cele $\sim$1 secundă de pornire să fie amortizate. R rămâne potrivit pentru explorare interactivă în pipeline-uri statistice familiare cercetătorilor din epidemiologie, însă runner-ul curent are o limitare operațională (inserțiile rând-cu-rând în baza de date) care domină wall-time-ul la orizonturi mari și poate fi corectată independent de schimbarea limbajului.

Pentru predicție cu LSTM, costul este dominat de antrenare și nu se compară direct cu solverele ODE. Este important ca interfața utilizatorului să trateze LSTM ca operație asincronă explicită (lansare, polling, finalizare după minute), așa cum este deja proiectată în aplicație.

Toate aceste afirmații sunt valide pentru profilul hardware al clusterului `kube-4kdhb`. Reproducerea pe alte configurații (mașini locale, noduri mai puternice, eventual GPU pentru LSTM) este lăsată ca extensie a campaniei, prin scripturi rulate în afara aplicației.

Capitolul 9
Aplicația Web E-Medicine

9.1 Rolul aplicației

Aplicația web este interfața prin care modelul matematic și infrastructura de execuție devin utilizabile. Ea nu este un sistem clinic complet, ci un mediu de lucru pentru organizarea datelor pe studii epidemiologice, încărcarea dataseturilor, configurarea simulărilor SIDARTHE, SIR, SIS, SEIR sau SIRS și compararea rezultatelor obținute în R, Julia și Python. Din această perspectivă, aplicația are rol de instrument experimental și colaborativ.

Fluxul principal începe cu autentificarea utilizatorului. După autentificare, acesta alege un studiu sau creează unul nou. Toate operațiile ulterioare se raportează la studiul curent, iar interfața afișează acțiunile disponibile în funcție de rolul utilizatorului: `admin`, `researcher`, `contributor` sau `viewer`. Utilizatorii cu rol suficient pot încărca un fișier CSV cu date epidemiologice, pot alege modelul, pot introduce parametrii și pot urmări statusul execuției. Când runner-ul finalizează job-ul, interfața afișează traiectoriile compartimentale și permite exportul rezultatelor în format CSV. O pagină separată permite compararea mai multor rulări finalizate, iar pagina de benchmark prezintă agregările calculate la nivel de model și limbaj.

Pe lângă aplicația principală există o interfață `frontend-admin`, servită de pe host-ul `admin.sir-workbench.ro`. Aceasta este disponibilă doar conturilor cu `is_super_admin=true` și oferă statistici globale, gestionarea utilizatorilor și vizualizarea tuturor studiilor.

9.2 Backend Go

Backend-ul este implementat în Go și expune un API REST sub prefixul `/api`. Serverul folosește routerul `chi`, middleware pentru logging, recuperare după panică, timeout de request și CORS, iar accesul la rutele protejate este controlat prin JWT. Alegerea Go este potrivită pentru această componentă deoarece API-ul este stateless, are nevoie de latențe predictibile și poate fi distribuit ușor sub formă de binar static în container.

Structura codului separă intrarea HTTP de accesul la date. Pachetul `handlers` parsează cererile, validează input-ul și construiește răspunsurile JSON. Pachetul `repository` conține interogările SQL și operațiile tranzacționale peste `pgx`. Pachetul `runner` definește schema mesajului trimis în Redis și publicarea job-urilor. Contractul de simulare include tipul modelului, parametrii, condițiile inițiale și conexiunea la baza de date.

Rutele aplicației principale sunt organizate sub `/api/studies/{studyId}`. Middleware-ul RBAC verifică membership-ul și rolul înainte ca handler-ul să ajungă la date. Super-adminii au acces global, dar consola administrativă folosește rute dedicate sub `/api/admin`.

9.3 Gestionarea dataseturilor

Dataseturile sunt încărcate printr-un formular multipart, în contextul studiului curent. API-ul acceptă câmpurile `name`, `disease_code` și `file`; fișierul trebuie să conțină cel puțin coloanele `date`, `region`, `infected` și `population`. Coloanele `recovered` și `susceptible` sunt opționale,

dar atunci când sunt prezente sunt folosite pentru condițiile inițiale. Pentru SIDARTHE, cazurile active sunt plasate inițial în compartimentul I, persoanele ieșite din infecție în H, iar compartimentul E al deceselor pornește de la zero. Pentru SEIR, compartimentul expușilor pornește de asemenea de la zero, deoarece nu este observat direct în CSV.

Persistarea datasetului se face într-o tranzacție. Mai întâi este creat rândul din `datasets`, apoi înregistrările epidemiologice sunt inserate prin `CopyFrom`. Această alegere reduce costul încărcării CSV-urilor față de inserarea rând-cu-rând și păstrează atomicitatea: dacă una dintre înregistrări nu poate fi scrisă, setul de date nu rămâne într-o stare parțială. La ștergere, `admin`-ul poate elimina orice set de date din studiu, în timp ce `contributor`-ul poate elimina doar seturile de date încărcate de el.

9.4 Execuția simulărilor

Crearea unei simulări este un proces scurt în API și un proces asincron în ansamblul sistemului. Endpointul `POST /api/studies/{studyId}/simulations` verifică existența datasetului în studiul curent, modelul, limbajul selectat, parametrii și $t_{\max}$, apoi extrage prima înregistrare epidemiologică pentru a determina valorile inițiale. După ce rândul din `simulation_runs` este creat cu status `pending`, API-ul publică mesajul JSON în coada Redis corespunzătoare limbajului.

Runner-ele sunt responsabile pentru tranzițiile ulterioare. Ele marchează rularea ca `running`, execută solver-ul pentru `model_type`, inserează rezultatele și benchmark-ul, apoi marchează rularea ca `completed`. Dacă apare o eroare, statusul devine `failed`, iar mesajul este salvat în baza de date. Interfața web nu trebuie să țină o conexiune deschisă pe durata calculului; pagina de rezultat interoghează periodic endpointul de status până la finalizare.

9.5 Editor de cod pentru cercetători

Pe lângă cele șase modele built-in și runner-ele plug-in înregistrate de super-admin (secțiunea 4.7), aplicația include un *editor de cod* accesibil oricărui membru al studiului cu drept de scriere de simulări. Această componentă răspunde unei cerințe distincte: cercetătorul vrea să încerce rapid un algoritm propriu (o regresie, un filtru, o euristică de prognoză) peste dataset-ul disponibil, fără a parcurge procesul de a construi o imagine container și a o înregistra ca runner persistent.

9.5.1 Flux de lucru

Pagina `/script-editor` prezintă un layout în două coloane: în stânga, o listă verticală cu draft-urile salvate ale cercetătorului în studiul curent; în dreapta, un editor Monaco (același engine ca VS Code) preîncărcat cu un script demonstrativ de netezire exponențială și un set inițial de parametri. Sub editor se găsesc trei controale: dropdown-ul de dataset, un textarea cu parametrii în format JSON și butonul “Rulează”. La apăsarea butonului, clientul trimite o cerere `POST /simulations` cu `model_type = 'script'` și conținutul curent al editorului; aceeași pagină pollează apoi endpointul de status și, la finalizare, afișează rezultatele cu același component grafic folosit pentru toate celelalte tipuri de rulări.

Salvarea draft-urilor este o operație separată, decuplată de execuție. Cercetătorul introduce un nume în antetul workspace-ului și apasă “Salvează”; clientul face `POST /script-drafts` (sau `PATCH /script-drafts/{id}` dacă draftul are deja un id), iar în sidebar apare o intrare nouă marcată cu data ultimei actualizări. Click pe un draft din sidebar îl reîncarcă în editor împreună cu parametrii salvați, iar butonul “×” îl șterge după confirmare. Cu acest flux, codul

scris în pagină nu se pierde la reîncărcarea browser-ului și este accesibil din orice device pe care cercetătorul se autentifică.

9.5.2 Contractul scriptului

Scriptul are aceeași formă cu cea declarată pentru runner-ele plug-in: o singură funcție `main` cu o semnătură fixă.

```
def main(records, parameters, context):
    """
    records:    randurile dataset-ului, fiecare cu
                cheile: date, region, susceptible,
                infected, recovered, population
                + (opt) deaths, new_cases, new_deaths,
                hospitalized, icu, vaccinated, tested,
                stringency_index, reproduction_rate
    parameters: dict cu parametri (validati JSON
                in UI)
    context:    {simulation_id, population_n,
                t_max}
    return:     [{"t": int, "compartments":
                  {nume: float, ...}}, ...]
    """
```

Listing 9.1: Contractul scriptului din editor.

Returnarea este aceeași listă de pași temporali cu obiecte `compartments` ca pentru runner-ele built-in, ceea ce permite graficului existent să afișeze rezultatele fără tratare specială. Cheile epidemiologice extinse din `records` sunt populate atunci când sursa CSV de la upload le-a furnizat (*e.g.*, dataseturile OWID COVID au de regulă `deaths`, `stringency_index` și `reproduction_rate` pentru mare parte din 2020–2023); altfel valoarea cheii este `None`, iar scriptul trebuie să trateze explicit absența. Stack-ul Python disponibil în executor este fix (`numpy`, `scipy`, `pandas`, `scikit-learn`); cercetătorul care are nevoie de dependențe netriviale poate trece pe traseul de runner cu imagine externă.

9.5.3 Execuție și separare de runner-e persistente

Scriptul este executat de un Deployment permanent numit `runner-script`, care ascultă pe coada Redis `epimodel:jobs:script`. Pentru fiecare job, executorul scrie conținutul scriptului într-un fișier temporar, îl compilează ca modul Python, apelează `main(records, parameters, context)` cu măsurarea timpilor wall și CPU și a memoriei de vârf, validează structura rezultatului și scrie traiectoriile în `simulation_results` alături de o intrare în `benchmarks` — exact ca runner-ele built-in. Modulul este reîncărcat la fiecare job pentru a nu lăsa stare globală între execuții.

Această arhitectură separă explicit două responsabilități. Editorul de cod este pentru *explorare ad-hoc*: nu există resurse Kubernetes create per script, executorul partajat absoarbe toate cererile, iar permisiunea este deschisă oricărui cercetător cu drept de scriere de simulări. Runner-ele cu imagine container (capitolul 4, secțiunea 4.7) sunt pentru *algoritmi operaționali*: imagine construită extern, Deployment dedicat în cluster, acces de înregistrare restrâns la super-admin.

9.5.4 Izolare între utilizatori și securitate

Draft-urile sunt strict private cercetătorului care le-a creat. Tabela `script_drafts` are coloana `user_id` NOT NULL, iar toate interogările din `ScriptDraftRepository` filtrează cumulativ pe `(user_id, study_id, id)`: chiar dacă un utilizator încearcă să ceară direct id-ul unui draft creat de un alt cercetător (atac de tip IDOR), API-ul răspunde `404 not found`. Listarea

expune doar metadatele (`script_content` este omis); conținutul este servit doar la deschiderea explicită a unui draft, ceea ce păstrează compact răspunsul de listare și clarifică intenția fiecărei cereri.

Dimensiunea scriptului trimis la execuție și a draft-ului salvat este plafonată la 1 MB, suficient pentru orice script de explorare obișnuit și strict mai mic decât payload-urile pe care le-ar tolera baza de date. Validarea formatului parametrilor (JSON) și a structurii rezultatului (`list[{t, compartments}]`) se face server-side, astfel încât un script rău formatat este respins cu un mesaj clar înainte de a polua tabelele de rezultate.

Frontend-ul principal este o aplicație Vue 3 construită cu Vite și TypeScript. Rutarea este gestionată cu Vue Router, iar starea aplicației cu Pinia. Clientul HTTP este o instanță Axios configurată cu prefixul `/api`; un interceptor adaugă automat headerul `Authorization` atunci când există token în `localStorage`. Store-ul `studies.store.ts` păstrează studiul curent și rolul curent, iar store-urile pentru seturi de date, simulări și benchmark-uri construiesc URL-urile în funcție de acest `studyId`.

Paginile sunt organizate după fluxurile principale ale aplicației: `StudiesPage` selectează contextul de lucru, `CreateStudyPage` creează un studiu nou, `StudyManagementPage` gestionează membrii și rolurile, `DashboardPage` sumarizează seturile de date și simulările recente, `DatasetsPage` gestionează încărcarea CSV-urilor, `NewSimulationPage` colectează modelul și parametrii, `SimulationResultPage` afișează statusul și rezultatele, `ComparePage` suprapune seriile mai multor rulări, iar `BenchmarksPage` afișează agregările pe model și limbaj. Componenta `CompartmentChart` folosește Chart.js prin `vue-chartjs` și construiește dinamic seriile în funcție de cheile disponibile în `compartments`. Astfel, același grafic poate afișa modele cu două, trei, patru sau opt compartimente fără modificări suplimentare.

Decizia de a folosi polling la două secunde pentru actualizarea statusului unei simulări este simplă, dar adecvată pentru durata așteptată a job-urilor. O soluție bazată pe WebSocket sau Server-Sent Events ar reduce numărul de cereri, însă ar introduce stare suplimentară în backend. Pentru un API stateless și pentru job-uri care durează de la câteva secunde la câteva minute, polling-ul păstrează implementarea previzibilă.

9.6 Consola super-admin

Consola administrativă este un build Vue separat, cu propriul router și propriul token în `localStorage`. La autentificare, aplicația verifică răspunsul `/api/auth/login`; dacă utilizatorul nu este super-admin, tokenul nu este păstrat și login-ul este refuzat. Această separare reduce riscul ca acțiunile globale de administrare să fie amestecate cu fluxurile obișnuite de lucru ale unui studiu. Toate paginile sunt deliberat *de citire*: mutațiile asupra studiilor și seturilor de date rămân în aplicația principală, controlate prin RBAC pe studiu.

Pagina inițială de dashboard afișează cele opt contoare globale (`/api/admin/stats`) și șase distribuții agregate între studii, obținute din `/api/admin/breakdown`: simulări pe status, model și limbaj; seturi de date și studii pe boală; top zece studii după volumul cumulat de înregistrări. Distribuțiile sunt randate ca bare orizontale proporționale, iar top-ul de studii are link-uri direct către pagina de detaliu.

Pagina de utilizatori (`/api/admin/users`) listează toate conturile cu un buton care comută flag-ul `is_super_admin` prin `PATCH /api/admin/users/{userId}/super-admin`. Pagina de studii (`/api/admin/studies`) listează toate studiile cu un câmp de filtrare text și permite navigarea către pagina de detaliu prin click pe rând.

Pentru detaliul unui studiu, `StudyDetailPage` reutilizează endpointurile `/api/studies/{studyId}/*` (membri, seturi de date, simulări, benchmark-uri agregate), de care super-adminul beneficiază datorită bypass-ului din middleware-ul RBAC. Pagina

prezintă, în ordine, un antet cu metadatele studiului și patru contoare (membri, seturi de date cu volum cumulat, simulări, combinații model × limbaj cu rulări complete), urmate de tabelele detaliate. Simulările au badge de status colorat, iar tabelul de benchmark-uri arată wall-time median, percentila 95, CPU median și memorie mediană pentru fiecare combinație model–limbaj.

Pagina `/activity` consumă `/api/admin/recent-simulations` și `/api/admin/recent-datasets` pentru a afișa cele mai recente N intrări (configurabil între 20 și 200), indiferent de studiu, cu link-uri către studiul corespunzător. Feed-ul este util pentru audit operațional, pentru identificarea rapidă a simulărilor blocate în starea `running` și pentru verificarea încărcărilor de seturi de date proaspete.

9.7 Considerații de securitate

Fiind un sistem care gestionează date de cercetare grupate pe studii, cu acces diferențiat pe roluri, securitatea este o cerință funcțională, nu un adaos ulterior. Această secțiune documentează mecanismele implementate efectiv și limitările cunoscute, pe baza unei treceri în revistă a codului backend.

9.7.1 Mecanisme implementate

Identitatea este stabilită prin token JWT semnat cu HMAC-SHA256. Middleware-ul de validare verifică explicit că metoda de semnare a token-ului este HMAC înainte de a accepta cheia, respingând astfel atacul de confuzie de algoritm (un token forjat cu `alg=none` sau cu o cheie publică RSA reinterpretată ca secret HMAC). Expirarea (`exp`) este verificată la fiecare cerere, iar secretul JWT este obligatoriu la pornirea serviciului: în absența lui aplicația refuză să pornească, ceea ce elimină riscul unui secret implicit slab.

Parolele nu sunt niciodată stocate în clar, ci ca hash bcrypt cu factor de cost 12. La autentificare se folosește o comparație în timp constant, iar mesajul de eroare este identic pentru email inexistent și parolă greșită (*invalid credentials*), pentru a nu dezvălui ce conturi există.

Autorizarea este contextuală (RBAC pe studii), cu patru roluri: `admin`, `researcher`, `contributor`, `viewer`, plus flag-ul global `is_super_admin`. Regulile sunt aplicate ca lanț de middleware-uri pe rute: citirea este permisă oricărui membru, încărcarea de seturi de date cere rol `admin` sau `contributor`, crearea de simulări cere `admin` sau `researcher`, iar administrarea studiului (membri, ștergere) este rezervată rolului `admin`. Endpointurile de platformă (`/api/admin/*`) sunt protejate separat prin verificarea super-adminului.

Resursele imbricate sunt interogate întotdeauna în contextul studiului din URL: o cerere de tipul `/studies/{A}/simulations/{id}` verifică nu doar că utilizatorul este membru al studiului `A`, ci și că simularea `id` aparține studiului `A`. Astfel, un membru al unui studiu nu poate citi simulări sau seturi de date dintr-un alt studiu prin substituirea identificatorului (atac de tip IDOR).

Toate interogările către YugabyteDB folosesc parametri legați (`$1, $2, ...`), nu concatenare de șiruri, eliminând injecția SQL. Parametrii simulărilor sunt validați pe server: tipul modelului și limbajul sunt verificate față de liste albe, ratele negative sunt respinse, iar hiperparametrii LSTM sunt restrânși la intervale întregi valide. Încărcările de fișiere sunt plafonate la 32 MB prin `MaxBytesReader`, prevenind epuizarea memoriei la parsarea CSV.

9.7.2 Limitări cunoscute și remedieri

Trecerea în revistă a identificat și două probleme punctuale, remediate în cursul acestei lucrări. Prima: endpointul de creare a simulărilor accepta orice $t_{\max} > 0$, fără plafon superior, ceea ce permitea unei cereri legitime să declanșeze o integrare arbitrar de lungă și scrieri masive în baza de date — o cale de epuizare a resurselor. S-a introdus un plafon explicit ($t_{\max} \leq 100\,000$), care acoperă cu marjă orizonturile folosite în campania de benchmark. A doua: la autentificare, un email inexistent returna răspunsul înainte de verificarea bcrypt, astfel încât diferența de latență putea fi folosită pentru a enumera conturile valide (timing oracle). S-a adăugat o comparație bcrypt cu un hash fals pe ramura "email inexistent", egalizând timpul de răspuns.

Rămân limitări tratate ca direcții ulterioare. Nu există încă limitare a ratei cererilor pe endpointul de autentificare, ceea ce lasă deschisă posibilitatea atacurilor de tip brute-force sau credential stuffing; soluția naturală este un middleware de rate limiting per IP. Token-urile JWT sunt fără stare, deci nu pot fi revocate înainte de expirare (24 h); pentru un control mai fin ar fi necesară o listă de revocare sau token-uri de reîmprospătare cu durată scurtă. În fine, politica CORS este permisivă (`AllowedOrigins: *`), ceea ce este acceptabil în configurația curentă — autentificarea se face prin antet `Authorization` cu token Bearer, nu prin cookie, iar `AllowCredentials` este dezactivat, deci nu există suprafață de atac CSRF — dar ar trebui restrânsă la originile cunoscute într-un deployment public.

9.8 Validare funcțională

Validarea funcțională a aplicației poate fi făcută prin fluxuri end-to-end care urmează operațiile reale ale utilizatorului: înregistrare, autentificare, creare sau selectare studiu, adăugare membru, încărcare set de date, pornire simulare SIDARTHE, așteptare finalizare, afișare grafic și export CSV. Aceeași suită ar trebui să verifice și rulările SIR/SIS/SEIR/SIRS, deoarece acestea asigură compatibilitatea cu familia de modele discutată în capitolul teoretic și cu comparațiile simple. Un flux separat validează consola admin: login cu un cont super-admin, listarea utilizatorilor, modificarea flag-ului `is_super_admin` și listarea studiilor globale.

Capitolul 10
Concluzii și Direcții de Dezvoltare Viitoare

10.1 Sinteza lucrării

Lucrarea a fost reorientată în jurul modelului SIDARTHE, deoarece acesta descrie mai bine situațiile în care diagnosticul, simptomatologia, severitatea clinică și mortalitatea influențează evoluția observată a unei epidemii. Modelele SIR/SIS/SEIR/SIRS rămân prezente, dar rolul lor este secundar: oferă repere matematice și numerice simple, utile pentru comparații și verificări de regresie.

Pe această bază a fost construit un sistem software în care datele, simulările și metricile de performanță sunt tratate unitar. API-ul Go oferă autentificare, organizare pe studii, control RBAC, încărcare de seturi de date și pornire de simulări; Redis decuplează cererile utilizatorului de execuția numerică; runner-ele R, Julia și Python rezolvă modelul cerut prin `model_type`; cele două interfețe Vue.js separă fluxurile de lucru ale cercetătorilor de operațiile globale ale super-adminilor. Contribuția practică a sistemului constă în această legătură între model, execuție, izolare organizațională și observabilitatea rezultatelor.

Lucrarea a fost extinsă cu un cadru de comparație numerică și calibrare. Această extensie nu este doar metodologică: a fost executată concret o campanie de benchmark prin aplicație (180 rulări ODE plus 4 rulări LSTM, fără eșecuri), iar calibrarea SIR cu $\gamma = 1/14$ a fost aplicată pe România și pe trei țări europene suplimentare (Italia, Germania, Spania), rezultatele fiind raportate ca tabele și figuri în capitolele 7 și 8. În plus, a fost introdusă o direcție predictivă bazată pe LSTM, tratată ca extensie statistică pentru prognoză pe termen scurt și pentru analiza reziduurilor rămase după simularea compartimentală, cu rezultate operaționale pe seria România OWID.

10.1.1 Răspuns la obiectivele propuse

Cele patru obiective formulate în secțiunea 1.3 au fost atinse concret, după cum urmează.

Pentru obiectivul principal (proiectarea unui sistem care leagă modelarea matematică de implementarea software, cu SIDARTHE ca model de lucru) a fost construit întregul lanț, de la formalismul cu opt compartimente și șaisprezece rate (capitolul 2) până la implementarea în trei limbaje (capitolul 5), expus printr-un API Go cu organizare pe studii și control RBAC (capitolul 4), rulat pe un cluster Kubernetes real (`kube-4kdhb`). SIDARTHE este tratat ca model clinic principal pe tot parcursul lucrării, iar familia SIR/SIS/SEIR/SIRS oferă reperele de bază.

Pentru al doilea obiectiv (același contract de simulare în R, Julia și Python), cele trei runner-e au fost implementate cu bibliotecile uzuale ale fiecărui ecosistem (`deSolve`, `DifferentialEquations.jl`, `scipy.integrate`) și folosesc același contract de mesaj prin Redis și aceeași schemă de rezultate în YugabyteDB. Verificarea de regresie cross-limbaj (Tabelul 8.10) arată că abaterile între traiectoriile produse sunt sub $10^{-4} \cdot N$ pentru toate modelele, iar Tabelul 8.4 compară direct costurile de execuție.

Al treilea obiectiv (metodologie explicită de comparație și calibrare) se sprijină pe operatori de observație definiți per model (secțiunea 7.3), eroare MAE/RMSE pe ferestre separate de

antrenare și test, iar calibrarea a fost rulată efectiv: pe România cu analiză de identifiabilitate (Figura 7.2) și pe patru țări simultan (Tabelul 7.4). Costul software al calibrării a fost măsurat și raportat alături de eroarea numerică (Tabelul 7.5), conform principiului enunțat în obiectiv.

Obiectivul suplimentar (extensie predictivă bazată pe LSTM) este acoperit de un runner Python dedicat (`runner-lstm`), antrenat hand-rolled în NumPy, cu rezultate operaționale pe seria România (Figura 6.1). Validarea față de predictori naivi (Tabelul 6.2) este onestă: LSTM-ul univariat nu bate persistența la orizontul de o zi pe această serie, ceea ce delimitează regimul în care componenta are sens (corecție de reziduuri sau intrări multivariate), așa cum era anticipat în obiectiv.

10.2 Concluzii privind modelele

SIDARTHE introduce o structură mai apropiată de realitatea raportării epidemiologice decât SIR. Separarea cazurilor nedepistate de cele diagnosticate, a formelor ușoare de cele severe și a deceselor permite formularea unor întrebări care nu pot fi abordate natural într-un model cu trei compartimente. În același timp, această granularitate crește numărul de parametri și face calibrarea mai dificilă.

Modelele SIR, SIS, SEIR și SIRS rămân utile tocmai prin simplitate. Ele permit verificarea conservării populației, a pozitivității soluției și a comportamentului solverelor într-un cadru ușor de controlat. În sistemul implementat, aceste modele sunt tratate ca repere secundare, nu ca explicații complete ale datelor reale.

Comparația numerică trebuie interpretată împreună cu structura modelului. Un RMSE mai mic nu este suficient pentru a declara un model superior dacă acesta necesită mai mulți parametri neobservabili sau dacă supraînvață fereastra de calibrare. Pentru SIDARTHE, valoarea reală este dată de capacitatea de a formula întrebări clinice mai bogate, nu doar de reducerea unei erori agregate.

10.3 Concluzii privind implementarea

Implementarea arată că același contract de simulare poate fi rulat în ecosisteme diferite fără a schimba API-ul sau interfața web. R oferă o formulare compactă prin `deSolve`; Python oferă integrare directă cu infrastructura și bibliotecile științifice uzuale; Julia este potrivită pentru experimente numerice mai ambițioase, mai ales atunci când costul de compilare este amortizat.

Schimbarea schemei rezultatelor din coloane fixe `s/i/r` într-un obiect JSONB `compartments` este una dintre modificările importante ale pivotării. Ea face sistemul mai flexibil și permite adăugarea unor modele noi fără migrații ale tabelei de rezultate. Costul acestei flexibilități este o serializare mai generală și o nevoie mai mare de validare la nivel de aplicație.

În aceeași direcție, schema datelor de intrare a fost îmbogățită dincolo de nucleul SIR-centric. Tabela `epidemiological_records` găzduiește acum nouă metrici epidemiologice suplimentare (decese cumulate, cazuri și decese noi, spitalizări, terapie intensivă, vaccinări, teste, indicele Oxford de stringență, R_t observat), populate din sursa OWID atunci când există; Section 5.1 detaliază propagarea, iar Section 7.7.1 explică levierul deschis: o calibrare bivariată care fitează simultan I și D rupe valea de identifiabilitate slabă vizibilă în peisajul de pierdere MSE(β, γ). Cuplată cu editorul de scripturi Python expus în interfață (Section 9.5), această îmbogățire face ca extinderea cu algoritmi noi de analiză să nu mai necesite migrații de bază de date sau modificări în API; cercetătorul accesează direct seriile prin `records[i]` și formulează modele ad-hoc.

Pe latura operațională, sistemul a fost deployat efectiv pe un cluster Kubernetes v1.31 găzduit pe o platformă cloud privată, cu trei noduri control plane și trei workeri, YugabyteDB la

factor de replicare 3 și Helm pentru ciclul de viață al bazei de date. Această experiență confirmă portabilitatea arhitecturii și pune în evidență câteva ajustări specifice mediilor distribuite, precum aplicarea migrațiilor prin `psql` în locul `golang-migrate` sau adăugarea extensiei `pgcrypto` la momentul instalării, datorită bazei PostgreSQL 11 a YugabyteDB. Aceste ajustări sunt locale și nu afectează nici codul aplicativ, nici interfața REST.

10.4 Concluzii experimentale

Cele patru zone empirice ale lucrării (calibrarea pe date reale, campania de benchmark prin aplicație, verificarea de regresie cross-limbaj și măsurătorile cross-hardware) produc rezultate care nu trebuie citite în izolare, ci ca un set coerent de constrângeri pentru deciziile de modelare și de implementare.

Calibrarea susține alegerea constrângerilor din literatură. Comparația dintre SIR liber și SIR constrâns (cu $\gamma = 1/14$) pe valul 1 COVID-19 în România (subsecțiunea 7.6.1) arată că erorile pe fereastra de antrenare sunt aproape identice ($\sim$1 600 cazuri MAE pentru ambele scenarii), dar parametrii estimați diferă substanțial ($\hat{\gamma}$ liber = 0,335, adică 3 zile de infecțiozitate, contra valorii clinice de 14 zile). Peisajul de pierdere (subsecțiunea 7.7.1) confirmă vizual cauza: valea de pierdere mică este alungită în (β, γ), deci datele nu disting între aceste combinații. Extinderea la patru țări (subsecțiunea 7.7.2) arată că procedura constrânsa este robustă cross-regiune: $\hat{\mathcal{R}}_0$ rămâne între 0,94 (Italia) și 1,45 (România), valori sub intervalul de literatură 2–3 pentru tulpina originală, ceea ce este consistent cu intervențiile non-farmaceutice deja active în fereastră. În toate cazurile, divergența pe fereastra de test este atribuibilă unei schimbări de regim epidemic (val 2), nu unei deficiențe a calibrării: este o limitare structurală a SIR cu coeficienți constanți.

Benchmark-ul relevă trei regimuri de cost calitativ diferite. Pe clusterul `kube-4kdhb` (capitolul 8), Julia este solver-ul cel mai rapid în regim amortizat (5–6 ms la $t_{\max} = 10\,000$, indiferent de model), dar plătește un cost de pornire JIT de $\sim$900–1 500 ms pe prima rulare. Python (SciPy) este cel mai stabil ca performanță (4–9 ms peste tot, fără cold-start), iar R (`deSolve`) este de două până la trei ordine de mărime mai lent la orizonturi mari, nu din cauza solver-ului în sine, ci a buclei de inserări rând-cu-rând în baza de date. Distincția contează direct la nivel de decizii practice: pentru calibrare, care face zeci sau sute de evaluări succesive ale solver-ului, Julia este alegerea naturală deoarece costul JIT se amortizează; pentru interfața interactivă a aplicației, unde predictibilitatea contează mai mult decât viteza brută, Python rămâne default-ul rezonabil.

Implementările sunt compatibile numeric, dar nu identice. Verificarea cross-limbaj (secțiunea 8.8) arată că Julia și Python produc traiectorii identice la nivelul preciziei flotante ($\sim 10^{-8} \cdot N$), deoarece ambele folosesc solvere Runge–Kutta adaptive cu toleranțe similare (`Tsit5` și respectiv `RK45`). Implementarea R, care folosește RK4 cu pas fix $h = 1$ zi, deviază de la celelalte două cu $\sim 10^{-4} \cdot N$ (0,013% din populație), diferență deliberată din alegerea solver-ului, nu eroare de implementare. Rezultatul dă o limită cantitativă a interschimbabilității runnerelor: pentru analize cu cerințe de precizie sub $10^{-3} \cdot N$, R este acceptabil; pentru reproducerea exactă a unei traiectorii Julia, Python este interschimbabilul direct.

Hardware-ul contează, dar nu înlocuiește analiza per stack. Comparația cross-hardware (subsecțiunea 8.7.3) arată că același cod Python rulează de 4 până la 6 ori mai rapid pe Apple M1 Pro arm64 nativ decât pe un worker amd64 din cluster, pe workload-uri solver-heavy. Pentru workload-uri solver-light (cu condiție inițială trivială), raportul crește la 10× sau mai mult, dar diferența nu mai reflectă hardware-ul brut, ci costul fix al persistării în baza de date distribuită. Operațional, etapele de explorare rapidă (calibrări experimentale, sensitivity sweeps, prototipare LSTM cu hiperparametri variați) sunt sensibil mai eficiente local, în timp ce clusterul aduce concurența, izolarea pe studii și persistența necesare pentru o utilizare colaborativă reală.

LSTM este o componentă calitativ diferită de solverele ODE. Cele patru rulări LSTM (subsecțiunea 8.7.2) au costat ∼290 secunde fiecare pe seria România OWID (1 440 zile, 80%/20% split), de aproximativ *cinci ordine de mărime* peste solverele ODE pe același cluster. Decalajul confirmă decizia arhitecturală de a izola `runner-lstm` într-o coadă Redis dedicată, separată de runnerele compartimentale; figura observat/fitted/predicted pentru România (Figura 6.1) confirmă că predicțiile o-zi-înainte urmăresc rezonabil seria, dar prognoza autoregresivă se aplatizează rapid: LSTM-ul actual este un instrument de prognoză pe termen scurt și de detectare a reziduurilor, nu un model autonom de extrapolare epidemiologică pe orizonturi mari. Validarea față de repere triviale (Tabelul 6.2) întărește concluzia cu cifre: pe fereastra de test din seria României, persistența rămâne superioară LSTM-ului la orizontul de o zi, deoarece 85% din zile sunt plate (raportare săptămânală). Componenta are valoare adăugată plauzibilă doar în regimul hibrid de corecție a reziduurilor sau cu intrări multivariate, nu ca predictor univariat de sine stătător.

10.5 Limitări

Prima limitare este calibrarea parametrilor SIDARTHE. Datele OWID folosite în lucrare nu oferă direct distribuția pe compartimentele D, A, R, T și E. Inițializarea folosită de sistem este reproductibilă, dar nu înlocuiește o estimare epidemiologică dedicată. Calibrarea SIR a fost realizată concret pe patru țări europene cu rezultate interpretabile; pentru SIDARTHE, însă, este necesară integrarea unor serii suplimentare (testare, spitalizare, terapie intensivă, vindecări) și a unor operatori de observație multipli, care să constrângă subgrupuri de parametri separat.

Calibrarea este realizată în lucrare ca script offline, în afara aplicației (`scripts/calibrate.py` și `scripts/calibrate_multiregion.py`). Promovarea acesteia la un tip de job în Redis, alături de `simulation_runs`, ar permite expunerea în interfața web și stocarea istoricului parametrilor estimați — o extensie de sistem, nu o re-derivare metodologică.

A doua limitare privește acoperirea evaluării empirice. Campania de benchmark prin aplicație a fost executată pe un singur profil hardware (clusterul `kube-4kdhb`, 3 worker-i × 4 vCPU / 8 GiB pe OpenStack); reproducerea aceleiași matrici pe alte configurații (mașini locale, noduri mai puternice, eventual cu accelerare GPU pentru LSTM) este planificată ca script separat, în afara aplicației, după cum este menționat în secțiunea 8.4. De asemenea, evaluarea concurenței (mai multe job-uri lansate simultan pe coadă) și măsurarea latențelor specifice ale API-ului sunt încă tratate metodologic, fără rezultate empirice.

Componenta LSTM este implementată operațional și are rezultate cantitative (cost de antrenare măsurat și serie observed/fitted/predicted pe România). Validarea predictivă completă — comparația sistematică cu liniile de bază (persistență, medie mobilă) și cu modelul compartimental calibrat, pe mai multe regiuni — este următorul pas natural; LSTM-ul actual este o componentă univariată hand-rolled, iar versiunile hibride (LSTM peste reziduurile unui model SIR calibrat) sunt formulate în capitolul 6 doar ca direcție.

O altă limitare este nivelul de maturitate al aplicației web. Fluxurile principale sunt acoperite, inclusiv studiile, rolurile și consola super-admin, însă lipsesc încă teste automate end-to-end, mecanisme de retry mai sofisticate pentru job-uri, politici de rate limiting și audit al operațiilor utilizatorilor. Aceste elemente ar fi necesare înainte de folosirea sistemului într-un context operațional.

10.6 Direcții viitoare

Direcția imediată este calibrarea modelului SIDARTHE pe date mai bogate decât seria de cazuri active, în special date despre testare, spitalizare, terapie intensivă, vindecări și decese. Această

extindere preia infrastructura demonstrată deja pentru SIR (calibrare cross-regiune cu γ fixat din literatură) și o extinde la operatori de observație multipli, necesari pentru distingerea grupurilor de parametri ale SIDARTHE. Pasul tehnic este acum direct: tabela `epidemiological_records` expune toate seriile necesare (`deaths`, `hospitalized`, `icu`, `tested`, `vaccinated`); rămâne de formulat funcția obiectiv bivariată sau trivariată (cu ponderi $w_I, w_D, w_T, \ldots$ stabilite din scările seriilor) și de extins scriptul `calibrate.py` pentru a fita simultan operatori de observație multipli. Calibrarea bivariată $\mathcal{L}(\beta, \gamma, \mu) = w_I \mathrm{MAE}_I + w_D \mathrm{MAE}_D$, discutată în Section 7.7.1, este prima țintă naturală: oferă identifiabilitate practică pentru un model deja calibrat în lucrare (SIR) și servește ca reper metodologic pentru extensia la SIDARTHE.

O direcție tehnică imediată este promovarea calibrării din script offline într-un tip de job asincron în mecanismul Redis al aplicației, alături de `simulation_runs`. Rezultatul ar trebui să includă parametrii estimați, erorile MAE/RMSE pe train și test, numărul de evaluări ale solverului și timpul total de optimizare — toate stocate într-o tabelă dedicată și expuse prin API.

Pentru componenta ML, comparația cu liniile de bază a fost deja realizată pe seria României (Tabelul 6.2) și a arătat că persistența domină la orizontul de o zi. Următorul pas realist este repetarea acestei comparații pe toate cele 38 de țări europene din setul de date OWID, cu modelul compartimental calibrat ca reper suplimentar, agregând MAE, RMSE, nRMSE și timpul de antrenare per regiune. Având în vedere rezultatul de pe România, prioritatea ar trebui să fie varianta hibridă LSTM-peste-reziduuri — singurul regim în care rețeaua ar putea depăși un reper trivial.

La nivel de evaluare, următorul pas este extinderea campaniei de benchmark pe configurații hardware diferite (mașini locale, noduri mai puternice, eventual cu GPU pentru LSTM). Tot la nivel de sistem, lipsește o suită automată de teste: teste unitare și de integrare în backend cu o bază de date izolată și teste end-to-end pentru frontend care să verifice fluxurile reale din browser.

În zona de infrastructură, ar fi utilă introducerea unor mecanisme explicite de retry și dead-letter queue pentru job-urile care eșuează repetat, precum și activarea autoscalării orizontale (HPA) a runner-elor pe metrici personalizate (de exemplu lungimea cozii Redis) și a politicilor de rețea (NetworkPolicy) pentru izolare zero-trust intra-cluster. Terminarea TLS automată prin `cert-manager` și un stack de observabilitate (Prometheus + Grafana) ar completa profilul operațional al deployment-ului Kubernetes.

Pe termen mai lung, sistemul poate fi extins cu vizualizări geografice, surse de date actualizate periodic și module de alertare. Aceste extensii trebuie construite pe aceeași disciplină metodologică care a ghidat lucrarea de față: ipotezele modelului, calitatea datelor și metricile software trebuie să rămână vizibile, iar afirmațiile cantitative trebuie susținute prin rulări concrete și reproductibile, așa cum sunt cele din capitolele 7 și 8.

Bibliografie

[1] Roy M. Anderson and Robert M. May. *Infectious Diseases of Humans: Dynamics and Control*. Oxford University Press, Oxford, 1991.

[2] Herbert W. Hethcote. The mathematics of infectious diseases. *SIAM Review*, 42(4):599–653, 2000.

[3] W. O. Kermack and A. G. McKendrick. A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society of London. Series A*, 115(772):700–721, 1927.

[4] Karline Soetaert, Thomas Petzoldt, and R. Woodrow Setzer. Solving differential equations in R: Package deSolve. *Journal of Statistical Software*, 33(9):1–25, 2010.

[5] Christopher Rackauckas and Qing Nie. DifferentialEquations.jl – a performant and feature-rich ecosystem for solving differential equations in Julia. *Journal of Open Research Software*, 5(1):15, 2017.

[6] Pauli Virtanen et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

[7] Samuel M. Jenness, Steven M. Goodreau, and Martina Morris. EpiModel: An R package for mathematical modeling of infectious disease over networks. *Journal of Statistical Software*, 84(8):1–47, 2018.

[8] Cliff C. Kerr, Robyn M. Stuart, Dina Mistry, Romesh G. Abeysuriya, Katherine Rosenfeld, Gregory R. Hart, Rafael C. Núñez, Jamie A. Cohen, Prashanth Selvaraj, Brittany Hagedorn, and Daniel J. Klein. Covasim: An agent-based model of COVID-19 dynamics and interventions. *PLOS Computational Biology*, 17(7):e1009149, 2021.

[9] Neil M. Ferguson, Daniel Laydon, Gemma Nedjati-Gilani, et al. Report 9: Impact of non-pharmaceutical interventions (NPIs) to reduce COVID-19 mortality and healthcare demand. Technical report, Imperial College London, 2020.

[10] Bob Carpenter, Andrew Gelman, Matthew D. Hoffman, Daniel Lee, Ben Goodrich, Michael Betancourt, Marcus Brubaker, Jiqiang Guo, Peter Li, and Allen Riddell. Stan: A probabilistic programming language. *Journal of Statistical Software*, 76(1):1–32, 2017.

[11] Perry de Valpine, Daniel Turek, Christopher J. Paciorek, Clifford Anderson-Bergman, Duncan Temple Lang, and Rastislav Bodik. Programming with models: Writing statistical algorithms for general model structures with NIMBLE. *Journal of Computational and Graphical Statistics*, 26(2):403–413, 2017.

[12] Estee Y. Cramer, Evan L. Ray, Velma K. Lopez, et al. Evaluation of individual and ensemble probabilistic forecasts of COVID-19 mortality in the United States. *Proceedings of the National Academy of Sciences*, 119(15):e2113561119, 2022.

[13] Odo Diekmann, Johan A. P. Heesterbeek, and Johan A. J. Metz. On the definition and the computation of the basic reproduction ratio $\mathcal{R}_0$ in models for infectious diseases in heterogeneous populations. *Journal of Mathematical Biology*, 28(4):365–382, 1990.

[14] Giulia Giordano, Franco Blanchini, Raffaele Bruno, Patrizio Colaneri, Alessandro Di Filippo, Angela Di Matteo, and Marta Colaneri. Modelling the COVID-19 epidemic and implementation of population-wide interventions in italy. *Nature Medicine*, 26:855–860, 2020.

[15] Pauline van den Driessche and James Watmough. Reproduction numbers and sub-threshold endemic equilibria for compartmental models of disease transmission. *Mathematical Biosciences*, 180(1–2):29–48, 2002.

[16] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*, volume 8 of *Springer Series in Computational Mathematics*. Springer, Berlin, 2 edition, 1993.

[17] John R. Dormand and Peter J. Prince. A family of embedded Runge-Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980.

[18] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2024.

[19] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017.

[20] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. CreateSpace, Scotts Valley, CA, 2009.

[21] Yugabyte, Inc. YugabyteDB architecture documentation. Documentație tehnică, 2024.

[22] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and Kubernetes. *ACM Queue*, 14(1):70–93, 2016.

[23] The Kubernetes Authors. Kubernetes documentation. Documentație tehnică, 2024.

[24] The Go Authors. The Go programming language specification. Specificație oficială, 2024.

[25] Evan You et al. Vue.js documentation. Documentație oficială, 2024.

[26] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC'14)*, pages 305–319, Philadelphia, PA, 2014. USENIX Association.

[27] Rakyll and contributors. Standard Go project layout. Depozit GitHub, 2023.

[28] Michael Jones, John Bradley, and Nat Sakimura. RFC 7519: JSON web token (JWT). Request for Comments, Internet Engineering Task Force, 2015.

[29] Evan You and contributors. Pinia — the intuitive store for Vue.js. Documentație oficială, 2024.

[30] Edouard Mathieu, Hannah Ritchie, Lucas Rodés-Guirao, Cameron Appel, Charlie Giattino, Joe Hasell, Bobbie Macdonald, Saloni Dattani, Diana Beltekian, Esteban Ortiz-Ospina, and Max Roser. Coronavirus pandemic (COVID-19). Our World in Data, 2020.

[31] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[32] Paul J. Werbos. Backpropagation through time: What it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990.

[33] J. A. Nelder and R. Mead. A simplex method for function minimization. *The Computer Journal*, 7(4):308–313, 1965.

Anexa A
Cod Sursă — Implementarea R

Această anexă include runner-ul R folosit de sistem pentru consumul job-urilor Redis, rezolvarea modelelor compartimentale suportate și persistarea rezultatelor în YugabyteDB. Implementarea respectă *runner protocol*-ul descris în Section 4.7.1: ascultă `BRPOP` pe coada `epimodel:jobs:r`, parsează un mesaj JSON cu cheile `simulation_id`, `dataset_id`, `model_type`, `parameters`, `initial`, `population_n` și `t_max`, citește înregistrările dataset-ului prin `RPostgres`, integrează ecuațiile diferențiale corespunzătoare modelului prin `deSolve::ode` (metoda RK4 cu pas $h = 1$ zi) și scrie seria temporală în `simulation_results` (câte un rând per pas temporal, cu `compartments` ca JSONB) și metricile de execuție în `benchmarks`. La final, marchează `simulation_runs.status = 'completed'` sau `'failed'` cu `error_message`. Dependențele necesare sunt declarate în `code/runners/r/Dockerfile` (`r-base:4.x`, `deSolve`, `redux`, `RPostgres`, `jsonlite`).

```r
## Epidemiological model runner -- R.
##
## Consumes simulation jobs from Redis list ``epimodel:jobs:r`` (BRPOP via redux),
## solves SIS/SIR/SEIR/SIRS/SIDARTHE ODE systems with
## ``deSolve::ode`` (RK4, fixed step h = 1),
## and inserts compartment trajectories into YugabyteDB.

suppressPackageStartupMessages({
  library(deSolve)
  library(DBI)
  library(RPostgres)
  library(redux)
  library(jsonlite)
})

QUEUE <- "epimodel:jobs:r"
BRPOP_TIMEOUT_SEC <- 5L
ERROR_BACKOFF_SEC <- 2L

# Flag de oprire; R are signal handling limitat, ne bazam pe SIGTERM de la docker stop.
.running <- TRUE

sir_rhs <- function(t, state, parms) {
  invisible(t)
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
  })
}

sis_rhs <- function(t, state, parms) {
  invisible(t)
  with(as.list(c(state, parms)), {
    infections <- beta * S * I / N
    recoveries <- gamma * I
    dS <- -infections + recoveries
    dI <-  infections - recoveries
    list(c(dS, dI))
  })
}

seir_rhs <- function(t, state, parms) {
  invisible(t)
  with(as.list(c(state, parms)), {
```

```
    infections <- beta * S * I / N
    dS <- -infections
    dE <-  infections - sigma * E
    dI <-  sigma * E - gamma * I
    dR <-  gamma * I
    list(c(dS, dE, dI, dR))
  })
}

sirs_rhs <- function(t, state, parms) {
  invisible(t)
  with(as.list(c(state, parms)), {
    infections <- beta * S * I / N
    recoveries <- gamma * I
    waning <- xi * R
    dS <- -infections + waning
    dI <-  infections - recoveries
    dR <-  recoveries - waning
    list(c(dS, dI, dR))
  })
}

sidarthe_rhs <- function(t, state, parms) {
  invisible(t)
  with(as.list(c(state, parms)), {
    force <- (alpha * S_I + beta * D + gamma * A + delta * R) / N
    new_infections <- S * force
    dS <- -new_infections
    dI <- new_infections - (epsilon + zeta + lambda) * S_I
    dD <- epsilon * S_I - (eta + rho) * D
    dA <- zeta * S_I - (theta + mu + kappa) * A
    dR <- eta * D + theta * A - (nu + xi) * R
    dT <- mu * A + nu * R - (sigma + tau) * T
    dH <- lambda * S_I + rho * D + kappa * A + xi * R + sigma * T
    dE <- tau * T
    list(c(dS, dI, dD, dA, dR, dT, dH, dE))
  })
}

model_layout <- function(model_type) {
  if (identical(model_type, "sis")) {
    return(list(
      names = c("S", "I"),
      json_names = c("s", "i"),
      rhs = sis_rhs
    ))
  }
  if (identical(model_type, "sir")) {
    return(list(
      names = c("S", "I", "R"),
      json_names = c("s", "i", "r"),
      rhs = sir_rhs
    ))
  }
  if (identical(model_type, "seir")) {
    return(list(
      names = c("S", "E", "I", "R"),
      json_names = c("s", "e", "i", "r"),
      rhs = seir_rhs
    ))
  }
  if (identical(model_type, "sirs")) {
    return(list(
      names = c("S", "I", "R"),
      json_names = c("s", "i", "r"),
      rhs = sirs_rhs
    ))
  }
  if (identical(model_type, "sidarthe")) {
    return(list(
      names = c("S", "S_I", "D", "A", "R", "T", "H", "E"),
      json_names = c("s", "i", "d", "a", "r", "t", "h", "e"),
      rhs = sidarthe_rhs
    ))
  }
  stop(sprintf("unsupported model_type: %s", model_type))
```

```
}

peak_rss_mb <- function() {
  # /proc/self/status VmHWM is Linux-only; on macOS fall back to gc()
  status <- tryCatch(readLines("/proc/self/status", warn = FALSE), error = function(e) NULL)
  if (!is.null(status)) {
    line <- grep("^VmHWM:", status, value = TRUE)
    if (length(line) == 1) {
      kb <- as.numeric(sub("[^0-9]+([0-9]+).*", "\\1", line))
      return(kb / 1024)
    }
  }
  used_mb <- sum(gc()[, 2])
  used_mb
}

parse_dsn <- function(dsn) {
  # postgres://user:pass@host:port/db?sslmode=disable
  m <- regmatches(dsn, regexec(
    "postgres(?:ql)?://([^:]+):([^@]*)@([^:/]+):([0-9]+)/([^?]+)", dsn
  ))[[1]]
  if (length(m) < 6) stop(sprintf("invalid DSN: %s", dsn))
  list(user = m[2], password = m[3], host = m[4],
       port = as.integer(m[5]), dbname = m[6])
}

db_connect <- function(dsn) {
  p <- parse_dsn(dsn)
  dbConnect(RPostgres::Postgres(),
            host = p$host, port = p$port, dbname = p$dbname,
            user = p$user, password = p$password)
}

run_simulation <- function(conn, job) {
  sim_id <- job$simulation_id
  model_type <- if (!is.null(job$model_type)) job$model_type else "sidarthe"
  layout <- model_layout(model_type)
  parameters <- as.list(job$parameters)
  parameters <- lapply(parameters, as.numeric)
  N <- as.numeric(job$population_n)
  t_max <- as.integer(job$t_max)
  initials <- as.list(job$initial)
  y0 <- setNames(
    vapply(layout$json_names, function(name) {
      value <- initials[[name]]
      if (is.null(value)) 0 else as.numeric(value)
    }, numeric(1)),
    layout$names
  )

  dbExecute(conn, "UPDATE simulation_runs SET status = 'running' WHERE id = $1",
            params = list(sim_id))

  rss_before <- peak_rss_mb()
  wall_start <- proc.time()["elapsed"]
  cpu_start  <- proc.time()["user.self"] + proc.time()["sys.self"]

  times <- seq(0, t_max, by = 1)
  parms <- c(unlist(parameters), N = N)
  out <- ode(y = y0, times = times, func = layout$rhs, parms = parms, method = "rk4")

  wall_ms <- as.integer((proc.time()["elapsed"] - wall_start) * 1000)
  cpu_ms  <- as.integer(((proc.time()["user.self"] + proc.time()["sys.self"]) - cpu_start) * 1000)
  peak_mb <- max(peak_rss_mb(), rss_before)

  times_out <- as.integer(out[, "time"])

  dbWithTransaction(conn, {
    for (idx in seq_along(times_out)) {
      values <- as.list(pmax(0, as.numeric(out[idx, layout$names])))
      names(values) <- layout$json_names
      dbExecute(conn,
        "INSERT INTO simulation_results (simulation_id, t, compartments) VALUES ($1, $2, $3::jsonb)",
        params = list(sim_id, times_out[idx], toJSON(values, auto_unbox = TRUE)))
    }
    dbExecute(conn,
```

```
      "INSERT INTO benchmarks (simulation_id, wall_time_ms, cpu_time_ms, peak_memory_mb) VALUES ($1, $2, $3, $4)",
      params = list(sim_id, wall_ms, cpu_ms, peak_mb))
    dbExecute(conn,
      "UPDATE simulation_runs SET status = 'completed', completed_at = NOW() WHERE id = $1",
      params = list(sim_id))
  })

  message(sprintf("sim %s done: wall=%dms cpu=%dms mem=%.1fMB rows=%d",
                  sim_id, wall_ms, cpu_ms, peak_mb, length(times_out)))
}

mark_failed <- function(conn, sim_id, msg) {
  tryCatch(
    dbExecute(conn,
      "UPDATE simulation_runs SET status = 'failed', error_message = $1, completed_at = NOW() WHERE id = $2",
      params = list(substr(msg, 1, 2000), sim_id)),
    error = function(e) message(sprintf("could not mark failed: %s", e$message))
  )
}

main <- function() {
  redis_url <- Sys.getenv("REDIS_URL", "redis://redis:6379/0")
  db_dsn_env <- Sys.getenv("DATABASE_URL", "")

  # redux uses host/port/db number; parse URL
  m <- regmatches(redis_url, regexec(
    "redis://(?:[^@]*@)?([^:/]+):([0-9]+)(?:/([0-9]+))?", redis_url
  ))[[1]]
  if (length(m) < 4) stop(sprintf("invalid REDIS_URL: %s", redis_url))
  rdb <- redux::hiredis(host = m[2], port = as.integer(m[3]),
                        db = if (m[4] == "") 0L else as.integer(m[4]))
  rdb$PING()
  message(sprintf("runner-r connected to redis, consuming %s", QUEUE))

  while (.running) {
    item <- tryCatch(rdb$BRPOP(QUEUE, BRPOP_TIMEOUT_SEC),
                     error = function(e) {
                       message(sprintf("redis error: %s", e$message))
                       Sys.sleep(ERROR_BACKOFF_SEC)
                       NULL
                     })
    if (is.null(item)) next

    raw <- item[[2]]
    job <- tryCatch(fromJSON(raw), error = function(e) {
      message(sprintf("invalid job json: %s", e$message))
      NULL
    })
    if (is.null(job)) next

    sim_id <- if (!is.null(job$simulation_id)) job$simulation_id else "?"
    dsn <- if (!is.null(job$db_dsn) && nzchar(job$db_dsn)) job$db_dsn else db_dsn_env
    if (!nzchar(dsn)) {
      message(sprintf("sim %s: no db_dsn", sim_id))
      next
    }

    conn <- tryCatch(db_connect(dsn), error = function(e) {
      message(sprintf("sim %s db connect failed: %s", sim_id, e$message))
      NULL
    })
    if (is.null(conn)) next

    tryCatch({
      run_simulation(conn, job)
    }, error = function(e) {
      message(sprintf("sim %s failed: %s", sim_id, e$message))
      mark_failed(conn, sim_id, e$message)
    }, finally = {
      dbDisconnect(conn)
    })
  }
  message("runner-r stopped")
}

```

```
274 main()
```

Listing A.1: Runner-ul R pentru modelele SIDARTHE/SIR/SIS/SEIR/SIRS.

Anexa B
Cod Sursă — Implementarea Julia

Această anexă include runner-ul Julia, bazat pe `DifferentialEquations.jl`, `Redis.jl` și `LibPQ.jl`. Respectă același *runner protocol* ca runner-ul R (Section 4.7.1): ascultă coada `epimodel:jobs:julia`, preia mesajul JSON cu parametrii și starea inițială, integrează modelul cerut prin `Tsit5` (Runge–Kutta adaptiv de ordin 5(4), tolerantă 10^{-8} relativ) și scrie traiectoria în `simulation_results`. Specificul Julia este pre-alocarea vectorului de derivate du pentru zero alocări per pas și costul de pornire JIT amortizat doar după prima rulare a unui model dat în pod, vizibil empiric ca un overhead de ∼900–1 500 ms pe primele job-uri (Table 8.4). Dependențele sunt declarate prin `Project.toml` în `code/runners/julia/`, iar imaginea de bază este `julia:1.x-bookworm` pentru reproductibilitatea versiunilor compilate.

```
#=
Epidemiological model runner -- Julia.

Consumes simulation jobs from Redis list ‘epimodel:jobs:julia‘ (BRPOP),
solves SIS/SIR/SEIR/SIRS/SIDARTHE ODE systems with
DifferentialEquations.jl (Tsit5, saveat=1),
and inserts compartment trajectories into YugabyteDB via LibPQ.
=#

using DifferentialEquations
using JSON3
using LibPQ
using Redis

const QUEUE = "epimodel:jobs:julia"
const BRPOP_TIMEOUT_SEC = 5
const ERROR_BACKOFF_SEC = 2

mutable struct RunCtx
    running::Bool
end
const CTX = RunCtx(true)

function sir_rhs!(du, u, p, t)
    _ = t
    S, I, R = u
    beta = p["beta"]
    gamma = p["gamma"]
    N = p["N"]
    du[1] = -beta * S * I / N
    du[2] =  beta * S * I / N - gamma * I
    du[3] =  gamma * I
end

function sis_rhs!(du, u, p, t)
    _ = t
    S, I = u
    beta = p["beta"]
    gamma = p["gamma"]
    N = p["N"]
    infections = beta * S * I / N
    recoveries = gamma * I
    du[1] = -infections + recoveries
    du[2] =  infections - recoveries
end

function seir_rhs!(du, u, p, t)
    _ = t
    S, E, I, R = u
```

```
    beta = p["beta"]
    sigma = p["sigma"]
    gamma = p["gamma"]
    N = p["N"]
    infections = beta * S * I / N
    du[1] = -infections
    du[2] =  infections - sigma * E
    du[3] =  sigma * E - gamma * I
    du[4] =  gamma * I
end

function sirs_rhs!(du, u, p, t)
    _ = t
    S, I, R = u
    beta = p["beta"]
    gamma = p["gamma"]
    xi = p["xi"]
    N = p["N"]
    infections = beta * S * I / N
    recoveries = gamma * I
    waning = xi * R
    du[1] = -infections + waning
    du[2] =  infections - recoveries
    du[3] =  recoveries - waning
end

function sidarthe_rhs!(du, u, p, t)
    _ = t
    S, I, D, A, R, T, H, E = u
    _ = E
    N = p["N"]
    force = (p["alpha"] * I + p["beta"] * D + p["gamma"] * A + p["delta"] * R) / N
    new_infections = S * force

    du[1] = -new_infections
    du[2] = new_infections - (p["epsilon"] + p["zeta"] + p["lambda"]) * I
    du[3] = p["epsilon"] * I - (p["eta"] + p["rho"]) * D
    du[4] = p["zeta"] * I - (p["theta"] + p["mu"] + p["kappa"]) * A
    du[5] = p["eta"] * D + p["theta"] * A - (p["nu"] + p["xi"]) * R
    du[6] = p["mu"] * A + p["nu"] * R - (p["sigma"] + p["tau"]) * T
    du[7] = p["lambda"] * I + p["rho"] * D + p["kappa"] * A + p["xi"] * R + p["sigma"] * T
    du[8] = p["tau"] * T
    nothing
end

function model_layout(model_type::AbstractString)
    if model_type == "sis"
        return (state_names = ["s", "i"], rhs = sis_rhs!)
    elseif model_type == "sir"
        return (state_names = ["s", "i", "r"], rhs = sir_rhs!)
    elseif model_type == "seir"
        return (state_names = ["s", "e", "i", "r"], rhs = seir_rhs!)
    elseif model_type == "sirs"
        return (state_names = ["s", "i", "r"], rhs = sirs_rhs!)
    elseif model_type == "sidarthe"
        return (state_names = ["s", "i", "d", "a", "r", "t", "h", "e"], rhs = sidarthe_rhs!)
    end
    error("unsupported model_type: $model_type")
end

peak_rss_mb() = Sys.maxrss() / (1024 * 1024)

function parse_redis_url(url::AbstractString)
    m = match(r"^redis://(?:[^@]*@)?(?<host>[^:/]+):(?<port>\d+)(?:/(?<db>\d+))?$", url)
    m === nothing && error("invalid REDIS_URL: $url")
    (host = m[:host],
     port = parse(Int, m[:port]),
     db   = m[:db] === nothing ? 0 : parse(Int, m[:db]))
end

function run_simulation(conn::LibPQ.Connection, job::Dict{Symbol, Any})
    sim_id = String(job[:simulation_id])
    model_type = String(get(job, :model_type, "sidarthe"))
    layout = model_layout(model_type)
    params = Dict{String, Float64}(String(k) => Float64(v) for (k, v) in pairs(job[:parameters]))
    N = Float64(job[:population_n])
```

```
    t_max = Int(job[:t_max])
    params["N"] = N
    initial = Dict{String, Float64}(String(k) => Float64(v) for (k, v) in pairs(job[:initial]))
    y0 = Float64[get(initial, name, 0.0) for name in layout.state_names]

    LibPQ.execute(conn,
        "UPDATE simulation_runs SET status = 'running' WHERE id = \$1",
        [sim_id])

    rss_before = peak_rss_mb()
    wall_start = time_ns()
    cpu_start  = time()  # process CPU not directly exposed; use elapsed as fallback

    prob = ODEProblem(layout.rhs, y0, (0.0, Float64(t_max)), params)
    sol = solve(prob, Tsit5(); saveat = 0:1.0:t_max, reltol = 1e-8, abstol = 1e-10)

    wall_ms = round(Int, (time_ns() - wall_start) / 1_000_000)
    cpu_ms  = round(Int, (time() - cpu_start) * 1000)
    peak_mb = max(peak_rss_mb(), rss_before)

    LibPQ.execute(conn, "BEGIN")
    try
        stmt = LibPQ.prepare(conn,
            "INSERT INTO simulation_results (simulation_id, t, compartments) VALUES (\$1, \$2, \$3::jsonb)")
        for (idx, t) in enumerate(sol.t)
            u = sol.u[idx]
            values = Dict(name => max(0.0, Float64(u[pos])) for (pos, name) in enumerate(layout.state_names))
            LibPQ.execute(stmt, (sim_id, Int(t), JSON3.write(values)))
        end
        LibPQ.execute(conn,
            "INSERT INTO benchmarks (simulation_id, wall_time_ms, cpu_time_ms, peak_memory_mb) VALUES (\$1,
↪ \$2, \$3, \$4)",
            [sim_id, wall_ms, cpu_ms, peak_mb])
        LibPQ.execute(conn,
            "UPDATE simulation_runs SET status = 'completed', completed_at = NOW() WHERE id = \$1",
            [sim_id])
        LibPQ.execute(conn, "COMMIT")
    catch e
        LibPQ.execute(conn, "ROLLBACK")
        rethrow(e)
    end

    println("sim $sim_id done: wall=$(wall_ms)ms cpu=$(cpu_ms)ms mem=$(round(peak_mb, digits=1))MB
↪ rows=$(length(sol.t))")
end

function mark_failed(conn::LibPQ.Connection, sim_id::AbstractString, msg::AbstractString)
    try
        LibPQ.execute(conn,
            "UPDATE simulation_runs SET status = 'failed', error_message = \$1, completed_at = NOW() WHERE
↪ id = \$2",
            [first(msg, 2000), sim_id])
    catch e
        @warn "could not mark failed" exception=e
    end
end

function main()
    redis_url = get(ENV, "REDIS_URL", "redis://redis:6379/0")
    db_dsn_env = get(ENV, "DATABASE_URL", "")
    rinfo = parse_redis_url(redis_url)
    rcli = RedisConnection(host = rinfo.host, port = rinfo.port, db = rinfo.db)
    Redis.ping(rcli)
    println("runner-julia connected to redis, consuming $QUEUE")

    while CTX.running
        item = try
            Redis.brpop(rcli, [QUEUE], BRPOP_TIMEOUT_SEC)
        catch e
            @warn "redis error" exception=e
            sleep(ERROR_BACKOFF_SEC)
            nothing
        end
        item === nothing && continue
        # Redis.brpop returns Vector{String} [queue_name, payload] or nil
        length(item) < 2 && continue
```

```
        raw = item[2]

        job = try
            JSON3.read(raw, Dict{Symbol, Any})
        catch e
            @warn "invalid job json" exception=e
            continue
        end

        sim_id = get(job, :simulation_id, "?")
        dsn = get(job, :db_dsn, "")
        if isempty(dsn)
            dsn = db_dsn_env
        end
        if isempty(dsn)
            @warn "no db_dsn for sim" sim_id
            continue
        end

        conn = try
            LibPQ.Connection(dsn)
        catch e
            @warn "db connect failed" sim_id exception=e
            continue
        end
        try
            run_simulation(conn, job)
        catch e
            @warn "sim failed" sim_id exception=e
            mark_failed(conn, String(sim_id), sprint(showerror, e))
        finally
            close(conn)
        end
    end
    println("runner-julia stopped")
end

main()
```

Listing B.1: Runner-ul Julia pentru modelele SIDARTHE/SIR/SIS/SEIR/SIRS.

Anexa C
Cod Sursă — Implementarea Python

Această anexă include runner-ul Python, care folosește `scipy.integrate.solve_ivp` pentru integrarea numerică și `psycopg` pentru scrierea rezultatelor. Respectă același *runner protocol* ca celelalte două (Section 4.7.1): ascultă coada `epimodel:jobs:python`, preia mesajul JSON și integrează modelul cerut cu `RK45` (Dormand–Prince, adaptiv, aceeași toleranță cu `Tsit5` din Julia). Acest runner este alegerea *default* a aplicației pentru flux-uri interactive: nu are cost de pornire JIT, are stack-ul cel mai stabil ca performanță (4–9 ms per job în benchmark, Table 8.4) și beneficiază de ecosistemul Python pentru extensii ulterioare (sensitivity sweeps, profiling, conexiuni la alte servicii). Dependențele sunt declarate în `code/runners/python/requirements.txt`, iar imaginea de bază este `python:3.11-slim`.

```
"""Epidemiological model runner -- Python.

Consumes simulation jobs from Redis list ``epimodel:jobs:python`` (BRPOP),
solves SIS/SIR/SEIR/SIRS/SIDARTHE ODE systems with
``scipy.integrate.solve_ivp`` (RK45),
inserts compartment trajectories into YugabyteDB and records the benchmark
(wall time, CPU time, peak RSS) into ``benchmarks``.
"""

from __future__ import annotations

import json
import os
import resource
import signal
import sys
import time
from typing import Any

import numpy as np
import psycopg
import redis
from psycopg.rows import dict_row
from psycopg.types.json import Json
from scipy.integrate import solve_ivp

QUEUE = "epimodel:jobs:python"
BRPOP_TIMEOUT_SEC = 5
ERROR_BACKOFF_SEC = 2

_running = True


def _handle_signal(signum, _frame):
    global _running
    print(f"signal {signum} received, shutting down", file=sys.stderr, flush=True)
    _running = False


def sir_rhs(t: float, y: np.ndarray, params: dict[str, float], n: float) -> list[float]:
    del t
    s, i, r = y
    beta = params["beta"]
    gamma = params["gamma"]
    return [
        -beta * s * i / n,
        beta * s * i / n - gamma * i,
        gamma * i,
    ]
```

```python


def sis_rhs(t: float, y: np.ndarray, params: dict[str, float], n: float) -> list[float]:
    del t
    s, i = y
    beta = params["beta"]
    gamma = params["gamma"]
    infections = beta * s * i / n
    recoveries = gamma * i
    return [-infections + recoveries, infections - recoveries]


def seir_rhs(t: float, y: np.ndarray, params: dict[str, float], n: float) -> list[float]:
    del t
    s, e, i, r = y
    beta = params["beta"]
    sigma = params["sigma"]
    gamma = params["gamma"]
    infections = beta * s * i / n
    return [
        -infections,
        infections - sigma * e,
        sigma * e - gamma * i,
        gamma * i,
    ]


def sirs_rhs(t: float, y: np.ndarray, params: dict[str, float], n: float) -> list[float]:
    del t
    s, i, r = y
    beta = params["beta"]
    gamma = params["gamma"]
    xi = params["xi"]
    infections = beta * s * i / n
    recoveries = gamma * i
    waning = xi * r
    return [-infections + waning, infections - recoveries, recoveries - waning]


def sidarthe_rhs(t: float, y: np.ndarray, params: dict[str, float], n: float) -> list[float]:
    del t
    s, i, d, a, r, threatened, h, extinct = y
    del extinct
    force = (params["alpha"] * i + params["beta"] * d + params["gamma"] * a + params["delta"] * r) / n
    new_infections = s * force

    return [
        -new_infections,
        new_infections - (params["epsilon"] + params["zeta"] + params["lambda"]) * i,
        params["epsilon"] * i - (params["eta"] + params["rho"]) * d,
        params["zeta"] * i - (params["theta"] + params["mu"] + params["kappa"]) * a,
        params["eta"] * d + params["theta"] * a - (params["nu"] + params["xi"]) * r,
        params["mu"] * a + params["nu"] * r - (params["sigma"] + params["tau"]) * threatened,
        params["lambda"] * i + params["rho"] * d + params["kappa"] * a + params["xi"] * r + params["sigma"]
↪ * threatened,
        params["tau"] * threatened,
    ]


def model_layout(model_type: str) -> tuple[list[str], Any]:
    if model_type == "sis":
        return ["s", "i"], sis_rhs
    if model_type == "sir":
        return ["s", "i", "r"], sir_rhs
    if model_type == "seir":
        return ["s", "e", "i", "r"], seir_rhs
    if model_type == "sirs":
        return ["s", "i", "r"], sirs_rhs
    if model_type == "sidarthe":
        return ["s", "i", "d", "a", "r", "t", "h", "e"], sidarthe_rhs
    raise ValueError(f"unsupported model_type: {model_type}")


def _peak_rss_mb() -> float:
    """Return peak RSS in MB across platforms.

```

```
    Linux returns ``ru_maxrss`` in KiB, macOS in bytes.
    """
    rss = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss
    if sys.platform == "darwin":
        return rss / (1024 * 1024)
    return rss / 1024


def run_simulation(conn: psycopg.Connection, job: dict[str, Any]) -> None:
    sim_id = job["simulation_id"]
    model_type = job.get("model_type", "sidarthe")
    params = {str(k): float(v) for k, v in job["parameters"].items()}
    initial = {str(k): float(v) for k, v in job["initial"].items()}
    n = float(job["population_n"])
    t_max = int(job["t_max"])
    compartments, rhs = model_layout(model_type)
    y0 = [initial.get(name, 0.0) for name in compartments]

    with conn.transaction():
        conn.execute(
            "UPDATE simulation_runs SET status = 'running' WHERE id = %s",
            (sim_id,),
        )

    rss_before = _peak_rss_mb()
    wall_start = time.perf_counter()
    cpu_start = time.process_time()

    sol = solve_ivp(
        fun=rhs,
        t_span=(0.0, float(t_max)),
        y0=y0,
        method="RK45",
        t_eval=np.arange(0, t_max + 1, dtype=float),
        args=(params, n),
        rtol=1e-8,
        atol=1e-10,
    )
    if not sol.success:
        raise RuntimeError(f"solve_ivp failed: {sol.message}")

    wall_ms = int((time.perf_counter() - wall_start) * 1000)
    cpu_ms = int((time.process_time() - cpu_start) * 1000)
    peak_mb = max(_peak_rss_mb(), rss_before)

    rows = []
    for idx, t in enumerate(sol.t):
        values = {
            name: max(0.0, float(sol.y[pos][idx]))
            for pos, name in enumerate(compartments)
        }
        rows.append((sim_id, int(t), Json(values)))

    with conn.transaction():
        with conn.cursor() as cur:
            cur.executemany(
                "INSERT INTO simulation_results (simulation_id, t, compartments) VALUES (%s, %s, %s)",
                rows,
            )
            cur.execute(
                "INSERT INTO benchmarks (simulation_id, wall_time_ms, cpu_time_ms, peak_memory_mb) VALUES
↪ (%s, %s, %s, %s)",
                (sim_id, wall_ms, cpu_ms, peak_mb),
            )
            cur.execute(
                "UPDATE simulation_runs SET status = 'completed', completed_at = NOW() WHERE id = %s",
                (sim_id,),
            )
    print(
        f"sim {sim_id} done: wall={wall_ms}ms cpu={cpu_ms}ms mem={peak_mb:.1f}MB rows={len(rows)}",
        flush=True,
    )


def mark_failed(conn: psycopg.Connection, sim_id: str, msg: str) -> None:
    try:
```

```
        with conn.transaction():
            conn.execute(
                "UPDATE simulation_runs SET status = 'failed', error_message = %s, completed_at = NOW()
↪ WHERE id = %s",
                (msg[:2000], sim_id),
            )
    except Exception as exc:  # noqa: BLE001
        print(f"could not mark failed: {exc}", file=sys.stderr, flush=True)


def main() -> int:
    signal.signal(signal.SIGINT, _handle_signal)
    signal.signal(signal.SIGTERM, _handle_signal)

    redis_url = os.environ["REDIS_URL"]
    db_dsn = os.environ.get("DATABASE_URL")

    rdb = redis.from_url(redis_url)
    rdb.ping()
    print(f"runner-python connected to redis, consuming {QUEUE}", flush=True)

    while _running:
        try:
            item = rdb.brpop(QUEUE, timeout=BRPOP_TIMEOUT_SEC)
        except redis.RedisError as exc:
            print(f"redis error: {exc}", file=sys.stderr, flush=True)
            time.sleep(ERROR_BACKOFF_SEC)
            continue
        if item is None:
            continue

        _, raw = item
        try:
            job = json.loads(raw)
        except json.JSONDecodeError as exc:
            print(f"invalid job json: {exc}", file=sys.stderr, flush=True)
            continue

        sim_id = job.get("simulation_id", "?")
        dsn = job.get("db_dsn") or db_dsn
        if not dsn:
            print(f"sim {sim_id}: no db_dsn", file=sys.stderr, flush=True)
            continue

        try:
            with psycopg.connect(dsn, row_factory=dict_row) as conn:
                run_simulation(conn, job)
        except Exception as exc:  # noqa: BLE001
            print(f"sim {sim_id} failed: {exc}", file=sys.stderr, flush=True)
            try:
                with psycopg.connect(dsn) as conn:
                    mark_failed(conn, sim_id, str(exc))
            except Exception as inner:  # noqa: BLE001
                print(f"could not connect to mark failed: {inner}", file=sys.stderr, flush=True)

    print("runner-python stopped", flush=True)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Listing C.1: Runner-ul Python pentru modelele SIDARTHE/SIR/SIS/SEIR/SIRS.

Anexa D
Cod Sursă — Runner LSTM

Această anexă include runner-ul LSTM separat, responsabil pentru consumul cozii `epimodel:jobs:lstm`, antrenarea modelului recurent pe seria `infected` și persistarea seriilor `observed`, `fitted` și `predicted`. Spre deosebire de runner-ele compartimentale, această implementare nu rezolvă ecuații diferențiale, ci antrenează o rețea LSTM hand-rolled în NumPy (forward pass cu poartă de intrare, de uitare, de ieșire și celulă; backward pass prin BPTT, cu Adam ca optimizator) — detalii matematice în Chapter 6. Split-ul cronologic 80%/20% separă fereastra de antrenare de cea de test, normalizarea este ajustată doar pe train pentru a evita scurgerea informației, iar metricile MAE, RMSE și nRMSE sunt calculate pe fereastra de test. Validarea împotriva predictorilor naivi (Table 6.2) este executată separat prin `scripts/lstm_baselines.py`, nu de runner. Dependențele sunt declarate în `code/runners/lstm/requirements.txt` (`numpy`, `redis`, `psycopg`), iar imaginea separată permite scalarea independentă a calculului LSTM (costisitor, ∼290 s per rulare) față de solverele ODE (sub-secunde).

```
"""Dedicated LSTM runner.

Consumes prediction jobs from Redis list ``epimodel:jobs:lstm``, trains a
small univariate LSTM over the dataset's infected time series, writes observed,
fitted and forecast values into ``simulation_results``, and records execution
metrics in ``benchmarks``.
"""

from __future__ import annotations

import json
import os
import resource
import signal
import sys
import time
from dataclasses import dataclass
from typing import Any

import numpy as np
import psycopg
import redis
from psycopg.rows import dict_row
from psycopg.types.json import Json

QUEUE = "epimodel:jobs:lstm"
BRPOP_TIMEOUT_SEC = 5
ERROR_BACKOFF_SEC = 2
TEST_FRACTION = 0.2  # ultima parte a seriei, tinuta deoparte pentru evaluarea generalizarii

_running = True


def _handle_signal(signum, _frame):
    global _running
    print(f"signal {signum} received, shutting down", file=sys.stderr, flush=True)
    _running = False


def _peak_rss_mb() -> float:
    rss = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss
    if sys.platform == "darwin":
        return rss / (1024 * 1024)
```

```
    return rss / 1024


def _sigmoid(x: np.ndarray) -> np.ndarray:
    return 1.0 / (1.0 + np.exp(-np.clip(x, -50.0, 50.0)))


@dataclass
class SampleSet:
    x: np.ndarray
    y: np.ndarray


@dataclass
class TrainOutcome:
    fitted: list[float | None]  # predictii o-zi-inainte, lungime egala cu seria
    forecast: list[float]       # prognoza autoregresiva dincolo de ultima zi observata
    train_end: int              # primul indice al ferestrei de test
    mae: float
    rmse: float
    nrmse: float


class LSTMRegressor:
    def __init__(self, hidden_size: int, learning_rate: float, seed: int = 42) -> None:
        self.hidden_size = hidden_size
        self.learning_rate = learning_rate
        rng = np.random.default_rng(seed)
        width = hidden_size + 1
        scale = 0.1

        self.wf = rng.normal(0.0, scale, (hidden_size, width))
        self.wi = rng.normal(0.0, scale, (hidden_size, width))
        self.wo = rng.normal(0.0, scale, (hidden_size, width))
        self.wg = rng.normal(0.0, scale, (hidden_size, width))
        self.bf = np.full((hidden_size, 1), 0.5)
        self.bi = np.zeros((hidden_size, 1))
        self.bo = np.zeros((hidden_size, 1))
        self.bg = np.zeros((hidden_size, 1))
        self.wy = rng.normal(0.0, scale, (1, hidden_size))
        self.by = np.zeros((1, 1))

    def forward(self, seq: np.ndarray) -> tuple[float, list[dict[str, np.ndarray]]]:
        h = np.zeros((self.hidden_size, 1))
        c = np.zeros((self.hidden_size, 1))
        cache: list[dict[str, np.ndarray]] = []

        for value in seq:
            x = np.array([[float(value)]])
            h_prev = h
            c_prev = c
            z = np.vstack([h_prev, x])

            f = _sigmoid(self.wf @ z + self.bf)
            i = _sigmoid(self.wi @ z + self.bi)
            o = _sigmoid(self.wo @ z + self.bo)
            g = np.tanh(self.wg @ z + self.bg)

            c = f * c_prev + i * g
            h = o * np.tanh(c)
            cache.append({
                "z": z,
                "f": f,
                "i": i,
                "o": o,
                "g": g,
                "c": c,
                "c_prev": c_prev,
                "h": h,
            })

        y_hat = float((self.wy @ h + self.by)[0, 0])
        return y_hat, cache

    def predict(self, seq: np.ndarray) -> float:
        y_hat, _ = self.forward(seq)
```

```
        return y_hat

    def train_one(self, seq: np.ndarray, target: float) -> float:
        y_hat, cache = self.forward(seq)
        error = y_hat - float(target)

        grads = {
            "wf": np.zeros_like(self.wf),
            "wi": np.zeros_like(self.wi),
            "wo": np.zeros_like(self.wo),
            "wg": np.zeros_like(self.wg),
            "bf": np.zeros_like(self.bf),
            "bi": np.zeros_like(self.bi),
            "bo": np.zeros_like(self.bo),
            "bg": np.zeros_like(self.bg),
            "wy": np.zeros_like(self.wy),
            "by": np.zeros_like(self.by),
        }

        dy = 2.0 * error
        grads["wy"] += dy * cache[-1]["h"].T
        grads["by"] += dy

        dh_next = self.wy.T * dy
        dc_next = np.zeros((self.hidden_size, 1))

        for step in reversed(cache):
            z = step["z"]
            f = step["f"]
            i = step["i"]
            o = step["o"]
            g = step["g"]
            c = step["c"]
            c_prev = step["c_prev"]

            tanh_c = np.tanh(c)
            do = dh_next * tanh_c
            dao = do * o * (1.0 - o)

            dc = dh_next * o * (1.0 - tanh_c * tanh_c) + dc_next
            df = dc * c_prev
            daf = df * f * (1.0 - f)
            di = dc * g
            dai = di * i * (1.0 - i)
            dg = dc * i
            dag = dg * (1.0 - g * g)

            grads["wf"] += daf @ z.T
            grads["wi"] += dai @ z.T
            grads["wo"] += dao @ z.T
            grads["wg"] += dag @ z.T
            grads["bf"] += daf
            grads["bi"] += dai
            grads["bo"] += dao
            grads["bg"] += dag

            dz = self.wf.T @ daf + self.wi.T @ dai + self.wo.T @ dao + self.wg.T @ dag
            dh_next = dz[: self.hidden_size]
            dc_next = dc * f

        for key, grad in grads.items():
            np.clip(grad, -5.0, 5.0, out=grad)
            setattr(self, key, getattr(self, key) - self.learning_rate * grad)

        return float(error * error)


def fetch_infected_series(conn: psycopg.Connection, dataset_id: str) -> np.ndarray:
    rows = conn.execute(
        """
        SELECT infected
        FROM epidemiological_records
        WHERE dataset_id = %s
        ORDER BY recorded_date ASC, id ASC
        """,
        (dataset_id,),
```

```
    ).fetchall()
    if not rows:
        raise RuntimeError("dataset has no epidemiological records")
    return np.array([float(row["infected"]) for row in rows], dtype=float)


def make_samples(values: np.ndarray, lookback: int) -> SampleSet:
    x_rows = []
    y_rows = []
    for idx in range(lookback, len(values)):
        x_rows.append(values[idx - lookback:idx])
        y_rows.append(values[idx])
    if not x_rows:
        raise RuntimeError("not enough data points for LSTM lookback")
    return SampleSet(x=np.array(x_rows, dtype=float), y=np.array(y_rows, dtype=float))


def fit_scaler(values: np.ndarray) -> tuple[float, float]:
    """Media si abaterea standard ale normalizarii, calibrate doar pe antrenare."""
    mean = float(np.mean(values))
    std = float(np.std(values))
    if std < 1e-9:
        std = 1.0
    return mean, std


def denormalize(value: float, mean: float, std: float) -> float:
    return max(0.0, float(value * std + mean))


def split_index(n: int, lookback: int) -> int:
    """Primul indice al ferestrei de test, dupa o impartire cronologica.

    Fereastra de test este ultima parte a seriei (``TEST_FRACTION``), dar
    portiunea de antrenare trebuie sa pastreze cel putin un esantion complet
    de lungime ``lookback``.
    """
    test_size = min(round(TEST_FRACTION * n), n - lookback - 1)
    if test_size < 1:
        raise RuntimeError("dataset prea scurt pentru o impartire train/test")
    return n - test_size


def evaluate(
    series: np.ndarray, fitted: list[float | None], train_end: int, population: float
) -> tuple[float, float, float]:
    """MAE, RMSE si nRMSE pe fereastra de test (definitii din capitolul 7)."""
    errors: list[float] = []
    for idx in range(train_end, len(series)):
        pred = fitted[idx]
        if pred is not None:
            errors.append(pred - float(series[idx]))
    if not errors:
        return 0.0, 0.0, 0.0
    arr = np.array(errors, dtype=float)
    mae = float(np.mean(np.abs(arr)))
    rmse = float(np.sqrt(np.mean(arr**2)))
    nrmse = rmse / population if population > 0 else 0.0
    return mae, rmse, nrmse


def train_model(series: np.ndarray, params: dict[str, float], population: float) -> TrainOutcome:
    lookback = int(params["lookback"])
    hidden_size = int(params["hidden_size"])
    epochs = int(params["epochs"])
    learning_rate = float(params["learning_rate"])
    horizon = int(params["forecast_horizon"])

    n = len(series)
    train_end = split_index(n, lookback)

    # Normalizarea se calibreaza exclusiv pe portiunea de antrenare, pentru a nu
    # permite scurgeri de informatie din fereastra de test.
    mean, std = fit_scaler(series[:train_end])
    normalized = (series - mean) / std

```

```
    samples = make_samples(normalized[:train_end], lookback)
    model = LSTMRegressor(hidden_size=hidden_size, learning_rate=learning_rate)
    rng = np.random.default_rng(123)
    for _ in range(epochs):
        for idx in rng.permutation(len(samples.x)):
            model.train_one(samples.x[idx], float(samples.y[idx]))

    # Predictii o-zi-inainte pe toata seria: in portiunea de antrenare sunt
    # in-sample, iar pe fereastra de test sunt out-of-sample, deoarece modelul
    # nu a vazut acele tinte.
    fitted: list[float | None] = [None] * n
    for idx in range(lookback, n):
        pred_norm = model.predict(normalized[idx - lookback:idx])
        fitted[idx] = denormalize(pred_norm, mean, std)

    mae, rmse, nrmse = evaluate(series, fitted, train_end, population)

    # Prognoza autoregresiva dincolo de ultima zi observata.
    window = list(normalized[-lookback:])
    forecast: list[float] = []
    for _ in range(horizon):
        pred_norm = model.predict(np.array(window, dtype=float))
        forecast.append(denormalize(pred_norm, mean, std))
        window = [*window[1:], pred_norm]

    return TrainOutcome(
        fitted=fitted, forecast=forecast, train_end=train_end,
        mae=mae, rmse=rmse, nrmse=nrmse,
    )


def run_lstm(conn: psycopg.Connection, job: dict[str, Any]) -> None:
    sim_id = job["simulation_id"]
    dataset_id = job["dataset_id"]
    params = {str(k): float(v) for k, v in job["parameters"].items()}
    params["forecast_horizon"] = float(int(job["t_max"]))
    population = float(job["population_n"])

    with conn.transaction():
        conn.execute(
            "UPDATE simulation_runs SET status = 'running' WHERE id = %s",
            (sim_id,),
        )

    rss_before = _peak_rss_mb()
    wall_start = time.perf_counter()
    cpu_start = time.process_time()

    series = fetch_infected_series(conn, dataset_id)
    if len(series) <= int(params["lookback"]) + 1:
        raise RuntimeError("dataset length must exceed lookback + 1 for a train/test split")
    outcome = train_model(series, params, population)

    wall_ms = int((time.perf_counter() - wall_start) * 1000)
    cpu_ms = int((time.process_time() - cpu_start) * 1000)
    peak_mb = max(_peak_rss_mb(), rss_before)

    rows = []
    for idx, observed in enumerate(series):
        values = {"observed": max(0.0, float(observed))}
        fit = outcome.fitted[idx]
        if fit is not None:
            values["fitted"] = float(fit)
        rows.append((sim_id, idx, Json(values)))

    for step, predicted in enumerate(outcome.forecast):
        rows.append((sim_id, len(series) + step, Json({"predicted": float(predicted)})))

    with conn.transaction():
        with conn.cursor() as cur:
            cur.executemany(
                "INSERT INTO simulation_results (simulation_id, t, compartments) VALUES (%s, %s, %s)",
                rows,
            )
            cur.execute(
```

```
                "INSERT INTO benchmarks (simulation_id, wall_time_ms, cpu_time_ms, peak_memory_mb) VALUES (%s, %s, %s, %s)",
                (sim_id, wall_ms, cpu_ms, peak_mb),
            )
            cur.execute(
                "UPDATE simulation_runs SET status = 'completed', completed_at = NOW() WHERE id = %s",
                (sim_id,),
            )

    print(
        f"lstm sim {sim_id} done: wall={wall_ms}ms cpu={cpu_ms}ms mem={peak_mb:.1f}MB "
        f"rows={len(rows)} train_end={outcome.train_end}/{len(series)} "
        f"MAE={outcome.mae:.2f} RMSE={outcome.rmse:.2f} nRMSE={outcome.nrmse:.3e}",
        flush=True,
    )


def mark_failed(conn: psycopg.Connection, sim_id: str, msg: str) -> None:
    try:
        with conn.transaction():
            conn.execute(
                "UPDATE simulation_runs SET status = 'failed', error_message = %s, completed_at = NOW() WHERE id = %s",
                (msg[:2000], sim_id),
            )
    except Exception as exc:  # noqa: BLE001
        print(f"could not mark failed: {exc}", file=sys.stderr, flush=True)


def main() -> int:
    signal.signal(signal.SIGINT, _handle_signal)
    signal.signal(signal.SIGTERM, _handle_signal)

    redis_url = os.environ["REDIS_URL"]
    db_dsn = os.environ.get("DATABASE_URL")

    rdb = redis.from_url(redis_url)
    rdb.ping()
    print(f"runner-lstm connected to redis, consuming {QUEUE}", flush=True)

    while _running:
        try:
            item = rdb.brpop(QUEUE, timeout=BRPOP_TIMEOUT_SEC)
        except redis.RedisError as exc:
            print(f"redis error: {exc}", file=sys.stderr, flush=True)
            time.sleep(ERROR_BACKOFF_SEC)
            continue
        if item is None:
            continue

        _, raw = item
        try:
            job = json.loads(raw)
        except json.JSONDecodeError as exc:
            print(f"invalid job json: {exc}", file=sys.stderr, flush=True)
            continue

        sim_id = job.get("simulation_id", "?")
        dsn = job.get("db_dsn") or db_dsn
        if not dsn:
            print(f"sim {sim_id}: no db_dsn", file=sys.stderr, flush=True)
            continue

        try:
            if job.get("model_type") != "lstm":
                raise RuntimeError(f"runner-lstm received unsupported model_type: {job.get('model_type')}")
            with psycopg.connect(dsn, row_factory=dict_row) as conn:
                run_lstm(conn, job)
        except Exception as exc:  # noqa: BLE001
            print(f"sim {sim_id} failed: {exc}", file=sys.stderr, flush=True)
            try:
                with psycopg.connect(dsn) as conn:
                    mark_failed(conn, sim_id, str(exc))
            except Exception as inner:  # noqa: BLE001
                print(f"could not connect to mark failed: {inner}", file=sys.stderr, flush=True)

```

```
    print("runner-lstm stopped", flush=True)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Listing D.1: Runner-ul LSTM dedicat predicțiilor epidemiologice.

Anexa E
Schema Bazei de Date

Anexa include migrațiile SQL care definesc schema relațională folosită de API și runner-e. Primele migrații creează structura inițială; cele intermediare adaugă studiile, membership-ul, flag-ul de super-admin, legătura `study_id` pentru resursele epidemiologice și extinderile pentru modelele `lstm` și `script`; ultimele introduc tabelele auxiliare (runner-e plug-in, draft-uri de script) și îmbogățirea schemei de date epidemiologice cu metrici suplimentare (decese, spitalizări, ATI, vaccinări, teste, indici de răspuns sanitar).

```sql
CREATE TABLE IF NOT EXISTS users (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email         VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_users_email ON users(email);
```

Listing E.1: Crearea tabelului `users`.

```sql
CREATE TABLE IF NOT EXISTS datasets (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id       UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    name          VARCHAR(255) NOT NULL,
    disease_code  VARCHAR(64) NOT NULL,
    uploaded_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    record_count  INTEGER NOT NULL DEFAULT 0
);

CREATE INDEX IF NOT EXISTS idx_datasets_user_id ON datasets(user_id);
CREATE INDEX IF NOT EXISTS idx_datasets_disease_code ON datasets(disease_code);
```

Listing E.2: Crearea tabelului `datasets`.

```sql
CREATE TABLE IF NOT EXISTS epidemiological_records (
    id            UUID DEFAULT gen_random_uuid(),
    dataset_id    UUID NOT NULL REFERENCES datasets(id) ON DELETE CASCADE,
    recorded_date DATE NOT NULL,
    region_code   VARCHAR(10) NOT NULL,
    susceptible   BIGINT,
    infected      BIGINT NOT NULL,
    recovered     BIGINT,
    population    BIGINT NOT NULL,
    PRIMARY KEY (dataset_id, recorded_date, id),
    CHECK (infected >= 0),
    CHECK (population > 0),
    CHECK (recovered IS NULL OR recovered >= 0),
    CHECK (susceptible IS NULL OR susceptible >= 0)
);

CREATE INDEX IF NOT EXISTS idx_epi_records_region ON epidemiological_records(region_code);
```

Listing E.3: Crearea tabelului `epidemiological_records`.

```sql
CREATE TABLE IF NOT EXISTS simulation_runs (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id       UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    dataset_id    UUID NOT NULL REFERENCES datasets(id) ON DELETE CASCADE,
    model_type    VARCHAR(16) NOT NULL DEFAULT 'sidarthe',
    language      VARCHAR(16) NOT NULL,
    parameters    JSONB NOT NULL,
```

```
    population_n  BIGINT NOT NULL,
    t_max         INTEGER NOT NULL,
    status        VARCHAR(16) NOT NULL DEFAULT 'pending',
    error_message TEXT,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at  TIMESTAMPTZ,
    CHECK (model_type IN ('sis', 'sir', 'seir', 'sirs', 'sidarthe')),
    CHECK (language IN ('r', 'julia', 'python')),
    CHECK (status IN ('pending', 'running', 'completed', 'failed')),
    CHECK (jsonb_typeof(parameters) = 'object'),
    CHECK (population_n > 0),
    CHECK (t_max > 0)
);

CREATE INDEX IF NOT EXISTS idx_sim_runs_user_id ON simulation_runs(user_id);
CREATE INDEX IF NOT EXISTS idx_sim_runs_dataset_id ON simulation_runs(dataset_id);
CREATE INDEX IF NOT EXISTS idx_sim_runs_status ON simulation_runs(status);
CREATE INDEX IF NOT EXISTS idx_sim_runs_language ON simulation_runs(language);
CREATE INDEX IF NOT EXISTS idx_sim_runs_model_type ON simulation_runs(model_type);
```

Listing E.4: Crearea tabelului `simulation_runs`.

```
CREATE TABLE IF NOT EXISTS simulation_results (
    simulation_id UUID NOT NULL REFERENCES simulation_runs(id) ON DELETE CASCADE,
    t             INTEGER NOT NULL,
    compartments  JSONB NOT NULL,
    PRIMARY KEY (simulation_id, t),
    CHECK (t >= 0),
    CHECK (jsonb_typeof(compartments) = 'object')
);
```

Listing E.5: Crearea tabelului `simulation_results`.

```
CREATE TABLE IF NOT EXISTS benchmarks (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    simulation_id   UUID NOT NULL REFERENCES simulation_runs(id) ON DELETE CASCADE,
    wall_time_ms    BIGINT NOT NULL,
    cpu_time_ms     BIGINT NOT NULL,
    peak_memory_mb  REAL NOT NULL,
    CHECK (wall_time_ms >= 0),
    CHECK (cpu_time_ms >= 0),
    CHECK (peak_memory_mb >= 0)
);

CREATE INDEX IF NOT EXISTS idx_benchmarks_simulation_id ON benchmarks(simulation_id);
```

Listing E.6: Crearea tabelului `benchmarks`.

```
CREATE TABLE IF NOT EXISTS studies (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name          VARCHAR(255) NOT NULL,
    disease_code  VARCHAR(64) NOT NULL,
    description   TEXT,
    created_by    UUID NOT NULL REFERENCES users(id),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_studies_disease_code ON studies(disease_code);
CREATE INDEX IF NOT EXISTS idx_studies_created_by ON studies(created_by);
```

Listing E.7: Crearea tabelului `studies`.

```
CREATE TABLE IF NOT EXISTS study_members (
    study_id   UUID NOT NULL REFERENCES studies(id) ON DELETE CASCADE,
    user_id    UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    role       VARCHAR(32) NOT NULL,
    joined_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (study_id, user_id),
    CHECK (role IN ('admin', 'researcher', 'contributor', 'viewer'))
);

CREATE INDEX IF NOT EXISTS idx_study_members_user_id ON study_members(user_id);
```

```sql
CREATE INDEX IF NOT EXISTS idx_study_members_role ON study_members(role);
```

Listing E.8: Crearea tabelului `study_members`.

```sql
-- Adauga campul is_super_admin pe users si promoveaza primul user existent
-- (cel mai vechi created_at) la super admin daca nu exista inca niciunul.
-- Idempotent: la rulari repetate nu modifica nimic daca exista deja un super admin.
ALTER TABLE users ADD COLUMN IF NOT EXISTS is_super_admin BOOLEAN NOT NULL DEFAULT FALSE;

UPDATE users
SET is_super_admin = TRUE
WHERE id = (
    SELECT id FROM users ORDER BY created_at ASC LIMIT 1
)
AND NOT EXISTS (
    SELECT 1 FROM users WHERE is_super_admin = TRUE
);
```

Listing E.9: Adăugarea flag-ului `is_super_admin`.

```sql
-- Adauga study_id la dataseturi si simulari. NOT NULL pentru consistenta --
-- presupunem ca nu exista date pre-existente (cluster fresh) sau ca un studiu
-- default e creat manual inainte de migrare in mediile cu date.
ALTER TABLE datasets
    ADD COLUMN IF NOT EXISTS study_id UUID NOT NULL REFERENCES studies(id) ON DELETE CASCADE;

ALTER TABLE simulation_runs
    ADD COLUMN IF NOT EXISTS study_id UUID NOT NULL REFERENCES studies(id) ON DELETE CASCADE;

CREATE INDEX IF NOT EXISTS idx_datasets_study_id ON datasets(study_id);
CREATE INDEX IF NOT EXISTS idx_simulation_runs_study_id ON simulation_runs(study_id);
```

Listing E.10: Adăugarea coloanei `study_id` pe dataseturi și simulări.

```sql
ALTER TABLE simulation_runs
    DROP CONSTRAINT IF EXISTS simulation_runs_model_type_check;

UPDATE simulation_runs
SET model_type = 'sidarthe'
WHERE model_type = 'sidarth';

-- Include si valorile adaugate de migratiile ulterioare (000012: 'lstm',
-- 000015: 'script') -- altfel re-rularea job-ului pe un DB matur esueaza pe
-- randurile existente cu acele valori. Comportamentul final (dupa 000015)
-- ramane identic.
ALTER TABLE simulation_runs
    ADD CONSTRAINT simulation_runs_model_type_check
    CHECK (model_type IN ('sis', 'sir', 'seir', 'sirs', 'sidarthe', 'lstm', 'script'));
```

Listing E.11: Extinderea constrângerii `model_type` pentru toate modelele suportate.

```sql
ALTER TABLE simulation_runs
    DROP CONSTRAINT IF EXISTS simulation_runs_model_type_check;

UPDATE simulation_runs
SET model_type = 'sidarthe'
WHERE model_type = 'sidarth';

-- Include si 'script' (adaugat de 000015), ca re-rularea pe DB matur sa nu
-- esueze. Comportamentul final dupa 000015 ramane identic.
ALTER TABLE simulation_runs
    ADD CONSTRAINT simulation_runs_model_type_check
    CHECK (model_type IN ('sis', 'sir', 'seir', 'sirs', 'sidarthe', 'lstm', 'script'));
```

Listing E.12: Adăugarea modelului `lstm` în constrângerea `model_type`.

```sql
-- Tabel pentru runner-ele de algoritmi definiţi de utilizator (plug-in-uri).
-- Fiecare runner este scope-uit la un singur studiu şi poate fi creat doar de
-- un super-admin. Scriptul (entry.py) este stocat ca text in baza de date.
-- Runner-ul trece prin starile: pending -> building -> ready -> stopped/failed.

CREATE TABLE IF NOT EXISTS runners (
```

```
    id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    study_id         UUID NOT NULL REFERENCES studies(id) ON DELETE CASCADE,
    uploaded_by      UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    name             VARCHAR(128) NOT NULL,
    description      TEXT NOT NULL DEFAULT '',
    script_content   TEXT NOT NULL,
    parameter_schema JSONB NOT NULL DEFAULT '{}'::jsonb,
    status           VARCHAR(32) NOT NULL DEFAULT 'pending',
    status_message   TEXT NOT NULL DEFAULT '',
    image_tag        VARCHAR(255) NOT NULL DEFAULT '',
    queue_name       VARCHAR(128) NOT NULL,
    deployment_name  VARCHAR(128) NOT NULL DEFAULT '',
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    ready_at         TIMESTAMPTZ,
    CHECK (status IN ('pending', 'building', 'ready', 'failed', 'stopped'))
);

CREATE INDEX IF NOT EXISTS idx_runners_study_id ON runners(study_id);
CREATE INDEX IF NOT EXISTS idx_runners_status ON runners(status);
CREATE UNIQUE INDEX IF NOT EXISTS idx_runners_study_name ON runners(study_id, name);
```

Listing E.13: Crearea tabelului `runners` pentru runner-ele plug-in.

```
-- Adauga modul "image" pentru runner-ele plug-in: in loc sa incarce un script
-- Python (procesat de wrapper-ul runner-custom), super-adminul poate da direct
-- o referinta catre o imagine container construita extern. Cele doua moduri
-- sunt mutual exclusive si toate runner-ele existente raman valide (script_mode).
--
-- NOTA: migratia 000015 ulterior dropeaza coloana script_content (pivotul:
-- runner-ele in mod script sunt eliminate, ramane doar mod image). Pentru ca
-- re-rularea job-ului de migrare pe un DB matur (post-15) sa nu esueze pe
-- referinte la script_content, incadram fiecare comanda intr-un bloc care o
-- executa doar daca coloana mai exista.

DO $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM information_schema.columns
        WHERE table_name='runners' AND column_name='script_content'
    ) THEN
        ALTER TABLE runners ALTER COLUMN script_content DROP NOT NULL;
    END IF;
END $$;

ALTER TABLE runners
    ADD COLUMN IF NOT EXISTS image VARCHAR(255) NOT NULL DEFAULT '';

-- XOR: exact unul dintre script_content (non-empty) sau image (non-empty)
-- trebuie sa fie prezent. Asta permite migrarea celor existente (toate au
-- script_content non-empty si image='') si validarea celor noi. Pe DB matur
-- (post-15) coloana nu mai exista; constrangerea finala e instalata de 000015.
DO $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM information_schema.columns
        WHERE table_name='runners' AND column_name='script_content'
    ) AND NOT EXISTS (
        SELECT 1 FROM pg_constraint WHERE conname='runners_source_check'
    ) THEN
        ALTER TABLE runners
            ADD CONSTRAINT runners_source_check CHECK (
                (script_content IS NOT NULL AND script_content <> '' AND image = '')
                OR
                (image <> '' AND (script_content IS NULL OR script_content = ''))
            );
    END IF;
END $$;
```

Listing E.14: Pivot către modul `image` pentru runner-e (DDL idempotent: condițional pe existența coloanei `script_content`, dropped ulterior).

```
-- Pivot: scripturile nu mai sunt "runner-e" persistente, ci o forma de
-- simulare ad-hoc care ruleaza pe un executor partajat. Tabela runners ramane
-- strict pentru runner-ele persistente bazate pe imagine externa.
```

```
4
5  -- 1) Curatare runners: pastram doar runner-ele in mod image. Eventualele
6  --    runner-e in mod script sunt sterse (Deployment-urile lor au fost
7  --    deja oprite de testele anterioare).
8  DELETE FROM runners WHERE script_content IS NOT NULL AND image = '';
9
10 ALTER TABLE runners DROP CONSTRAINT IF EXISTS runners_source_check;
11 ALTER TABLE runners DROP COLUMN IF EXISTS script_content;
12
13 -- Acum image este strict obligatoriu.
14 ALTER TABLE runners ADD CONSTRAINT runners_image_required CHECK (image <> '');
15
16 -- 2) Extinderea simulation_runs cu script_content (nullable, doar pentru
17 --    simularile de tip 'script'). Plus extinderea CHECK-ului pe model_type.
18 ALTER TABLE simulation_runs ADD COLUMN IF NOT EXISTS script_content TEXT;
19
20 ALTER TABLE simulation_runs DROP CONSTRAINT IF EXISTS simulation_runs_model_type_check;
21 ALTER TABLE simulation_runs ADD CONSTRAINT simulation_runs_model_type_check
22     CHECK (model_type IN ('sis', 'sir', 'seir', 'sirs', 'sidarthe', 'lstm', 'script'));
23
24 -- Coerenta: pentru model_type='script' script_content trebuie sa fie non-vid;
25 -- pentru celelalte trebuie sa fie NULL (sau lipsa).
26 ALTER TABLE simulation_runs ADD CONSTRAINT simulation_runs_script_content_check
27     CHECK (
28         (model_type = 'script' AND script_content IS NOT NULL AND script_content <> '')
29         OR
30         (model_type <> 'script' AND script_content IS NULL)
31     );
```

Listing E.15: Simulări de tip `script`: `script_content` mutat în `simulation_runs`, runner-ele devin strict *image-based*.

```
1  -- Pentru simularile de tip 'script', t_max este metadata optionala (scriptul
2  -- isi alege orizontul singur). Pentru celelalte tipuri ramane obligatorie.
3  ALTER TABLE simulation_runs DROP CONSTRAINT IF EXISTS simulation_runs_t_max_check;
4  ALTER TABLE simulation_runs ADD CONSTRAINT simulation_runs_t_max_check
5      CHECK ((model_type = 'script') OR (t_max > 0));
```

Listing E.16: Relaxarea constrângerii `t_max` pentru simulări de tip `script` (scriptul își alege singur orizontul).

```
1  -- Draft-uri de cod pentru editorul de scripturi din UI. Fiecare cercetator are
2  -- propriile draft-uri per studiu si nu vede draft-urile altor utilizatori.
3  -- Continutul este pastrat ca text in DB (scripturi <1 MB tipic); pentru scale-up
4  -- la sute de MB per draft, datele pot fi mutate la stocare obiect (S3/MinIO),
5  -- mentinand metadatele in tabela.
6
7  CREATE TABLE IF NOT EXISTS script_drafts (
8      id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
9      user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
10     study_id        UUID NOT NULL REFERENCES studies(id) ON DELETE CASCADE,
11     name            VARCHAR(128) NOT NULL,
12     description     TEXT NOT NULL DEFAULT '',
13     script_content  TEXT NOT NULL,
14     parameters_json JSONB NOT NULL DEFAULT '{}'::jsonb,
15     created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
16     updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
17 );
18
19 CREATE INDEX IF NOT EXISTS idx_script_drafts_user_study ON script_drafts(user_id, study_id);
20 CREATE UNIQUE INDEX IF NOT EXISTS idx_script_drafts_user_study_name ON script_drafts(user_id, study_id,
   ↪ name);
```

Listing E.17: Crearea tabelului `script_drafts` pentru persistența draft-urilor din editor.

```
1  -- Extindere schema epidemiological_records cu coloane suplimentare folosite
2  -- frecvent in literatura si in seturile OWID/WHO/ECDC, dincolo de cele 4
3  -- coloane SIR-centrice initiale (susceptible/infected/recovered/population).
4  -- Toate sunt nullable pentru ca nu orice dataset le include simultan.
5  -- Runner-ii built-in (SIR/SIDARTHE/LSTM) le ignora silentios; scripturile
6  -- definite de utilizator au acces la toate prin records[i] in main().
7
8  ALTER TABLE epidemiological_records
```

```
    ADD COLUMN IF NOT EXISTS deaths             BIGINT,    -- decese cumulate
    ADD COLUMN IF NOT EXISTS new_cases          BIGINT,    -- cazuri noi (interval = granularitatea
↪ dataset-ului)
    ADD COLUMN IF NOT EXISTS new_deaths         BIGINT,    -- decese noi (interval ca mai sus)
    ADD COLUMN IF NOT EXISTS hospitalized       BIGINT,    -- pacienti spitalizati la moment
    ADD COLUMN IF NOT EXISTS icu                BIGINT,    -- pacienti in terapie intensiva la moment
    ADD COLUMN IF NOT EXISTS vaccinated         BIGINT,    -- persoane vaccinate (cel putin o doza) cumulat
    ADD COLUMN IF NOT EXISTS tested             BIGINT,    -- teste cumulate
    ADD COLUMN IF NOT EXISTS stringency_index   REAL,      -- Oxford Stringency Index, 0..100
    ADD COLUMN IF NOT EXISTS reproduction_rate  REAL;      -- Rt observat (cand este disponibil)
```

Listing E.18: Extinderea schemei `epidemiological_records` cu metrici suplimentare (decese, spitalizări, ATI, vaccinări, teste, indicele Oxford de stringență, R_t observat).

Anexa F
Configurații Kubernetes

Această anexă include manifestele declarative pentru rularea sistemului pe un cluster Kubernetes. Toate resursele sunt aplicate în namespace-ul `epimodel` și urmează structura arhitecturală descrisă în capitolul 4: API stateless, două frontend-uri servite prin Nginx, runnere scalabile independent, Redis pentru coada de mesaje și YugabyteDB pentru persistență (3 master + 3 tserver, RF=3).

Manifestele se află în directorul `code/k8s/` al repository-ului și sunt organizate cu prefixe numerice pentru aplicare secvențială. Table F.1 sintetizează conținutul directorului.

Listările care urmează reproduc fișierele cele mai relevante pentru explicitarea arhitecturii: namespace-ul de izolare, fișierul de `values` pentru YugabyteDB, Service-ul de acces YSQL, coada Redis, Job-ul de migrații, API-ul, un exemplu de runner (Julia, cel mai mare consumator de memorie), runner-ul LSTM dedicat, frontend-ul admin și ingress-ul de rutare. Celelalte manifeste — ConfigMap-ul, template-ul Secret, frontend-ul principal, runner-ele R și Python — urmează aceleași convenții și pot fi consultate direct în repository.

Namespace dedicat

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: epimodel
  labels:
    app.kubernetes.io/part-of: epimodel
    app.kubernetes.io/managed-by: kubectl
```

Listing F.1: `00-namespace.yaml` — Namespace-ul dedicat aplicației.

YugabyteDB — valori Helm

YugabyteDB rulează ca două `StatefulSet`-uri (`yb-master` și `yb-tserver`) cu câte 3 replici, generate de Helm chart-ul oficial. Fișierul de `values` suprascrie sizing-ul implicit pentru a se potrivi resurselor cluster-ului (workeri 4 vCPU / 8 GiB) și dezactivează componentele neutilizate (`yugabytedUi`, `enableLoadBalancer`).

```yaml
# Helm values pentru YugabyteDB pe clusterul kube-4kdhb
# Profil "PoC pentru disertatie": RF=3, dar requests mici pentru workeri 2vCPU/4GiB.
#
# Instalare:
#   helm install yugabyte yugabytedb/yugabyte \
#     --namespace epimodel \
#     --create-namespace=false \
#     -f code/k8s/10-yugabyte-values.yaml \
#     --wait --timeout 10m
#
# Verifica:
#   kubectl -n epimodel get pods,svc,pvc
#   kubectl -n epimodel exec -it yb-tserver-0 -- ysqlsh -h yb-tserver-service -U yugabyte -c "\l"
#
# Deinstalare (atenţie, şterge PVC-urile):
#   helm uninstall yugabyte -n epimodel
```

Tabela F.1: Manifestele Kubernetes incluse în repository.

Fișier	**Resurse principale**	**Rol**
`00-namespace.yaml`	`Namespace`	Izolează aplicația într-un namespace dedicat
`01-configmap.yaml`	`ConfigMap`	Variabile non-sensibile (HTTP, Redis, JWT)
`02-secret.example.yaml`	`Secret` (template)	Șablon pentru `JWT_SECRET` și `DATABASE_URL`
`10-yugabyte-values.yaml`	Helm values	Configurarea chart-ului oficial `yugabytedb/yugabyte` (RF=3, sizing custom)
`12-yugabyte-tserver-service.yaml`	`Service`	`ClusterIP` `yb-tserver-service` expus pe portul 5433 (YSQL)
`20-redis.yaml`	`Deployment` + `Service`	Coadă Redis (`emptyDir`, fără persistență)
`30-migrate-job.yaml`	`Job`	Aplică migrațiile SQL prin `psql`
`40-api.yaml`	`Deployment` + `Service`	Backend Go (REST API + RBAC pe studii)
`50-runner-python.yaml`	`Deployment`	Runner Python (`scipy.solve_ivp`)
`51-runner-r.yaml`	`Deployment`	Runner R (`deSolve::ode`)
`52-runner-julia.yaml`	`Deployment`	Runner Julia (`DifferentialEquations.jl`)
`53-runner-lstm.yaml`	`Deployment`	Runner LSTM dedicat pentru predicții pe seria `infected`
`60-frontend.yaml`	`Deployment` + `Service`	Frontend user-facing servit prin Nginx
`61-frontend-admin.yaml`	`Deployment` + `Service`	Consolă super-admin servită prin Nginx
`70-ingress.yaml`	`Ingress`	Rutare HTTP pentru `sir-workbench.ro` și `admin.sir-workbench.ro`

```
#   kubectl -n epimodel delete pvc -l app.kubernetes.io/name=yb-master
#   kubectl -n epimodel delete pvc -l app.kubernetes.io/name=yb-tserver

Image:
  tag: "2024.1.3.0-b105"   # versiune stabilă LTS (2025.2.x e foarte recentă)
  pullPolicy: IfNotPresent

storage:
  ephemeral: false
  master:
    count: 1               # 1 PV per master (default e 2)
    size: 5Gi
    storageClass: "block-ceph"
  tserver:
    count: 1               # 1 PV per tserver
    size: 10Gi
    storageClass: "block-ceph"

resource:
  # Workeri 4 vCPU / 8 GiB: tserver primeste 2 GiB request / 3 GiB limit pentru
  # cache decent. Master ramane mai mic (1 GiB) -- tine metadata, nu date.
  master:
    requests:
      cpu: "500m"
      memory: 1Gi
    limits:
      cpu: "1"
      memory: 1.5Gi
  tserver:
    requests:
      cpu: "1"
      memory: 2Gi
    limits:
      cpu: "2"
      memory: 3Gi

replicas:
  master: 3
  tserver: 3

# Octavia nu functioneaza pe acest cluster, asa ca folosim ClusterIP peste tot.
enableLoadBalancer: false

# Disable UI yugabyted (consuma resurse, deschide ports pe care nu le expunem).
yugabytedUi:
  enabled: false

# Personalizam gflags pentru cluster mic.
gflags:
  master:
    default_memory_limit_to_ram_ratio: "0.6"
  tserver:
    default_memory_limit_to_ram_ratio: "0.6"
    ysql_num_shards_per_tserver: "2"

# Endpoints expuse -- toate ClusterIP, fara LoadBalancer.
serviceEndpoints:
  - name: "yb-master-ui"
    type: ClusterIP
    app: "yb-master"
    ports:
      http-ui: "7000"

  - name: "yb-tserver-service"
    type: ClusterIP
    app: "yb-tserver"
    ports:
      tcp-yql-port: "9042"
      tcp-ysql-port: "5433"

# Distribuim pod-urile pe noduri diferite (anti-affinity soft default in chart).
# Pe 3 workeri vrem cate un master si cate un tserver pe fiecare.
masterAffinity: |
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
```

```
        podAffinityTerm:
          labelSelector:
            matchLabels:
              app: "yb-master"
          topologyKey: kubernetes.io/hostname

tserverAffinity: |
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchLabels:
              app: "yb-tserver"
          topologyKey: kubernetes.io/hostname
```

Listing F.2: `10-yugabyte-values.yaml` — Valorile Helm pentru YugabyteDB.

Service de acces la YugabyteDB

Chart-ul oficial generează două Service-uri *headless* (`yb-masters` și `yb-tservers`) pentru DNS-ul intern necesar protocolului Raft, dar nu un Service `ClusterIP` clasic pe care îl pot folosi consumatorii YSQL. Acest Service este adăugat printr-un manifest separat.

```
apiVersion: v1
kind: Service
metadata:
  name: yb-tserver-service
  namespace: epimodel
  labels:
    app: yb-tserver
    app.kubernetes.io/part-of: epimodel
spec:
  type: ClusterIP
  selector:
    app: yb-tserver
  ports:
    - name: tcp-yql-port
      port: 9042
      targetPort: 9042
    - name: tcp-ysql-port
      port: 5433
      targetPort: 5433
```

Listing F.3: `12-yugabyte-tserver-service.yaml` — `ClusterIP` `yb-tserver-service` (port 5433, YSQL).

Redis

Redis este deployat ca `Deployment` cu o singură replică și volum efemer (`emptyDir`). Cozile de mesaje nu sunt persistente între restarturi, ceea ce este acceptabil pentru job-uri de simulare care durează secunde sau minute. Pentru retenție durabilă, structura poate fi convertită într-un `StatefulSet` cu `PersistentVolumeClaim` fără modificări la nivelul aplicației.

```
apiVersion: v1
kind: Service
metadata:
  name: redis
  namespace: epimodel
  labels:
    app.kubernetes.io/name: redis
    app.kubernetes.io/part-of: epimodel
spec:
  type: ClusterIP
  ports:
    - name: redis
```

```yaml
    port: 6379
    targetPort: 6379
  selector:
    app.kubernetes.io/name: redis
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  namespace: epimodel
  labels:
    app.kubernetes.io/name: redis
    app.kubernetes.io/part-of: epimodel
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app.kubernetes.io/name: redis
  template:
    metadata:
      labels:
        app.kubernetes.io/name: redis
        app.kubernetes.io/part-of: epimodel
    spec:
      terminationGracePeriodSeconds: 30
      volumes:
        - name: data
          emptyDir:
            sizeLimit: 512Mi
      containers:
        - name: redis
          image: redis:7-alpine
          imagePullPolicy: IfNotPresent
          # Fara persistenta -- folosim emptyDir pentru a economisi un volum OpenStack
          # (quota tenant 10 vol, prioritizam Yugabyte RF=3). Cozile se pierd la restart Pod,
          # dar simularile dureaza secunde, nu zile.
          args:
            - redis-server
            - --save
            - ""
            - --appendonly
            - "no"
          ports:
            - name: redis
              containerPort: 6379
          volumeMounts:
            - name: data
              mountPath: /data
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              cpu: 200m
              memory: 256Mi
          readinessProbe:
            exec:
              command: ["redis-cli", "ping"]
            initialDelaySeconds: 5
            periodSeconds: 5
            timeoutSeconds: 3
          livenessProbe:
            exec:
              command: ["redis-cli", "ping"]
            initialDelaySeconds: 15
            periodSeconds: 15
            timeoutSeconds: 5
```

Listing F.4: `20-redis.yaml` — Redis cu volum efemer.

Job-ul de migrare

Migrațiile SQL sunt aplicate printr-un `Job` care folosește clientul `psql` (în loc de `golang-migrate`) pentru a evita dependența de *advisory locks* PostgreSQL, pe care YugabyteDB nu le implementează încă. Job-ul iterează fișierele `*.up.sql` în ordine alfabetică și le aplică cu `ON_ERROR_STOP=1`.

```
# Job care aplica migraţiile SQL pe YugabyteDB.
# Migraţiile sunt încărcate dintr-un ConfigMap generat din ./db/migrations/.
#
# Înainte de a aplica acest Job, generează ConfigMap-ul:
#
#   kubectl create configmap db-migrations \
#     --namespace=epimodel \
#     --from-file=code/db/migrations/
#
# YugabyteDB nu suportă (încă) advisory locks pe care golang-migrate le foloseşte,
# aşa că rulăm migraţiile direct cu psql, în ordine alfabetică a fişierelor *.up.sql.
# Job-ul foloseste IF NOT EXISTS în migraţii pentru idempotenţă.
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
  namespace: epimodel
  labels:
    app.kubernetes.io/name: db-migrate
    app.kubernetes.io/part-of: epimodel
spec:
  backoffLimit: 3
  ttlSecondsAfterFinished: 600
  template:
    metadata:
      labels:
        app.kubernetes.io/name: db-migrate
        app.kubernetes.io/part-of: epimodel
    spec:
      restartPolicy: OnFailure
      initContainers:
        - name: wait-for-ysql
          image: postgres:16-alpine
          env:
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: epimodel-secrets
                  key: DATABASE_URL
          command:
            - /bin/sh
            - -c
            - |
              until pg_isready -d "$DATABASE_URL"; do
                echo "waiting for YSQL ..."
                sleep 3
              done
              echo "YSQL is ready"
      containers:
        - name: migrate
          image: postgres:16-alpine
          imagePullPolicy: IfNotPresent
          env:
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: epimodel-secrets
                  key: DATABASE_URL
          command:
            - /bin/sh
            - -c
            - |
              set -e
              echo "Applying migrations in order..."
              for f in $(ls /migrations/*.up.sql | sort); do
                echo ""
                echo "=== Applying $f ==="
```

```
              psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f "$f"
            done
            echo ""
            echo "All migrations applied successfully."
            psql "$DATABASE_URL" -c "\dt"
        volumeMounts:
          - name: migrations
            mountPath: /migrations
        resources:
          requests:
            cpu: 50m
            memory: 32Mi
          limits:
            cpu: 200m
            memory: 128Mi
    volumes:
      - name: migrations
        configMap:
          name: db-migrations
```

Listing F.5: `30-migrate-job.yaml` — Job care aplică migrațiile prin `psql`.

API Go

API-ul rulează ca `Deployment` cu două replici pentru toleranță la pierderea unui pod. Configurația non-sensibilă provine din `ConfigMap`, iar parolele și secretele din `Secret`. Probele HTTP pe `/api/health` verifică atât pod-ul, cât și conectivitatea efectivă către baza de date și Redis.

```
apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: epimodel
  labels:
    app.kubernetes.io/name: api
    app.kubernetes.io/part-of: epimodel
spec:
  type: ClusterIP
  ports:
    - name: http
      port: 8080
      targetPort: 8080
  selector:
    app.kubernetes.io/name: api
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: epimodel
  labels:
    app.kubernetes.io/name: api
    app.kubernetes.io/part-of: epimodel
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: api
  template:
    metadata:
      labels:
        app.kubernetes.io/name: api
        app.kubernetes.io/part-of: epimodel
    spec:
      serviceAccountName: api
```

```
      terminationGracePeriodSeconds: 30
      imagePullSecrets:
        - name: ghcr-pull
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              podAffinityTerm:
                labelSelector:
                  matchLabels:
                    app.kubernetes.io/name: api
                topologyKey: kubernetes.io/hostname
      containers:
        - name: api
          image: ghcr.io/masterprogramercpu/emedicine-api:latest
          imagePullPolicy: Always
          envFrom:
            - configMapRef:
                name: epimodel-config
            - secretRef:
                name: epimodel-secrets
          env:
            # Activeaza provisioning-ul K8s pentru runner-ele plug-in.
            - name: RUNNER_DEPLOY_ENABLED
              value: "true"
          ports:
            - name: http
              containerPort: 8080
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
            limits:
              cpu: 500m
              memory: 256Mi
          readinessProbe:
            httpGet:
              path: /api/health
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
            timeoutSeconds: 3
          livenessProbe:
            httpGet:
              path: /api/health
              port: 8080
            initialDelaySeconds: 30
            periodSeconds: 30
            timeoutSeconds: 5
```

Listing F.6: `40-api.yaml` — Deployment și Service pentru API-ul Go.

Runner Julia

Runner-ul Julia este `Deployment`-ul cu cele mai generoase limite de resurse (512 MiB request, 2 GiB limit) din cauza costului inițial de încărcare al pachetelor `DifferentialEquations.jl`. Runner-ele R și Python au structuri identice, dar cu limite mai mici.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: runner-julia
  namespace: epimodel
  labels:
    app.kubernetes.io/name: runner-julia
    app.kubernetes.io/component: runner
    app.kubernetes.io/part-of: epimodel
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
```

```
    matchLabels:
      app.kubernetes.io/name: runner-julia
  template:
    metadata:
      labels:
        app.kubernetes.io/name: runner-julia
        app.kubernetes.io/component: runner
        app.kubernetes.io/part-of: epimodel
    spec:
      terminationGracePeriodSeconds: 60
      imagePullSecrets:
        - name: ghcr-pull
      containers:
        - name: runner-julia
          image: ghcr.io/masterprogramercpu/emedicine-runner-julia:latest
          imagePullPolicy: Always
          env:
            - name: REDIS_URL
              valueFrom:
                configMapKeyRef:
                  name: epimodel-config
                  key: REDIS_URL
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: epimodel-secrets
                  key: DATABASE_URL
          resources:
            requests:
              cpu: 200m
              memory: 384Mi
            limits:
              cpu: "1"
              memory: 1Gi
```

Listing F.7: `52-runner-julia.yaml` — Deployment pentru runner-ul Julia.

Runner LSTM

Runner-ul LSTM este separat de runner-ul Python pentru simulări compartimentale. El consumă exclusiv coada `epimodel:jobs:lstm`, antrenează modelul recurent pe seria `infected` și persistă separat valorile observate, potrivite și prezise.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: runner-lstm
  namespace: epimodel
  labels:
    app.kubernetes.io/name: runner-lstm
    app.kubernetes.io/component: runner
    app.kubernetes.io/part-of: epimodel
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app.kubernetes.io/name: runner-lstm
  template:
    metadata:
      labels:
        app.kubernetes.io/name: runner-lstm
        app.kubernetes.io/component: runner
        app.kubernetes.io/part-of: epimodel
    spec:
      terminationGracePeriodSeconds: 60
      imagePullSecrets:
        - name: ghcr-pull
      containers:
        - name: runner-lstm
          image: ghcr.io/masterprogramercpu/emedicine-runner-lstm:latest
```

```
          imagePullPolicy: Always
          env:
            - name: REDIS_URL
              valueFrom:
                configMapKeyRef:
                  name: epimodel-config
                  key: REDIS_URL
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: epimodel-secrets
                  key: DATABASE_URL
          resources:
            requests:
              cpu: 300m
              memory: 512Mi
            limits:
              cpu: "2"
              memory: 2Gi
```

Listing F.8: `53-runner-lstm.yaml` — Deployment pentru runner-ul LSTM.

Frontend admin

Consola super-admin rulează ca un `Deployment` separat față de aplicația principală, deși ambele folosesc același API. Această separare produce imagini Docker distincte și permite rutarea prin host-ul `admin.sir-workbench.ro`.

```
apiVersion: v1
kind: Service
metadata:
  name: frontend-admin
  namespace: epimodel
  labels:
    app.kubernetes.io/name: frontend-admin
    app.kubernetes.io/part-of: epimodel
spec:
  type: ClusterIP
  ports:
    - name: http
      port: 80
      targetPort: 80
  selector:
    app.kubernetes.io/name: frontend-admin
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-admin
  namespace: epimodel
  labels:
    app.kubernetes.io/name: frontend-admin
    app.kubernetes.io/part-of: epimodel
spec:
  replicas: 1
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: frontend-admin
  template:
    metadata:
      labels:
        app.kubernetes.io/name: frontend-admin
        app.kubernetes.io/part-of: epimodel
    spec:
      terminationGracePeriodSeconds: 30
      imagePullSecrets:
        - name: ghcr-pull
```

```
      containers:
        - name: frontend-admin
          image: ghcr.io/masterprogramercpu/emedicine-frontend-admin:latest
          imagePullPolicy: Always
          ports:
            - name: http
              containerPort: 80
          resources:
            requests:
              cpu: 50m
              memory: 32Mi
            limits:
              cpu: 200m
              memory: 128Mi
          readinessProbe:
            httpGet:
              path: /healthz
              port: 80
            initialDelaySeconds: 3
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /healthz
              port: 80
            initialDelaySeconds: 15
            periodSeconds: 30
```

Listing F.9: `61-frontend-admin.yaml` — Deployment și Service pentru consola super-admin.

Ingress de rutare

Ingress-ul nginx rutează cererile externe pe două host-uri: `sir-workbench.ro` pentru aplicația principală și `admin.sir-workbench.ro` pentru consola super-admin. Pe ambele host-uri, prefixul `/api` este trimis către API-ul Go; restul traficului ajunge la frontend-ul corespunzător. Într-un deployment cu DNS public, host-urile se înlocuiesc cu FQDN-uri reale și se adaugă o secțiune `tls:` cu certificate gestionate prin `cert-manager`.

```
# Ingress nginx pentru rutarea traficului HTTP extern.
# Presupune că nginx-ingress controller este deja instalat în cluster
# (https://kubernetes.github.io/ingress-nginx/deploy/).
#
# Pentru testare locală, mapează în /etc/hosts:
#   127.0.0.1  sir-workbench.ro
#   127.0.0.1  admin.sir-workbench.ro
#
# Pentru producţie, înlocuieşte host-urile cu FQDN-uri reale şi adaugă o
# secţiune ‘tls:‘ cu un Secret de tip kubernetes.io/tls (cert-manager etc.).
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: epimodel-ingress
  namespace: epimodel
  labels:
    app.kubernetes.io/part-of: epimodel
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "64m"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "120"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "120"
spec:
  ingressClassName: nginx
  rules:
    # Host principal -- aplicaţia user-facing + API
    - host: sir-workbench.ro
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api
                port:
```

```
                number: 8080
        - path: /
          pathType: Prefix
          backend:
            service:
              name: frontend
              port:
                number: 80
  # Host dedicat pentru consola super-admin
  - host: admin.sir-workbench.ro
    http:
      paths:
        # API-ul rămâne accesibil şi de pe sub-domain (apelurile XHR
        # din admin SPA folosesc /api/... relativ la admin.sir-workbench.ro).
        - path: /api
          pathType: Prefix
          backend:
            service:
              name: api
              port:
                number: 8080
        - path: /
          pathType: Prefix
          backend:
            service:
              name: frontend-admin
              port:
                number: 80
```

Listing F.10: `70-ingress.yaml` — Ingress nginx pentru rutare HTTP.

Alternativă pentru dezvoltare locală

În paralel cu manifestele Kubernetes, repository-ul include și o configurație Docker Compose care reproduce aceeași topologie pe o singură mașină, fără nevoia unui cluster. Aceasta este utilă pentru dezvoltare locală și pentru verificări rapide ale fluxurilor end-to-end fără costul operațional al unui cluster K8s real.

```
services:
  yugabyte:
    image: yugabytedb/yugabyte:2.21.1.0-b271
    container_name: emedicine-yugabyte
    command:
      - bin/yugabyted
      - start
      - --background=false
      - --tserver_flags=ysql_num_shards_per_tserver=4
    ports:
      - "5433:5433"
      - "7000:7000"
      - "9000:9000"
    volumes:
      - yugabyte-data:/root/var
    healthcheck:
      test: ["CMD-SHELL", "bin/ysqlsh -h yugabyte -p 5433 -U yugabyte -c 'SELECT 1' >/dev/null 2>&1 || exit
↪ 1"]
      interval: 10s
      timeout: 5s
      retries: 12
      start_period: 30s

  redis:
    image: redis:7-alpine
    container_name: emedicine-redis
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10
```

```
  migrate:
    image: migrate/migrate:v4.17.1
    container_name: emedicine-migrate
    depends_on:
      yugabyte:
        condition: service_healthy
    volumes:
      - ./db/migrations:/migrations
    entrypoint:
      - migrate
      - -path=/migrations
      - -database=postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable
      - up
    restart: "no"

  api:
    build:
      context: ./api
      dockerfile: Dockerfile
    container_name: emedicine-api
    depends_on:
      migrate:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    environment:
      HTTP_ADDR: ":8080"
      DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable"
      REDIS_URL: "redis://redis:6379/0"
      JWT_SECRET: "${JWT_SECRET:-dev-secret-change-me}"
      RUNNER_DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable"
    ports:
      - "8080:8080"
    restart: unless-stopped

  runner-python:
    build:
      context: ./runners/python
      dockerfile: Dockerfile
    container_name: emedicine-runner-python
    depends_on:
      migrate:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    environment:
      REDIS_URL: "redis://redis:6379/0"
      DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable"
    restart: unless-stopped

  runner-lstm:
    build:
      context: ./runners/lstm
      dockerfile: Dockerfile
    container_name: emedicine-runner-lstm
    depends_on:
      migrate:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    environment:
      REDIS_URL: "redis://redis:6379/0"
      DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable"
    restart: unless-stopped

  runner-r:
    build:
      context: ./runners/r
      dockerfile: Dockerfile
    container_name: emedicine-runner-r
    depends_on:
      migrate:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
```

```
    environment:
      REDIS_URL: "redis://redis:6379/0"
      DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable"
    restart: unless-stopped

  runner-julia:
    build:
      context: ./runners/julia
      dockerfile: Dockerfile
    container_name: emedicine-runner-julia
    depends_on:
      migrate:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    environment:
      REDIS_URL: "redis://redis:6379/0"
      DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable"
    restart: unless-stopped

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: emedicine-frontend
    depends_on:
      - api
    ports:
      - "5173:80"
    restart: unless-stopped

volumes:
  yugabyte-data:
```

Listing F.11: `docker-compose.yml` — Configurația echivalentă pentru mediul de dezvoltare local.

Anexa G
Documentația API REST

API-ul REST este implementat în pachetul `internal/api` al backend-ului Go. Rutele sunt definite centralizat în routerul `chi`, iar handler-ele sunt separate pe domenii: autentificare, profil utilizator, studii, membri, dataseturi, simulări, benchmarkuri, super-admin și health check. Specificația tabelară a endpointurilor se află în Table 4.2; în această anexă sunt incluse fișierele relevante pentru definirea rutelor și a formatului comun de răspuns.

```go
package router

import (
    "net/http"
    "time"

    "github.com/go-chi/chi/v5"
    "github.com/go-chi/chi/v5/middleware"
    "github.com/go-chi/cors"

    "github.com/tudorgeana/emedicine-api/internal/api/handlers"
    authmw "github.com/tudorgeana/emedicine-api/internal/api/middleware"
    "github.com/tudorgeana/emedicine-api/internal/models"
)

type Deps struct {
    Health       *handlers.HealthHandler
    Auth         *handlers.AuthHandler
    Users        *handlers.UsersHandler
    Studies      *handlers.StudiesHandler
    Datasets     *handlers.DatasetsHandler
    Simulations  *handlers.SimulationsHandler
    Benchmarks   *handlers.BenchmarksHandler
    Admin        *handlers.AdminHandler
    Runners      *handlers.RunnersHandler
    ScriptDrafts *handlers.ScriptDraftsHandler
    Map          *handlers.MapHandler
    JWT          *authmw.JWTMiddleware
    RBAC         *authmw.RBAC
}

func New(deps Deps) http.Handler {
    r := chi.NewRouter()

    r.Use(middleware.RequestID)
    r.Use(middleware.RealIP)
    r.Use(middleware.Logger)
    r.Use(middleware.Recoverer)
    r.Use(middleware.Timeout(60 * time.Second))
    r.Use(cors.Handler(cors.Options{
        AllowedOrigins:   []string{"*"},
        AllowedMethods:   []string{"GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"},
        AllowedHeaders:   []string{"Accept", "Authorization", "Content-Type"},
        AllowCredentials: false,
        MaxAge:           300,
    }))

    r.Route("/api", func(r chi.Router) {
        r.Get("/health", deps.Health.Get)

        r.Route("/auth", func(r chi.Router) {
            r.Post("/register", deps.Auth.Register)
            r.Post("/login", deps.Auth.Login)
        })

```

```
        // Toate rutele de mai jos cer autentificare.
        r.Group(func(r chi.Router) {
            r.Use(deps.JWT.Authenticate)

            // Profilul userului curent
            r.Get("/me", deps.Users.Me)
            r.Get("/me/profile", deps.Users.MyProfile)
            r.Patch("/me/profile", deps.Users.UpdateMyProfile)
            r.Put("/me/avatar", deps.Users.SetMyAvatar)
            r.Delete("/me/avatar", deps.Users.DeleteMyAvatar)
            r.Get("/me/collaborators", deps.Users.MyCollaborators)

            // Profilul altor utilizatori (vizibil colaboratorilor)
            r.Get("/users/{userId}/profile", deps.Users.GetUserProfile)
            r.Get("/users/{userId}/avatar", deps.Users.GetUserAvatar)

            // Studii -- list / create sunt scope pe userul curent
            r.Route("/studies", func(r chi.Router) {
                r.Get("/", deps.Studies.List)
                r.Post("/", deps.Studies.Create)

                // Rute care lucreaza intr-un studiu specific
                r.Route("/{studyId}", func(r chi.Router) {
                    // Membership minim (orice rol) -- citire
                    r.Group(func(r chi.Router) {
                        r.Use(deps.RBAC.RequireStudyMembership())

                        r.Get("/", deps.Studies.Get)
                        r.Get("/members", deps.Studies.ListMembers)

                        // Datasets -- citire pentru toti membrii
                        r.Get("/datasets", deps.Datasets.List)
                        r.Get("/datasets/{id}", deps.Datasets.Get)

                        // Simulations -- citire pentru toti membrii
                        r.Get("/simulations", deps.Simulations.List)
                        r.Get("/simulations/{id}", deps.Simulations.Get)
                        r.Get("/simulations/{id}/results", deps.Simulations.Results)
                        r.Get("/simulations/{id}/export", deps.Simulations.Export)

                        // Benchmarks -- citire pentru toti membrii
                        r.Get("/benchmarks", deps.Benchmarks.Aggregate)
                        r.Get("/benchmarks/{model}/{language}", deps.Benchmarks.RunsForCombination)

                        // Harta studiului: observat + simulat per tara
                        r.Get("/map", deps.Map.Get)

                        // Runners (plug-in-uri) -- citire pentru toti membrii
                        r.Get("/runners", deps.Runners.List)
                        r.Get("/runners/{id}", deps.Runners.Get)

                        // Script drafts -- fiecare membru isi vede doar draft-urile
                        // proprii (filtrare la nivel de repo dupa user_id).
                        r.Get("/script-drafts", deps.ScriptDrafts.List)
                        r.Get("/script-drafts/{id}", deps.ScriptDrafts.Get)
                        r.Post("/script-drafts", deps.ScriptDrafts.Create)
                        r.Patch("/script-drafts/{id}", deps.ScriptDrafts.Update)
                        r.Delete("/script-drafts/{id}", deps.ScriptDrafts.Delete)
                    })

                    // Runners -- creare/stergere: doar super-admin.
                    // Lant: RequireStudyMembership (super-admin trece + seteaza
                    // study context) -> RequireSuperAdmin (filtreaza membrii normali).
                    r.Group(func(r chi.Router) {
                        r.Use(deps.RBAC.RequireStudyMembership())
                        r.Use(deps.RBAC.RequireSuperAdmin)
                        r.Post("/runners", deps.Runners.Create)
                        r.Delete("/runners/{id}", deps.Runners.Delete)
                    })

                    // Datasets -- scriere: admin sau contributor
                    r.Group(func(r chi.Router) {
                        r.Use(deps.RBAC.RequireStudyMembership(models.RoleAdmin, models.RoleContributor))
                        r.Post("/datasets", deps.Datasets.Create)
                        r.Delete("/datasets/{id}", deps.Datasets.Delete)
                    })
```

```
132
133                     // Simulations -- scriere: admin sau researcher
134                     r.Group(func(r chi.Router) {
135                         r.Use(deps.RBAC.RequireStudyMembership(models.RoleAdmin, models.RoleResearcher))
136                         r.Post("/simulations", deps.Simulations.Create)
137                     })
138
139                     // Admin studiu -- doar rolul admin
140                     r.Group(func(r chi.Router) {
141                         r.Use(deps.RBAC.RequireStudyMembership(models.RoleAdmin))
142                         r.Patch("/", deps.Studies.Update)
143                         r.Delete("/", deps.Studies.Delete)
144                         r.Post("/members", deps.Studies.AddMember)
145                         r.Patch("/members/{userId}", deps.Studies.UpdateMemberRole)
146                         r.Delete("/members/{userId}", deps.Studies.RemoveMember)
147                     })
148                 })
149             })
150
151             // Endpoint-uri super admin
152             r.Route("/admin", func(r chi.Router) {
153                 r.Use(deps.RBAC.RequireSuperAdmin)
154                 r.Get("/stats", deps.Admin.Stats)
155                 r.Get("/users", deps.Admin.ListUsers)
156                 r.Patch("/users/{userId}/super-admin", deps.Admin.SetSuperAdmin)
157                 r.Get("/studies", deps.Admin.ListStudies)
158                 r.Get("/breakdown", deps.Admin.Breakdown)
159                 r.Get("/recent-simulations", deps.Admin.RecentSimulations)
160                 r.Get("/recent-datasets", deps.Admin.RecentDatasets)
161             })
162         })
163     })
164
165     return r
166 }
```

Listing G.1: Routerul REST al API-ului Go.

```
1  package middleware
2
3  import (
4      "context"
5      "errors"
6      "net/http"
7
8      "github.com/go-chi/chi/v5"
9      "github.com/google/uuid"
10
11     "github.com/tudorgeana/emedicine-api/internal/db/repository"
12     "github.com/tudorgeana/emedicine-api/internal/models"
13 )
14
15 const (
16     studyIDKey   ctxKey = "study_id"
17     studyRoleKey ctxKey = "study_role"
18 )
19
20 // RBAC contine dependentele necesare pentru middleware-urile de autorizare.
21 type RBAC struct {
22     Users        *repository.UserRepository
23     StudyMembers *repository.StudyMemberRepository
24 }
25
26 func NewRBAC(users *repository.UserRepository, members *repository.StudyMemberRepository) *RBAC {
27     return &RBAC{Users: users, StudyMembers: members}
28 }
29
30 // RequireSuperAdmin permite trecerea doar daca userul curent are is_super_admin=true.
31 func (rb *RBAC) RequireSuperAdmin(next http.Handler) http.Handler {
32     return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
33         uid, ok := UserIDFrom(r.Context())
34         if !ok {
35             writeAuthError(w, "missing user context")
36             return
37         }
38         user, err := rb.Users.GetByID(r.Context(), uid)
```

```
        if err != nil || user == nil {
            writeAuthError(w, "user lookup failed")
            return
        }
        if !user.IsSuperAdmin {
            writeForbidden(w, "super admin required")
            return
        }
        next.ServeHTTP(w, r)
    })
}

// RequireStudyMembership verifica daca userul are unul din rolurile permise in studiul
// identificat de URL param "studyId". Super-adminul trece intotdeauna.
// Daca allowedRoles e gol, orice rol valid trece (doar membership-ul conteaza).
func (rb *RBAC) RequireStudyMembership(allowedRoles ...models.StudyRole) func(http.Handler) http.Handler {
    allowed := make(map[models.StudyRole]struct{}, len(allowedRoles))
    for _, r := range allowedRoles {
        allowed[r] = struct{}{}
    }

    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            uid, ok := UserIDFrom(r.Context())
            if !ok {
                writeAuthError(w, "missing user context")
                return
            }

            studyIDStr := chi.URLParam(r, "studyId")
            studyID, err := uuid.Parse(studyIDStr)
            if err != nil {
                writeBadRequest(w, "invalid studyId")
                return
            }

            // Super-adminul are acces direct, fara verificare de membership.
            user, err := rb.Users.GetByID(r.Context(), uid)
            if err != nil {
                writeAuthError(w, "user lookup failed")
                return
            }

            ctx := context.WithValue(r.Context(), studyIDKey, studyID)

            if user.IsSuperAdmin {
                ctx = context.WithValue(ctx, studyRoleKey, models.RoleAdmin)
                next.ServeHTTP(w, r.WithContext(ctx))
                return
            }

            role, err := rb.StudyMembers.GetRole(r.Context(), studyID, uid)
            if errors.Is(err, repository.ErrNotFound) {
                writeForbidden(w, "not a member of this study")
                return
            }
            if err != nil {
                writeForbidden(w, "membership lookup failed")
                return
            }

            if len(allowed) > 0 {
                if _, ok := allowed[role]; !ok {
                    writeForbidden(w, "insufficient role")
                    return
                }
            }

            ctx = context.WithValue(ctx, studyRoleKey, role)
            next.ServeHTTP(w, r.WithContext(ctx))
        })
    }
}

// StudyIDFrom returneaza ID-ul studiului curent din context.
func StudyIDFrom(ctx context.Context) (uuid.UUID, bool) {
```

```
    v := ctx.Value(studyIDKey)
    id, ok := v.(uuid.UUID)
    return id, ok
}

// StudyRoleFrom returneaza rolul userului curent in studiul din context.
func StudyRoleFrom(ctx context.Context) (models.StudyRole, bool) {
    v := ctx.Value(studyRoleKey)
    r, ok := v.(models.StudyRole)
    return r, ok
}

func writeForbidden(w http.ResponseWriter, msg string) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusForbidden)
    _, _ = w.Write([]byte(`{"error":{"code":403,"message":"` + msg + `"}}`))
}

func writeBadRequest(w http.ResponseWriter, msg string) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusBadRequest)
    _, _ = w.Write([]byte(`{"error":{"code":400,"message":"` + msg + `"}}`))
}
```

Listing G.2: Middleware-ul RBAC pentru studii și super-admin.

```
package handlers

import (
    "encoding/json"
    "net/http"
)

type envelope struct {
    Data  any         `json:"data,omitempty"`
    Error *errPayload `json:"error,omitempty"`
}

type errPayload struct {
    Code    int    `json:"code"`
    Message string `json:"message"`
}

func writeJSON(w http.ResponseWriter, status int, data any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    _ = json.NewEncoder(w).Encode(envelope{Data: data})
}

func writeError(w http.ResponseWriter, status int, message string) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    _ = json.NewEncoder(w).Encode(envelope{Error: &errPayload{Code: status, Message: message}})
}

func decodeJSON(r *http.Request, dst any) error {
    dec := json.NewDecoder(r.Body)
    dec.DisallowUnknownFields()
    return dec.Decode(dst)
}
```

Listing G.3: Formatul comun al răspunsurilor JSON.